

Chaos labs

Theory, evidence and controlled experiments

Read the relevant theory, predict, measure, explain, and check recovery. Each lab tests one question. The solutions are collected at the end so you can study and run the question without seeing its expected observations.

Run `./lab.sh` provision once. Choose any lab: `./lab.sh NN` setup prepares and verifies its fixture in the shared VM. Open `./lab.sh ssh`, run the procedure and record your evidence. Compare NN solution. Save results before NN reset removes that lab's resources and files.

Stop if the baseline fails. Confirm the fault actually reached its target. Expected results are not your observations. Use Bash without `set -e` and preserve the same shell when blocks share variables. Keep a second VM terminal available for recovery.

All labs use one VM. Each Kubernetes lab owns a separate named kind cluster inside it. No lab sequence is required. Reset finished labs to free resources. `./lab.sh stop` halts the whole VM while preserving files. See README.md for capacity.

Long command lines wrap visually in this print edition. Copy exact commands from docs/chaos-labs.md or the terminal question card. Theory citations use original section numbers in docs/chaos-theory.md.

Lab	Topic	Lab	Topic
00	Prepare the learning environment	19	Unscheduled Pods versus memory-killed containers
01	A timeout makes a silent failure manageable	20	Trace DNS, Service ports and the application listener
02	Prove why a process was killed	21	Configuration freshness and a stalled rollout
03	Restart policies have limits	22	Orphan reaping and the container process budget
04	CPU contention and a quota	23	Endpoint removal, preStop and the grace budget
05	A container is several Linux mechanisms	24	Search paths, ndots and a resolver outage
06	Separate containers can share a full filesystem	25	Admission control as a synchronous dependency
07	Network delay accumulates across sequential calls	26	Container privilege boundaries and hardening
08	A syscall error tests application handling	27	Reading a cluster against the CIS benchmark
09	A syscall denial need not crash the process	28	Pod Security admission and what a namespace label blocks
10	Ownership, disruption budgets and Service routing	29	What a Pod's ServiceAccount can reach
11	Health depends on what you test	30	What a Secret protects, and what it does not
12	Locate startup failures and measure recovery	31	Default deny, and what a policy does not stop
13	Placement, supervision and replacement	32	Grants that reach further than they name
14	Quorum, convergence and application availability	33	When probes cause or repair restarts
15	Count how layered retries multiply work	34	Volume binding, placement and retained data
16	Design one clear experiment	35	Namespace budgets and admission failures
17	Signal delivery and the shutdown budget	36	Secret encryption and key-loss recovery
18	Persistence, ownership and read-only mounts		

Lab 00 - Prepare the learning environment

Which checks establish a usable baseline before you inject a fault?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 00 setup` on the host. No other lab is required.

Theory you need

A chaos experiment compares a measured outcome before, during and after one controlled fault. Observability means you can measure that outcome; a baseline is its normal value or range. If the no-fault check fails, stop: a later failure would not tell you whether the injection caused it.

The method is: choose a measurement, establish normal behaviour, predict a result under a specified fault, run the comparison, and explain the evidence. For example, “the client still responds within one second when its cache is unavailable” is testable; “the service is resilient” is not.

Blast radius is everything the experiment can affect. These labs target named processes, containers or objects inside a disposable Linux VM. Namespaces separate resource views; cgroup v2 accounts for and limits resource use. The VM still shares CPU, memory and a kernel across labs.

Record versions so another run can be compared with yours. Keep evidence in this lab's workspace. Reset deletes that workspace; shared tooling and other labs remain available. Each later experiment must still measure its own service baseline and recovery.

Source: docs/chaos-theory.md: §§1.3, 2.1, 2.5; Appendix A.

Main lesson to learn in this lab

A meaningful fault experiment starts with a working, measured baseline and a specific prediction. Record the environment, bound what the fault can affect, and compare the same outcome before, during and after it. Passing these environment checks only prepares the tools: each later lab still needs its own service baseline and recovery evidence before you can draw a conclusion.

Experiment

Check the shared Ubuntu VM and save a baseline record inside Lab 00. No fault is injected.

`record_versions` saves the source commit, installed tool versions and cached image identities to `versions-images.txt`. The baseline checks remain visible in the steps.

Before running: If a prerequisite already fails, can a later failure tell you anything about the injected fault?

Measurement key

Signal	Meaning
<code>cgroup2fs / cgroups=2</code>	The kernel and Docker use the cgroup v2 interface used by these labs.
<code>hello-world</code>	A successful run checks image access, container creation and process execution together.
<code>df -h /</code>	Size is total filesystem capacity; Avail is unused space. The shared provisioner requires at least 60 GiB total.
<code>versions / baseline.txt</code>	Record the actual versions and baseline checks, not simply “setup passed”.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 00 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh` ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Check the kernel, cgroup, Docker and disk baseline

Run in: VM terminal 1

```
source ~/labs/lab00/exercise.sh
cd ~/labs/lab00
set -o pipefail
{
  uname -srmo
  stat -fc %T /sys/fs/cgroup
  docker info --format 'driver={{.CgroupDriver}} cgroups={{.CgroupVersion}}'
  docker run --rm --name ce-lab00-check hello-world
  df -h /
} 2>&1 | tee checks.txt
```

Record: Check table: each observed value, and pass or action needed. checks.txt keeps the terminal output.

Step 2. Record the source commit, versions and image digests

Run in: VM terminal 1

```
record_versions
```

Record: Versions / baseline record row: the source commit, tool versions, and each image tag, local ID and registry digest.

Step 3. Save the baseline record

Run in: VM terminal 1

```
cd ~/labs/lab00
{ date --iso-8601=seconds; cat checks.txt versions-images.txt; } > baseline.txt
cat baseline.txt
```

Record: Versions / baseline record row: the path `~/labs/lab00/baseline.txt`, and that it holds the hello-world result and the versions.

Recovery check: Prerequisite checks pass and baseline.txt records them.

Write your answer

Use the observations recorded beside each step.

Check	Observed value	Pass or action needed
cgroup / Docker		
Container execution		
Disk capacity		
Versions / baseline record		

- 1. Which baseline checks passed, and what value did each one report?
- 2. Where is your baseline record, and which environment identities does it hold?
- 3. Why must you fix a failed baseline before injecting a fault?

Apply the same reasoning

Why must Lab 01 check Redis again even when hello-world succeeds?

Save results before removing this lab's resources and files: `./lab.sh 00 reset`

Lab 01 - A timeout makes a silent failure manageable

Why does the handler run on refusal but not on a silent drop?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 01` setup on the host. No other lab is required.

Theory you need

A dependency call must return or raise an error before execution can reach the next statement or an exception handler. `try/except` supplies a reaction to an error; it does not give the call a deadline.

REJECT with a TCP reset gives the client a prompt refusal. DROP silently discards matching packets. Without a reply the network stack retries and waits; its eventual failure can be much later than a useful application deadline. These are two different failure modes of the same dependency.

A connect timeout bounds connection establishment. A read timeout bounds waiting for data after connecting. The prepared client disables retries and sets both to TIMEOUT when supplied. A socket timeout is not a general end-to-end deadline across many operations or retries.

The five-second timeout command is an external watchdog: it terminates the test process. It does not execute the client's fallback. Compare that termination with the client catching its own timeout, then remove the firewall rule and check the same request again.

Source: docs/chaos-theory.md: §1.5 and Experiment Card 1.1.

Main lesson to learn in this lab

An exception handler can react only after a dependency call returns or raises an error; it cannot end a silent wait. Refusal fails promptly, while dropped packets require an application timeout to make fallback useful. Bound connection and read waits, then account for all operations and retries in the total deadline: killing the client with an external watchdog is not application recovery.

Experiment

One `ce-lab01` firewall rule targets TCP 127.0.0.1:6381. All fault runs use a five-second watchdog; `TIMEOUT=0.5` sets the client timeouts. The comparison runs in `ce-lab01-runner.service` so reset can stop it before removing its firewall rules.

`run_fault` installs the named fault, runs the client with a five-second watchdog, prints its exit status and rule counters, then removes the rule. `check_firewall` reports whether removal succeeded.

Before running: Predict which of the three attempts reaches the handler: REJECT, DROP, and DROP with a 0.5-second timeout.

Measurement key

Signal	Meaning
OK / DEGRADED	OK means Redis answered. DEGRADED means the client caught a Redis error and executed its fallback.
elapsed	Seconds measured inside the client. A watchdog-killed, buffered process may print no elapsed line.
exit=124	The external timeout command reached its deadline. This is not the application's handled error.
iptables pkts	A nonzero counter confirms traffic matched the fault rule; no matches means you have not tested that fault.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 01 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Check the Redis baseline

Run in: VM terminal 1

```
source ~/labs/lab01/exercise.sh
cd ~/labs/lab01
redis-cli -p 6381 ping
env -u TIMEOUT timeout 5s python3 client.py
echo "client exit=$?"
```

Record: Baseline row: the response, the elapsed time and the client exit status.

Step 2. Case 1 - REJECT without an application timeout

Run in: VM terminal 1

```
run_fault reject
```

Record: REJECT row: handler message, elapsed time, client exit and matched packets.

Step 3. Case 2 - DROP without an application timeout

Run in: VM terminal 1

```
run_fault drop
```

Record: DROP row: whether a handler or elapsed line appeared, client exit, seconds waited and matched packets.

Step 4. Case 3 - DROP with a 0.5-second application timeout

Run in: VM terminal 1

```
run_fault drop-timeout
```

Record: DROP + 0.5 s timeout row: handler message, elapsed time, client exit and matched packets.

Step 5. Prove Redis and firewall recovery

Run in: VM terminal 1

```
redis-cli -p 6381 ping
env -u TIMEOUT timeout 5s python3 client.py
echo "client exit=$?"
check_firewall
```

Record: Recovered row: the response, elapsed time, exit status, and whether any `ce-lab01` rule remains.

Recovery check: The client succeeds again and no `ce-lab01` rule remains.

Only if normal recovery fails

Step 1. Manual rule removal, only if `run.sh` was killed

Run in: VM terminal 2

```
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j DROP
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j REJECT --reject-with
tcp-reset
env -u TIMEOUT timeout 5s python3 ~/labs/lab01/client.py
echo "client exit=$?"
```

Write your answer

Use the observations recorded beside each step.

Attempt	Handler ran?	Elapsed / exit	Rule packets	What ended the wait?
Baseline				
REJECT				
DROP				
DROP + 0.5 s timeout				
Recovered				

1. What proves the firewall fault was removed and the client works again?
2. What ended the wait in each of the three cases, and with which exit status?
3. Why does try/except need an application timeout to handle a silent dependency?

Apply the same reasoning

With three attempts of 0.5 s each, is the whole request bounded by 0.5 s?

Save results before removing this lab's resources and files: `./lab.sh 01 reset`

Lab 02 - Prove why a process was killed

Why is exit status 137 insufficient to diagnose an OOM kill?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 02 setup` on the host. No other lab is required.

Theory you need

A signal is a notification sent to a process. SIGTERM (15) can be handled for cleanup; SIGKILL (9) cannot be caught. Bash commonly reports signal termination as 128 + signal number, so a SIGKILL gives 137. A program may also explicitly exit with 137: status alone is not a diagnosis.

The kernel can invoke the out-of-memory (OOM) killer when it cannot satisfy memory demand. A cgroup has its own memory boundary: its processes can hit MemoryMax even while the rest of the VM has available memory. The allocator here repeatedly creates and touches bytes, so it demands real memory rather than merely reserving virtual addresses.

MemoryMax=128M and MemorySwapMax=0 constrain this experiment. RuntimeMaxSec=20s is a separate time bound. An OOM kill, that timer, and a deliberate SIGKILL can all stop the process; identify which happened by matching the unit, PID and time in the logs.

The systemd-run command reports the transient unit's outcome; its shell status need not equal the child's Bash status. Collect the unit result and kernel evidence. systemd-oomd is a userspace memory-pressure killer, so an oomd action is a different cause from a kernel OOM kill.

Source: docs/chaos-theory.md: §2.3 (exit codes, signals and OOM), §5.5.2 (memory limits).

Main lesson to learn in this lab

An exit code describes an outcome, not its cause. Exit 137 is consistent with SIGKILL, but cannot distinguish a deliberate kill from memory exhaustion or another cause; a cgroup can run out of memory while the VM still has available memory. Diagnose termination by matching the PID, time, unit result and kernel or userspace-killer logs before deciding what to repair.

Experiment

The allocator runs only in ce-lab02, with 128 MiB memory, no swap and a 20-second runtime bound.

Before running: Will a deliberate SIGKILL and a memory-limit kill have different exit statuses? Which evidence can distinguish them?

Measurement key

Signal	Meaning
deliberate KILL exit	Read \$? immediately after the process. An intervening command would replace it.
journalctl -u ce-lab02	Look for the recorded termination signal and result: oom-kill and timeout describe different causes.
journalctl -k	Match a memory-cgroup OOM / Killed process line to this allocator and experiment time.
journalctl -u systemd-oomd	A matching userspace-kill entry changes the diagnosis; unrelated historical entries do not.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 02 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Check the memory controller and unit state

Run in: VM terminal 1

```
cd ~/labs/lab02
cat /sys/fs/cgroup/cgroup.controllers
systemctl is-active ce-lab02.service; echo "unit status=$? (nonzero means not active)"
```

Record: Preconditions: the memory controller listing and the ce-lab02 unit state. Stop if either is wrong.

Step 2. Record a deliberate SIGKILL

Run in: VM terminal 1

```
python3 -c 'import os,signal; os.kill(os.getpid(),signal.SIGKILL)'
echo "deliberate KILL exit=$?"
```

Record: Known SIGKILL row: the printed exit status, and that the program signalled itself.

Step 3. Run the allocator within its memory and time limits

Run in: VM terminal 1

```
cd ~/labs/lab02
date --iso-8601=seconds > started-at.txt
sudo systemd-run --unit=ce-lab02 --wait --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p RuntimeMaxSec=20s \
  /usr/bin/python3 "$HOME/labs/lab02/allocate.py"
echo "systemd-run exit=$? (unit outcome, not the child's Bash status)"
date --iso-8601=seconds > ended-at.txt
```

Record: Allocator run: the printed systemd-run status and the start and end timestamps.

Step 4. Match the termination to the unit and kernel logs

Run in: VM terminal 1

```
cd ~/labs/lab02
since=$(cat started-at.txt)
cat started-at.txt ended-at.txt
sudo journalctl -u ce-lab02 --since "$since" --no-pager | tee unit.log
sudo journalctl -k --since "$since" --grep="oom|Killed process" --case-sensitive=no --no-pager | tee kernel.log
sudo journalctl -u systemd-oomd --since "$since" --no-pager | tee oomd.log
```

Record: Memory-limited allocator row: the PID, the termination result and the exact matching log lines.

Step 5. Stop the allocator if it remains active

Run in: VM terminal 1

```
sudo systemctl stop ce-lab02.service 2>/dev/null || true
systemctl is-active ce-lab02.service; echo "unit status=$? (nonzero means not active)"
```

Record: Final ce-lab02 unit state after stopping any leftover allocator.

Recovery check: The allocator is stopped and its termination evidence has been recorded.

Write your answer

Use the observations recorded beside each step.

Case	Termination	Matching cause evidence
Known SIGKILL		
Memory-limited allocator		

1. What exit status did the deliberate SIGKILL produce, and why is its cause certain?
2. What terminated the memory-limited allocator, and which log lines prove it?
3. Why can neither 137 nor the systemd-run status diagnose an OOM kill alone?

Apply the same reasoning

Does free memory elsewhere in the VM rule out a cgroup OOM kill?

Save results before removing this lab's resources and files: `./lab.sh 02 reset`

Lab 03 - Restart policies have limits

Why does one crash recover while a burst leaves the service failed?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 03 setup` on the host. No other lab is required.

Theory you need

A supervisor separates a service's desired state from its current process. Restart=always asks systemd to start a replacement after a crash. It is still subject to start-rate limiting; the request for a restart and permission to start are separate decisions.

This fixture explicitly sets `StartLimitIntervalSec=20` and `StartLimitBurst=4`. Starts consume the allowance, including initial or manual starts; successful uptime does not instantly erase it. A single kill after reset-failed can recover, whereas enough closely spaced starts exhaust the allowance and stop automatic recovery.

NGINX sends requests to A or B and can try another usable backend on a connection failure. If A gives up restarting, B may still answer. Measure A's state and client HTTP results independently: successful HTTP

through the proxy cannot identify which backend served it.

reset-failed clears the failed state and start counter; start requests recovery. Disabling the limit would permit more restart attempts, but would not fix the cause of the crashes. A passing single-crash experiment therefore says little about a repeated-crash condition.

Source: docs/chaos-theory.md: §§2.4-2.6 and Experiment Cards 2.1-2.2.

Main lesson to learn in this lab

Automatic restart is conditional: systemd can replace one crashed process yet stop trying when repeated starts exhaust its rate limit. A healthy second backend can keep proxy requests successful while the first remains failed. Check both the individual service and the client outcome; clearing the start counter restores the ability to retry, but does not fix why the service crashes.

Experiment

A and B serve through NGINX. The core experiment targets only A: Restart=always, at most four starts per 20 seconds. B stays available during this comparison.

Before running: Predict A's state after one kill and after six rapid kill attempts, with Restart=always unchanged.

Measurement key

Signal	Meaning
ActiveState / Result	ActiveState shows whether A is running or failed. Result and the journal explain a refused restart.
NRestarts	Counts automatic restarts; it is not the complete count of all starts used by the rate limiter.
Start request repeated too quickly	Evidence that the start-rate limit was reached. A later kill saying the unit is inactive is a consequence, not another successful injection.
HTTP response	A successful response tests the proxy path, not the health of both backends.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 03 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Check baseline and clear the previous start counter

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a
sudo systemctl start ce-lab03-a ce-lab03-b
systemctl is-active ce-lab03-a ce-lab03-b
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts -p Restart -p StartLimitBurst -p
StartLimitIntervalUSec
curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
```

Record: Baseline row: A's PID, state, result, restart count and the proxy HTTP status.

Step 2. Kill A once and inspect its recovery

Run in: VM terminal 1

```
sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a
sleep 1
curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
```

Record: One crash row: A's new PID, state and restart count, and the proxy HTTP status.

Step 3. Crash A repeatedly and inspect the refused restart

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a
burst_since=$(date --iso-8601=seconds)
for i in $(seq 1 6); do
  echo "crash attempt=$i"
  sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a || true
  sleep 1
done
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
sudo journalctl -u ce-lab03-a --since "$burst_since" --no-pager
curl -sS --max-time 3 -o /dev/null -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
```

Record: Crash burst row: A's final state, result, restart count, journal lines and proxy HTTP status.

Step 4. Recover and prove the proxy works again

Run in: VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a ce-lab03-b
sudo systemctl start ce-lab03-a ce-lab03-b
systemctl is-active ce-lab03-a ce-lab03-b
systemctl show ce-lab03-a -p MainPID -p ActiveState -p Result -p NRestarts
# The backends need a moment to bind, and NGINX retries a failed backend after fail_timeout=1s.
for i in $(seq 1 10); do curl -fsS --max-time 3 -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/ && break;
  sleep 0.5; done
```

Record: Recovered row: A's state, result and PID, B's state, and the proxy response.

Recovery check: Both backends are active and the proxy responds successfully.

Write your answer

Use the observations recorded beside each step.

Phase	A PID / state / result / restarts	HTTP result	Journal evidence
Baseline			
One crash			
Crash burst			
Recovered			

1. How did A's PID, state and restart count change after one crash?
2. What stopped A from restarting during the burst, and which journal line proves it?
3. Why does the proxy's HTTP result not establish A's health?
4. Which recovery commands restored A, and what proves both backends and the proxy recovered?

Apply the same reasoning

What would disabling the start limit prove, and what defect would remain?

Save results before removing this lab's resources and files: ./lab.sh 03 reset

Lab 04 - CPU contention and a quota

How does capping a CPU competitor change the time taken by the same job?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 04 setup` on the host. No other lab is required.

Theory you need

A CPU-bound job needs CPU execution time. Its wall time also includes waiting to be scheduled. Keeping the calculation fixed while adding another runnable job on the same CPU tests contention without changing the application's work.

USE means utilization, saturation and errors, checked per resource. CPU utilization says how busy the CPU is; saturation is runnable work waiting for CPU. High utilization alone does not prove a problem. Connect the resource measurements to a change in this job's elapsed time.

CPU placement and CPU bandwidth are different controls. `taskset` pins both jobs to the same allowed CPU. `CPUQuota=20%` caps the neighbour at one fifth of one CPU on average. It neither reserves a core nor promises the remaining time exclusively to the measured job.

In `cgroup v2`, `cpu.max` contains quota and period in microseconds. Divide quota by period to obtain the CPU budget, for example $20000 / 100000 = 0.2$. When the group uses that budget it waits until replenishment; rising throttling counters show enforcement. Compare repeated baseline timings, loaded timings, capped timings and recovery using the same calculation.

Source: docs/chaos-theory.md: §§3.2, 3.3.5 and the CPU-shares correction; §5.5.2.

Main lesson to learn in this lab

CPU contention increases elapsed time because runnable work must wait for a shared CPU. A quota caps CPU time per period; it neither reserves a core nor guarantees another job the remainder. Compare the same calculation across baseline, contention and recovery, and use rising throttling counters to confirm enforcement: judge the quota by its effect on the workload, not utilization alone.

Experiment

Select one allowed CPU automatically and pin both jobs to it. The neighbour runs in `ce-lab04` for at most 45 seconds. Each measurement uses the same six-iteration job.

`measure_baseline` runs the same job three times. `measure_quota` prints `cpu.max`, before/after counters and their increases, and rejects a measurement without a running neighbour. `compare_times` prints the recorded timings.

Before running: Rank the job time with no competitor, an unrestricted competitor and a competitor capped at 20%. Explain your prediction.

Measurement key

Signal	Meaning
work.py output	Column 1 is the iteration, column 2 the elapsed seconds. Mean is column 2 summed over the rows; range is its minimum to maximum. Use all samples in a phase, not one fast sample.
cpu.max / cpu.stat	cpu.max is configuration. nr_throttled counts throttled periods; throttled_usec is accumulated throttling time. Configuration alone does not show that a limit was reached.
CPU PSI / vmstat r	PSI reports time with runnable tasks waiting; r counts runnable tasks. Both are VM-wide supporting signals, not attribution to this one neighbour.
Neighbour active	If it ended before the job finished, the sample mixes loaded and recovered states and must be repeated.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 04 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Choose an allowed CPU

Run in: VM terminal 1

```
source ~/labs/lab04/exercise.sh
cd ~/labs/lab04
CPU=$(python3 -c 'import os; print(min(os.sched_getaffinity(0)))')
echo "Both workloads will use CPU $CPU"
systemctl is-active ce-lab04.service; echo "unit status=$? (nonzero means not active)"
```

Record: Preconditions: the chosen CPU number and the ce-lab04 unit state. Keep this shell; CPU stays defined.

Step 2. Record the baseline three times

Run in: VM terminal 1

```
measure_baseline
```

Record: Baseline row: the combined mean and range. Keep all three baseline files.

Step 3. Add a CPU neighbour on the same CPU

Run in: VM terminal 1

```
cd ~/labs/lab04
sudo systemd-run --unit=ce-lab04 --collect -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
systemctl is-active ce-lab04
taskset -c "$CPU" python3 work.py | tee loaded.txt
systemctl is-active ce-lab04 || echo 'Inconclusive: the neighbour ended before the measurement finished.'
cat /proc/pressure/cpu
vmstat 1 5
sudo systemctl stop ce-lab04
```

Record: Unrestricted neighbour row: the mean and range from loaded.txt, and the neighbour's state before and after.

Step 4. Cap the neighbour and read the throttling counters

Run in: VM terminal 1

```
wait_neighbour_stopped
sudo systemd-run --unit=ce-lab04 --collect -p CPUQuota=20% -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
measure_quota
sudo systemctl stop ce-lab04
```

Record: 20% neighbour row: the cpu.max ratio, both cpu.stat snapshots with their increases, and the neighbour's state.

Step 5. Recover and repeat the baseline

Run in: VM terminal 1

```
sudo systemctl stop ce-lab04
wait_neighbour_stopped
systemctl is-active ce-lab04
taskset -c "$CPU" python3 work.py | tee recovered.txt
compare_times
```

Record: Recovered row: the mean and range, and the neighbour's final state.

Recovery check: ce-lab04 is stopped and the final job timing is recorded.

Write your answer

Use the observations recorded beside each step.

Phase	Mean / range (s)	Neighbour active?	Throttle evidence
Baseline			
Unrestricted neighbour			
20% neighbour			
Recovered			

1. What were the mean and range of iteration times in each of the four phases?
2. Was the neighbour active throughout each loaded phase, and is any phase inconclusive?
3. What ratio did cpu.max show, and how much did the throttling counters rise?
4. Why did capping the competitor change the measured job's time?

Apply the same reasoning

Would pinning the neighbour to a different otherwise idle CPU test the same contention?

Save results before removing this lab's resources and files: `./lab.sh 04 reset`

Lab 05 - A container is several Linux mechanisms

Why does an isolated process view not reveal the sandbox's CPU budget?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 05 setup` on the host. No other lab is required.

Theory you need

A Linux container combines mechanisms; it is not a small machine with its own kernel. `chroot` changes where absolute filesystem paths begin. A mount namespace separates mount configuration. A PID namespace gives processes their own PID numbering and visibility.

`unshare --pid --fork` starts a child in a new PID namespace. Its first process is PID 1 there but has an ordinary PID visible from the VM. A fresh `proc` mount is needed for `ps` to display the new view; changing the filesystem root alone does not hide host processes.

Cgroups separately account for and constrain resource consumption. The `systemd` scope places the sandbox in a group with `CPUQuota=20%`, `MemoryMax=128M` and `TasksMax=50`. An isolated `ps` listing says nothing about those budgets.

A CPU quota is time per period: divide the two `cpu.max` numbers. Read `cpu.stat` before and during a busy loop to distinguish a configured cap from actual throttling. The sandbox still shares the VM kernel and, because no network namespace is requested, its network. Each isolation claim needs its own evidence.

Source: docs/chaos-theory.md: §§5.2-5.5.2 and §5.7.1.

Main lesson to learn in this lab

A container combines separate controls: filesystem roots and namespaces change what a process sees, while cgroups limit what it consumes. An isolated PID listing proves neither a CPU budget nor complete isolation; this sandbox still shares the VM kernel and network. Inspect each boundary separately, and distinguish a configured resource limit from evidence that the workload actually reached it.

Experiment

The prepared BusyBox sandbox uses `chroot`, PID and mount namespaces, and a `systemd` scope with CPU, memory and task limits.

In the host shell, `sandbox_limits` prints the active scope limits and PID namespaces. `sandbox_throttling` takes two `cpu.stat` readings five seconds apart and prints the counter increases.

Before running: Predict which observation shows a separate PID view and which shows enforced CPU limiting.

Measurement key

Signal	Meaning
<code>echo \$\$ / ps</code>	PID 1 and the small process list establish the sandbox’s PID view. The host can still see these processes.
<code>ls /</code>	Shows the exported BusyBox filesystem used as the sandbox’s root.
<code>cpu.max / memory.max / pids.max</code>	<code>cpu.max</code> gives quota and period in microseconds; $20000 / 100000 = 0.2$ of one CPU. <code>memory.max</code> is bytes; 128 MiB = 134217728 bytes. <code>pids.max</code> counts tasks, including threads.
<code>cpu.stat</code>	<code>nr_throttled</code> counts throttled periods and <code>throttled_usec</code> their microseconds. Take later minus earlier: an increase during the loop shows enforcement, an old total alone does not.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 05 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Confirm the exported BusyBox root filesystem

Run in: VM terminal 1

```
source ~/labs/lab05/exercise.sh
ls ~/labs/lab05/rootfs
systemctl is-active ce-lab05.scope; echo "scope status=$? (nonzero means not active)"
```

Record: Preconditions: the prepared root's directory names and the `ce-lab05` scope state.

Step 2. Start the sandbox; this shell becomes the sandbox

Run in: VM terminal 1

```
sudo systemd-run --unit=ce-lab05 --scope --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p TasksMax=50 -p CPUQuota=20% \
  unshare --fork --pid --mount --mount-proc="$HOME/labs/lab05/rootfs/proc" \
  chroot "$HOME/labs/lab05/rootfs" /bin/sh
```

Record: Terminal 1 is now the sandbox. Leave it open and run host commands in terminal 2.

Step 3. Inspect the view inside the sandbox

Run in: Sandbox shell (terminal 1)

```
echo $$
ps
ls /
```

Record: Filesystem view and PID view rows: the directory listing, the shell PID and the visible processes.

Step 4. Read the limits and PID namespaces from the host

Run in: VM terminal 2

```
source ~/labs/lab05/exercise.sh
sandbox_limits
```

Record: Configured limits row: cpu.max, memory.max, pids.max, and the sandbox namespace's host PID from lsns.

Step 5. Run a finite busy loop inside the sandbox

Run in: Sandbox shell (terminal 1); observe from terminal 2 while it runs

```
timeout 30 sh -c "while ;; do ;; done"
echo "busy-loop exit=$?"
```

Record: The start and completion of the loop. Switch to terminal 2 while it is still running.

Step 6. Read the throttling counters from the host

Run in: VM terminal 2

```
sandbox_throttling
```

Record: Busy-loop throttling row: nr_throttled and throttled_usec before and after; their increases. Compare these with the configured cpu.max.

Step 7. After the loop finishes, exit the sandbox

Run in: Sandbox shell (terminal 1)

```
exit
```

Record: That you exited the sandbox after recording the measurements.

Step 8. Confirm the scope stopped

Run in: VM terminal 2

```
systemctl is-active ce-lab05.scope; echo "status=$? (nonzero means not active)"
```

Record: The final ce-lab05 scope state.

Recovery check: The sandbox is exited and its scope is no longer active.

Only if normal recovery fails

Step 1. Fallback if the sandbox shell is unresponsive

Run in: VM terminal 2

```
sudo systemctl stop ce-lab05.scope
sudo systemctl reset-failed ce-lab05.scope 2>/dev/null || true
```

Write your answer

Use the observations recorded beside each step.

Observation	Value / change	Responsible mechanism
Filesystem view		
PID view		
Configured limits		
Busy-loop throttling		

1. Which mechanisms produced the sandbox's filesystem and PID views?
2. What CPU, memory and task limits did the host read from the sandbox's cgroup?
3. How much did `nr_throttled` and `throttled_usec` rise while the busy loop ran?
4. Why can a separate PID view exist with or without a CPU cap?

Apply the same reasoning

Would removing CPUQuota make host processes appear in the sandbox's `ps` output?

Save results before removing this lab's resources and files: `./lab.sh 05 reset`

Lab 06 - Separate containers can share a full filesystem

Why can one container prevent another from writing to a shared filesystem?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 06 setup` on the host. No other lab is required.

Theory you need

Containers can see different root filesystems yet share the same storage capacity. A bind mount exposes a host directory inside a container; mounting that same directory into two containers does not make two copies of its free space.

This experiment uses a 32 MiB `tmpfs`, a small memory-backed filesystem, as the shared capacity pool. It demonstrates the shared-capacity mechanism from the book without filling the VM's root disk. It does not measure disk throughput or reproduce Docker image-layer storage.

A write needs both permission and available capacity. `dd` writes a known number of bytes; when the filesystem cannot hold more, it can write only part of the data and then return `ENOSPC` ("No space left on device"). A fresh container still sees the same full mount.

Use the identical one-MiB probe before filling, while full, and after deleting the filling file. A failure only in the full phase, supported by `df`, connects the symptom to shared capacity. CPU and PID cgroup limits would not create more capacity on this mount.

Source: docs/chaos-theory.md: §5.4 and Experiment Card 5.1. The `tmpfs` fixture adapts the shared-storage example.

Main lesson to learn in this lab

Separate containers can consume the same filesystem's free space. When that shared mount fills, even a fresh container cannot write; replacing the consumer does not replenish capacity. The same write succeeding before and after freeing space, but failing with ENOSPC while full, identifies the cause. Repair the shared storage constraint rather than treating container isolation as storage isolation.

Experiment

Two disposable containers mount the same 32 MiB tmpfs. Only this small lab mount is filled.

Before running: Will a fresh container be able to write after another fills their shared mount?

Measurement key

Signal	Meaning
dd bs=1M count=1	Attempts to write one MiB. Record the write's error directly. In the filling command, df runs after dd, so the container can exit zero even though dd failed.
df -h /data	Shows capacity of the shared mount. Avail approaching zero supports the ENOSPC explanation.
--rm	Removes the container after exit. It does not delete data written into the host bind mount.
mountpoint	A false result after recovery confirms the temporary filesystem was unmounted.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 06 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Mount and identify the bounded shared filesystem

Run in: VM terminal 1

```
mkdir -p ~/labs/lab06/shared
if mountpoint -q "$HOME/labs/lab06/shared"; then
    echo 'STOP: a previous mount remains. Complete recovery or reset before restarting.'
else
    sudo mount -t tmpfs -o size=32m tmpfs "$HOME/labs/lab06/shared"
fi
findmnt --mountpoint "$HOME/labs/lab06/shared" -o TARGET,FSTYPE,SIZE,AVAIL
```

Record: Preconditions: the mountpoint, the filesystem type and the total capacity.

Step 2. Baseline - write one MiB before filling

Run in: VM terminal 1

```
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
    of=/data/probe bs=1M count=1'
echo "baseline probe exit=$?"
df -h ~/labs/lab06/shared
sudo rm -f ~/labs/lab06/shared/probe
```

Record: Empty row: the probe's exit status and the available space after the write.

Step 3. Fill the 32 MiB tmpfs from one container

Run in: VM terminal 1

```
if [ "$(findmnt --mountpoint "$HOME/labs/lab06/shared" --noheadings --output FSTYPE)" = tmpfs ]; then
  docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/full bs=1M count=40; echo "filling dd exit=$?"; df -h /data'
else
  echo 'STOP: the dedicated tmpfs is not mounted. Do not run the full-file-system probe.'
fi
```

Record: Filling write: its exact error, its exit status and the df capacity.

Step 4. Repeat the probe from a fresh container while full

Run in: VM terminal 1

```
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/probe bs=1M count=1'
echo "full probe exit=$?"
df -h ~/labs/lab06/shared
```

Record: Full row: the probe's exit status, its exact error and the available capacity.

Step 5. Free capacity and repeat the identical write

Run in: VM terminal 1

```
sudo rm -f ~/labs/lab06/shared/full ~/labs/lab06/shared/probe
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/probe bs=1M count=1'
echo "recovered probe exit=$?"
df -h ~/labs/lab06/shared
```

Record: Space freed row: the probe's exit status and the available space.

Step 6. Remove the probe and unmount the temporary filesystem

Run in: VM terminal 1

```
sudo rm -f ~/labs/lab06/shared/probe
sudo umount ~/labs/lab06/shared
mountpoint ~/labs/lab06/shared; echo "status=$? (nonzero means unmounted)"
docker ps -a --filter 'name=~ce-lab06-'
```

Record: Cleanup: the mountpoint result and the container listing.

Recovery check: No ce-lab06 test container remains and the shared tmpfs is unmounted.

Write your answer

Use the observations recorded beside each step.

Phase	Probe result	Available space / error
Empty		
Full		
Space freed		

1. What did the same one-MiB write return when the mount was empty, full and freed?
2. Which error and capacity readings prove the failure was shared-space exhaustion?
3. Why did a fresh container still fail, and which action restored its ability to write?

Apply the same reasoning

Why does deleting the container that filled the mount not recover the space?

Save results before removing this lab's resources and files: ./lab.sh 06 reset

Lab 07 - Network delay accumulates across sequential calls

How does the same network delay affect one dependency exchange versus four sequential exchanges?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 07` setup on the host. No other lab is required.

Theory you need

A network namespace holds interfaces, routes and packet queueing configuration. `nsenter --net` starts a host command in a target process's network namespace. The command uses the host's installed `tc` tool while changing the container's `eth0`. No extra tool needs to be installed in the target image.

`tc` configures a queueing discipline (`qdisc`), the rule for scheduling outgoing packets on an interface. `netem delay 25ms` delays each outgoing packet by approximately 25 ms. Here only the dependency's egress is changed; the VM interface and the opposite direction are not independently delayed.

The prepared client sends one byte, waits for its reply, and repeats on the same TCP connection. If a request contains N sequential exchanges, added delay is approximately $N \times d$, with packet and scheduling variation. Timing starts after connecting, so connection establishment is excluded. Compare $N=1$ and $N=4$ after subtracting each case's own baseline. Sequential waits add; independent parallel exchanges can overlap, so their completion time follows the slowest required exchange.

The `tc` command exits after installing the rule. The rule remains in the network namespace until removed or the interface is destroyed. Inspect `qdisc` state and counters, measure a near-zero control and the larger delay, then delete the rule and repeat the original measurement.

Source: docs/chaos-theory.md: §§5.8-5.8.1 and Experiment Card 5.5.

Main lesson to learn in this lab

Sequential dependency waits add: delaying each reply by d adds roughly N times d to N sequential exchanges, after subtracting the baseline. Parallel exchanges can overlap, so the same arithmetic does not apply to every request pattern. Confirm that the queueing rule handled the measured traffic and remove it explicitly; the injector exiting does not mean the network fault ended.

Experiment

A Python echo server runs in `ce-lab07` on port 8070. The host client measures one or four exchanges per sample. Change only the container's `eth0` egress: no delay -> 1 ms control -> 25 ms -> no delay.

`net` runs `tc` in this container's network namespace. `compare_delay` subtracts each case's own baseline from its recorded mean; it does not inject another fault.

Before running: Estimate the added time for one and four sequential exchanges at 25 ms per outgoing reply. Will exiting `tc` remove the delay?

Measurement key

Signal	Meaning
exchanges / mean_ms	Each sample is the elapsed time for N sequential echo exchanges after connecting. Five samples give a mean and range.
tc -s qdisc show	netem with the requested delay confirms configuration; Sent counters increasing after probes confirm traffic traversed it.
1 ms control	Checks the measurement path with a small injected dose. It includes that dose and timing noise, so it is not a pure measurement of tool overhead.
noqueue after deletion	The fault has been removed. The repeated client measurement checks service recovery separately.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 07 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Measure the unchanged dependency

Run in: VM terminal 1

```
cd ~/labs/lab07
source ~/labs/lab07/exercise.sh
net qdisc show dev eth0
python3 probe.py 1 | tee baseline-1.txt
python3 probe.py 4 | tee baseline-4.txt
```

Record: None row: both means and ranges, and the qdisc state. Keep this shell; net stays defined.

Step 2. Measure a 1 ms control

Run in: VM terminal 1

```
net qdisc add dev eth0 root netem delay 1ms
net -s qdisc show dev eth0 | tee control-before.txt
python3 probe.py 1 | tee control-1.txt
python3 probe.py 4 | tee control-4.txt
net -s qdisc show dev eth0 | tee control-after.txt
```

Record: 1 ms control row: both means and ranges, and the Sent counters before and after.

Step 3. Change only the dose to 25 ms

Run in: VM terminal 1

```
net qdisc change dev eth0 root netem delay 25ms
net -s qdisc show dev eth0 | tee delayed-before.txt
python3 probe.py 1 | tee delayed-1.txt
python3 probe.py 4 | tee delayed-4.txt
net -s qdisc show dev eth0 | tee delayed-after.txt
```

Record: 25 ms row: both means and ranges, the configured delay, and the Sent counters before and after.

Step 4. Remove the rule and repeat both probes

Run in: VM terminal 1

```
net qdisc del dev eth0 root
net qdisc show dev eth0
python3 probe.py 1 | tee recovered-1.txt
python3 probe.py 4 | tee recovered-4.txt
```

Record: Removed row: both means and ranges, and the final qdisc state.

Step 5. Calculate added time from each case's own baseline

Run in: VM terminal 1
compare_delay

Record: The printed added time for each exchange count, and the recovery differences.
Recovery check: The container has no netem qdisc and both client measurements return toward their own baseline.

Only if normal recovery fails

Step 1. Remove only this container's injected queue

Run in: VM terminal 2
PID=\$(docker inspect -f '{{.State.Pid}}' ce-lab07)
if ["\$PID" -gt 1]; then
 sudo nsenter --target "\$PID" --net -- tc qdisc del dev eth0 root
fi

Write your answer

Use the observations recorded beside each step.

Delay	1 exchange mean / range (ms)	4 exchanges mean / range (ms)	qdisc / traffic evidence
None			
1 ms control			
25 ms			
Removed			

- 1. What were the mean and range for one and four exchanges in each phase?
- 2. How much time did the 25 ms delay add to one exchange, and to four?
- 3. Which qdisc and counter evidence proves the delay reached the measured traffic?
- 4. Did exiting tc remove the delay, and which measurements show recovery?

Apply the same reasoning

If the four exchanges ran in parallel, would their delays still add up?
Save results before removing this lab's resources and files: ./lab.sh 07 reset

Lab 08 - A syscall error tests application handling

Why can one request succeed even though its close error stops the server?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 08 setup` on the host. No other lab is required.

Theory you need

A syscall crosses the application/kernel boundary. `write` sends bytes through a file descriptor; `close` releases a descriptor. The application receives a result or error and decides whether to retry, continue or exit. A syscall error and a process exit status are different observations.

`strace -p` attaches to one running process. `-e trace=close` selects the displayed calls; `-e inject=close:error=EIO` skips the selected real calls and makes them report an I/O error. The INJECTED marker distinguishes this artificial error from a naturally occurring one. Attaching after startup avoids injecting into unrelated initialization calls.

The server's request path writes the response before closing the connection. Failure on a later close can therefore occur after the client received bytes. Test the triggering request, the process state, and the next request separately; a successful first response cannot establish that the service continues.

Observe a normal request with tracing before injecting. Tracing adds overhead, so this lab tests error handling rather than throughput. The normal trace may include an ignored `fsync` error: seeing an error does not prove that it caused the exit. Follow the actual sequence from injected call to server log to later availability.

On Linux, a real close can release the descriptor even when it reports an error. Skipping close with this injection tests the application's error path, not every real close side effect. Do not infer a descriptor leak from the error code alone or blindly retry close on a descriptor number that may have been reused.

Source: docs/chaos-theory.md: §§6.2-6.4 and Experiment Cards 6.1-6.2.

Main lesson to learn in this lab

A syscall error matters through the application's response to it. An ignored error can leave service intact, while a close error after writing an HTTP response can terminate the server even though that request succeeded. Follow the injected call, application log and next request together; one successful response proves neither continued availability nor that every side effect of a real error was reproduced.

Experiment

Target only the PID saved in `server.pid`. Use two VM terminals; Ctrl-C in the tracing terminal detaches `strace`.

Before running: Predict the triggering response, process state and next request after close returns EIO.

Measurement key

Signal	Meaning
<code>strace return / INJECTED</code>	The syscall returns -1 with EIO. The marker confirms the injected path was reached.
<code>HTTP code / size_download</code>	Code is the response status and <code>size_download</code> is body bytes received. 000 means no HTTP status was obtained, not a server status code.
<code>wait "\$server" / log</code>	In the shell that started the child, <code>wait</code> reports its exit status. Correlate that with the "error closing socket" message.
Next request	Tests continued service after the triggering request; this is independent of whether the earlier response completed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 08 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Start the server and save its PID

Run in: VM terminal 1

```
cd ~/labs/lab08
./legacy_server > ~/labs/lab08/server.log 2>&1 & server=$!
echo "$server" > server.pid
sleep 1
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code} bytes=%{size_download}\n' http://127.0.0.1:8080/
echo "server PID=$server"
```

Record: Server start: the PID, and the starting HTTP status and bytes. Keep terminal 1 open.

Step 2. Attach the normal tracer and leave it running

Run in: VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)
sudo strace -p "$PID" -e trace=write,close,fsync -o ~/labs/lab08/trace.txt
```

Record: That strace is attached. Leave it running and continue in terminal 1.

Step 3. Send one normal traced request

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
kill -0 "$server" && echo 'server still running'
```

Record: Normal traced request row: the HTTP status and bytes, and whether the server still runs. Then press Ctrl-C in terminal 2.

Step 4. Read the normal trace after detaching

Run in: VM terminal 2

```
cat ~/labs/lab08/trace.txt
```

Record: Normal trace: identify write, close and fsync calls; note any return value of -1. Read the short trace directly.

Step 5. Attach the close-error injector and leave it running

Run in: VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)
sudo strace -p "$PID" -e trace=close -e inject=close:error=EIO -o ~/labs/lab08/injected-trace.txt
```

Record: That the injector is attached, before sending the next request in terminal 1.

Step 6. Trigger the fault and collect the process result

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
sleep 1
if kill -0 "$server" 2>/dev/null; then echo 'server still running'; else wait "$server"; echo "server exit=$?";
fi
tail -3 ~/labs/lab08/server.log
cat ~/labs/lab08/injected-trace.txt
```

Record: Injected request row: HTTP and bytes, the close trace line, the log error and the exit status.

Step 7. Test the next request independently

Run in: VM terminal 1, same shell

```
curl -sS --max-time 3 -o /dev/null -w 'next: HTTP=%{http_code} bytes=%{size_download}\n' http://127.0.0.1:8080/
echo "curl exit=$?"
```

Record: Next request row: the HTTP status, the bytes and the curl exit.

Step 8. Restart, verify HTTP recovery and stop the server

Run in: VM terminal 1, same shell

```
cd ~/labs/lab08
if kill -0 "$server" 2>/dev/null; then kill "$server"; wait "$server"; fi
./legacy_server > ~/labs/lab08/server-restarted.log 2>&1 & server=$!
echo "$server" > server.pid
sleep 1
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
kill "$server"; wait "$server"
rm -f ~/labs/lab08/server.pid
```

Record: Restarted row: the HTTP status and bytes. Keep the first log and both traces.
Recovery check: The server responds after restart, then the test process is stopped.

Write your answer

Use the observations recorded beside each step.

Phase	HTTP code / bytes	Process state / exit	Trace or log evidence
Normal traced request			
Injected request			
Next request			
Restarted			

- 1. What did the normal trace show, and did its fsync error stop the server?
- 2. What did the injected close return, and why did the server exit?
- 3. Why did the triggering request still return 200 while the next one failed?

Apply the same reasoning

Does this test show that every real close error leaves a file descriptor open?
Save results before removing this lab's resources and files: ./lab.sh 08 reset

Lab 09 - A syscall denial need not crash the process

What changes when the same process installs a filter that denies getpid?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 09 setup` on the host. No other lab is required.

Theory you need

`seccomp` is a kernel filter on a process's syscall attempts. A filter can allow a call, return a chosen error, or use other actions. This example allows calls by default and adds one rule: `getpid` returns `EACCES`. It installs the policy inside its own process using `libseccomp`.

The program calls `syscall(SYS_getpid)` directly before and after loading the filter. That makes the kernel-boundary test explicit rather than relying on a library's implementation of `getpid`. Before filtering a successful call returns a positive PID; an error through syscall returns `-1` and sets `errno`.

Returning an error is not the same as killing the process. write and process-exit operations remain allowed, so the program can print the denied result and finish. Its exit status reports whether the demonstration's check passed; it is not the `getpid` return value.

A loaded filter applies to the test process and is inherited by descendants. Freeing the userspace `libseccomp` context does not remove the kernel policy. The process cannot simply undo the filter; ending it ends this experiment. A fresh process launched by the original shell does not inherit a filter installed only in the child.

Source: docs/chaos-theory.md: §§6.2 and 6.5.2 (syscalls and `libseccomp`).

Main lesson to learn in this lab

A `seccomp` denial can return an error without killing the process: the outcome depends on the selected filter action and the application's handling. The denied syscall's result, `errno` and program exit status describe different things. The installed filter persists in the process and its descendants, not its parent shell; evaluate the denied operation and the policy's scope rather than assuming every denial is a crash.

Experiment

Setup compiles `filter.c`. Running `./filter` changes only that test process's syscall policy. No container is required.

Before running: Predict `getpid`'s result before and after the filter, and whether the program can still print the error.

Measurement key

Signal	Meaning
<code>getpid</code> result	A positive number is a successful PID result. <code>-1</code> indicates a syscall error; read <code>errno</code> with it.
<code>errno=13</code>	<code>EACCES</code> : the rule's chosen permission-denied error. Only interpret <code>errno</code> after an error return.
<code>exit</code>	0 means the expected denial was observed; 1-3 mean filter setup failed; 4 means the observation did not match the rule.
<code>Seccomp</code>	In <code>/proc/self/status</code> , 0 means disabled and 2 means filter mode. This reports the reading process's state, not proof of which syscall was denied.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 09 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Run the filter and capture its exit status

Run in: VM terminal 1

```
cd ~/labs/lab09
./filter
filter_exit=$?
echo "filter exit=$filter_exit"
```

Record: Before filter and After filter rows: the PID, the result, the errno and the filter exit.

Step 2. Start a fresh instance from the original shell

Run in: VM terminal 1, same shell

```
cd ~/labs/lab09
./filter
echo "fresh instance exit=$?"
grep '^Seccomp' /proc/self/status
```

Record: Fresh process row: the PID, the later result and errno, the exit, and the Seccomp reading.

Recovery check: The demonstration has exited; the filter was scoped to that process. Reset removes the compiled fixture.

Write your answer

Use the observations recorded beside each step.

Phase	Syscall result	errno / process exit
Before filter		
After filter		
Fresh process		

1. What did `getpid` return before and after the filter, and with which `errno`?
2. Why can the program still print, and what does its exit status mean?
3. Does a fresh process inherit the filter, and what shows that?

Apply the same reasoning

Would `seccomp_release` restore `getpid` inside the filtered process?

Save results before removing this lab's resources and files: `./lab.sh 09 reset`

Lab 10 - Ownership, disruption budgets and Service routing

What do ownership, disruption budgets and Service routing each guarantee during Pod replacement?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 10 setup` on the host. No other lab is required.

Theory you need

A Pod groups containers and has its own object identity (UID). A Deployment expresses the desired replica count; its ReplicaSet creates Pods to maintain that count. Deleting one managed Pod leaves the desired count unchanged, so reconciliation creates a different Pod rather than resurrecting the deleted object.

The controller creates the replacement, the scheduler chooses a node, and kubelet starts its containers. These are asynchronous steps. The old Pod may still be terminating while the new one starts; a temporarily larger object count is not itself a failure.

A Service selects Pods by labels and routes through eligible endpoints. Readiness determines whether a Pod is eligible; Running alone is insufficient. Here readiness opens TCP port 8080, whereas the client asks `/healthz` for an HTTP result. Endpoint changes and existing connections can lag behind Pod changes.

Measure both recovery of replica count and sampled client availability. With 100 probes, sampled availability is successful HTTP responses / 100. Zero failed probes means none of these probes saw failure; requests between samples or other paths remain untested. Goldpinger's local `/healthz` response is not a measurement of every peer connection.

Follow `ownerReferences` from Pod to ReplicaSet to Deployment. Labels select resources; ownership identifies the controller responsible for replacement. Kubernetes does not recreate an unmanaged Pod merely because a Service selects it.

A PodDisruptionBudget (PDB) constrains voluntary evictions through the Eviction API. With three healthy selected Pods and `minAvailable=3`, `disruptionsAllowed` is zero. An eviction should be refused with a disruption-budget error; Forbidden would instead mean an authorization failure. Direct Pod DELETE bypasses this check, and a PDB cannot prevent node failures.

After replacement, deliberately change only the Service selector to match no Pods. The Pods can stay Ready while the Service loses eligible endpoints. Restoring the selector repairs discovery without restarting the application. EndpointSlices report addresses and conditions, not a per-request success guarantee.

A failure domain is a set of replicas that one failure can remove together. Replicas on the same node share node failure; separate kind nodes still share the host VM. Replica count alone does not establish resilience to either shared failure.

Source: docs/chaos-theory.md: §§10.4, 10.5.1-10.5.2; Experiment Card 10.1.

Main lesson to learn in this lab

Pod replacement, disruption protection and traffic routing are separate mechanisms. A controller creates a new Pod identity; a PDB constrains Eviction API requests, not direct deletion or node loss; a Service needs matching, ready endpoints. Measure sampled client availability alongside replica recovery, and check whether replica placement spans the failure domains you intend to tolerate.

Experiment

The chaos cluster has three Goldpinger replicas. The readiness probe checks TCP 8080; HTTP probes sample /healthz through the Service.

sample_http records 100 HTTP probes, 0.2 seconds apart with a one-second request limit. summarize_http reports successes, failures and availability only after all 100 probes are saved. In describe output, follow Controlled By; in EndpointSlices, count addresses with ready: true.

Before running: Predict whether the PDB blocks eviction and direct deletion separately. Then predict Pod readiness and HTTP results when only the Service selector is wrong.

Measurement key

Signal	Meaning
Pod name / UID	Compare before and after deletion to identify the replacement. A changed process restart count inside the same Pod would be a different event.
READY / endpointslices	READY reports the configured readiness result. EndpointSlice addresses and readiness show the backend set used for routing.
probes.log	Each row contains a timestamp and HTTP code. Count non-200 rows only after all 100 samples finish.
k helper	Runs kubectl against kind-lab10 in namespace chaos-labs. Node queries are cluster-scoped even though the helper specifies a namespace.
ownerReferences	Read the Pod owner, then that ReplicaSet's owner. A selected Pod and an owned Pod are different relationships.
disruptionsAllowed / eviction error	Wait for the PDB controller to report currentHealthy=3 and disruptionsAllowed=0. Only a budget rejection tests the intended guard.
Service selector / ready endpoints	An extra unmatched selector label makes the selector's AND expression false. Count endpoints with ready=true, including an empty set.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 10 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Checkpoint the prepared chaos cluster

Run in: VM terminal 1

```
source ~/labs/lab10/exercise.sh
source ~/labs/lab10/env.sh
k get nodes
k get pods -l app=goldpinger -o wide
k auth can-i list pods --as=system:serviceaccount:chaos-labs:goldpinger
k auth can-i delete pods --as=system:serviceaccount:chaos-labs:goldpinger
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab10-control-plane)
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: Baseline ownership row: the Ready count, Pod names and UIDs, endpoints, HTTP result and both authorization answers.

Step 2. Trace the Pod, ReplicaSet and Deployment owners

Run in: VM terminal 1, same shell

```
victim=$(k get pods -l app=goldpinger -o jsonpath='{.items[0].metadata.name}')
rs=$(k get pod "$victim" -o jsonpath='{.metadata.ownerReferences[0].name}')
k describe pod "$victim"
k describe rs "$rs"
```

Record: The victim name and UID, its ReplicaSet owner and that ReplicaSet's owner. Keep victim defined.

Step 3. Create the disruption budget and attempt eviction

Run in: VM terminal 1, same shell

```
k create pdb lab10-budget --selector=app=goldpinger --min-available=3 --dry-run=client -o yaml | k apply -f -
k wait pdb/lab10-budget --for=jsonpath='{.status.currentHealthy}'=3 --timeout=60s
k wait pdb/lab10-budget --for=jsonpath='{.status.disruptionsAllowed}'=0 --timeout=60s
k get pdb lab10-budget
printf '{"apiVersion":"policy/v1","kind":"Eviction","metadata":{"name":"%s","namespace":"chaos-labs"}}\n'
"$victim" > ~/labs/lab10/eviction.json
k create --raw "/api/v1/namespaces/chaos-labs/pods/$victim/eviction" -f ~/labs/lab10/eviction.json
k get pod "$victim" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: Eviction request row: currentHealthy, disruptionsAllowed, the exact eviction response and the victim UID afterwards.

Step 4. Start 100 probes just before the deletion

Run in: VM terminal 2

```
source ~/labs/lab10/exercise.sh
sample_http
```

Record: Leave this loop running and start the deletion in terminal 1 now. Wait for all 100 rows.

Step 5. Delete the same Pod directly and watch the replacement

Run in: VM terminal 1, same shell

```
echo "victim=$victim deleted_at=$(date --iso-8601=ns)"
k delete pod "$victim" --wait=false
timeout 60s kubectl --kubeconfig="$HOME/labs/lab10/kubeconfig" --context=kind-lab10 --namespace=chaos-labs get
pods -l app=goldpinger -w
# timeout exit 124 ends observation, not the Kubernetes experiment.
```

Record: The deletion timestamp and the replacement events. The watch ends itself after 60 seconds.

Step 6. Collect recovery evidence after the probes finish

Run in: VM terminal 1, same shell

```
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k get pods -l app=goldpinger -o wide
k get pod "$victim"
k get endpointslices -l kubernetes.io/service-name=goldpinger -o yaml
summarize_http
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

Record: Direct deletion (100 probes) row: the Pod UIDs, the Ready count, the endpoints, failed out of 100 and the availability percentage.

Step 7. Break only the Service selector

Run in: VM terminal 1, same shell

```
k delete pdb lab10-budget --ignore-not-found
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":"lab10-no-match"}}}'
sleep 5
k get pods -l app=goldpinger
k get svc goldpinger -o jsonpath='{.spec.selector}{\n}'
k get endpointslices -l kubernetes.io/service-name=goldpinger -o yaml
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$NODE_IP:30080/healthz"
```

Record: Wrong selector row: the selector, the Ready Pod count, the ready endpoint count and the HTTP status.

Step 8. Restore the selector and confirm routing recovery

Run in: VM terminal 1, same shell

```
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
for i in $(seq 1 15); do
  curl -fs --max-time 2 "http://$NODE_IP:30080/healthz" && break
  sleep 1
done
k get pods -l app=goldpinger
k get endpointslices -l kubernetes.io/service-name=goldpinger -o yaml
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
```

Record: Selector restored row: the Ready Pod count, the ready endpoint count and the HTTP response.

Recovery check: Three replicas and three eligible Service endpoints are restored, HTTP succeeds, and lab10-budget is absent.

Only if normal recovery fails

Step 1. Restore Service selection and remove the experiment PDB

Run in: VM terminal 1, same shell

```
source ~/labs/lab10/env.sh
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
k delete pdb lab10-budget --ignore-not-found
```

Write your answer

Use the observations recorded beside each step.

Phase	Identity / Ready Pods	PDB / ready endpoints	HTTP evidence
Baseline ownership			
Eviction request			
Direct deletion (100 probes)			
Wrong selector			
Selector restored			

1. Which controller replaced the deleted Pod, and how does the UID prove replacement?
2. Why was the eviction refused while the direct deletion succeeded?
3. How many of the 100 probes failed, and what does that number establish?
4. Why did Ready Pods stop answering when the Service selector changed?

Apply the same reasoning

Would three replicas on the same node establish resilience to losing that node?

Save results before removing this lab's resources and files: `./lab.sh 10 reset`

Lab 11 - Health depends on what you test

Why can the same slow dependency pass one readiness probe and fail another?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 11` setup on the host. No other lab is required.

Theory you need

Goldpinger discovers labelled peer Pods and periodically asks each for an HTTP response. A peer is healthy to a caller only if the operation succeeds within that caller's timeout. A slow but running process can fail that test.

The added Pod contains Goldpinger on port 9090 and Toxiproxy on port 8080. Containers in the Pod share a network namespace, so the proxy forwards to 127.0.0.1:9090. Peers still call port 8080 and traverse the proxy; this leaves the original replicas unchanged. Its app label matches the Service, but it lacks `component=baseline`, so the baseline ReplicaSet does not own or adopt it.

A latency toxic delays traffic in a selected direction. Here it delays upstream application data travelling from the caller through the proxy to Goldpinger. A TCP connection to the proxy can be accepted before that data is forwarded. Kubernetes' TCP readiness probe therefore tests a weaker property than the peer's timed HTTP request.

Compare no delay, 100 ms, and 400 ms against the 300 ms peer budget. Under-budget delay can show up in response time without changing a boolean health result. Over-budget delay can change peer health while TCP readiness remains true. Reports are periodically refreshed, so wait for fresh samples and remove the toxic to test recovery.

A second run changes readiness from TCP to HTTP GET `/healthz` through the same proxy, with `timeoutSeconds=1`, `periodSeconds=2` and `failureThreshold=2`. At 400 ms the peer's 300 ms request can fail while the one-second readiness request succeeds. At 1400 ms both budgets are exceeded; readiness needs consecutive failed samples before changing.

Readiness failure makes this Pod ineligible for ordinary Service traffic; it does not restart a container. A liveness failure could restart it, and a startup probe would defer readiness and liveness until startup succeeds. This fixture intentionally has neither liveness nor startup probes. Record UID and restart counts to distinguish readiness recovery from replacement. Restarting a caller does not remove delay in its dependency. A liveness check tied to that dependency can repeatedly restart otherwise functioning callers and reduce available capacity.

Probe configuration on this standalone Pod cannot be changed in place. Recreate it between the TCP and HTTP comparisons, establish a fresh baseline and record its new UID. Within each comparison, changing only the toxic must not require recreation. Goldpinger discovers labelled Pod IPs directly, so removing an unready Pod from Service routing does not necessarily stop peer pings to it.

Source: docs/chaos-theory.md: §§10.4.4, 10.5.3-10.5.4; Experiment Card 10.2.

Main lesson to learn in this lab

Health depends on the operation tested and its deadline: accepting TCP says less than completing HTTP within a caller's budget. Readiness controls ordinary Service eligibility, not container restarts, and clients using Pod IPs can bypass that routing decision. Compare probe and caller results with endpoint, UID and restart evidence; a running or Ready Pod is not a universal guarantee of useful service.

Experiment

Use this lab's three Goldpinger replicas. Compare 0, 100 and 400 ms using TCP readiness, then recreate only the extra Pod with one-second HTTP readiness and compare 0, 400 and 1400 ms. The peer budget remains 300 ms throughout.

add_delay and change_delay take milliseconds and change only the upstream latency toxic, with zero jitter. remove_delay deletes it. Each following API read shows the actual configured fault.

Before running: Compare the 300 ms peer budget with TCP readiness and one-second HTTP readiness at 400 ms and 1400 ms. Predict endpoint eligibility and restart counts separately.

Measurement key

Signal	Meaning
OK / ms / error / PingTime	Whether the selected peer met its ping budget, its latency and any error. Check PingTime against the printed observation time; a missing report is missing evidence, not a failed ping.
Ready	In the TCP comparison it tests only the listener. In the HTTP comparison it tests /healthz through the proxy within one second, needing two consecutive failures to go unready.
Toxiproxy toxic	The API shows the configured direction and dose. Check it before interpreting peer observations.
Repeated samples	Read each original replica directly. That avoids accidentally sampling only one backend through the Service.
HTTP readiness / EndpointSlice ready	Read the slow Pod's own EndpointSlice condition. Other ready replicas can keep the Service available and hide this Pod's failure.
UID / restartCount	A new UID between configurations is deliberate recreation. Within one configuration, unchanged UID and restart counts support readiness-only recovery.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 11 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Connect the proxy and record the TCP baseline

Run in: VM terminal 1

```
source ~/labs/lab11/exercise.sh
source ~/labs/lab11/connect-proxy.sh
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
pings
readiness
```

Record: TCP / 0 ms row: peer reports and PingTime, Ready and endpoint conditions, UID and restart counts.

Step 2. Set 100 ms delay and record the TCP comparison

Run in: VM terminal 1, same shell

```
add_delay 100
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 8
pings
readiness
```

Record: TCP / 100 ms row: the configured toxic, all three peer reports, Ready and endpoint conditions, UID and restarts.

Step 3. Set 400 ms delay and record the TCP comparison

Run in: VM terminal 1, same shell

```
change_delay 400
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 8
pings
readiness
```

Record: TCP / 400 ms row: the same measurements. Repeat pings if the report times are stale.

Step 4. Switch the extra Pod to HTTP readiness

Run in: VM terminal 1, same shell

```
remove_delay
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
source ~/labs/lab11/connect-proxy.sh ~/labs/lab11/slow-http.yaml
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
pings
readiness
```

Record: HTTP / 0 ms row: the new UID, and the baseline peer, Ready, endpoint and restart readings.

Step 5. Set 400 ms delay and record the HTTP comparison

Run in: VM terminal 1, same shell

```
add_delay 400
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 10
pings
readiness
```

Record: HTTP / 400 ms row: the toxic, peer reports, Ready and endpoint conditions, UID and restarts.

Step 6. Set 1400 ms delay and watch HTTP readiness change

Run in: VM terminal 1, same shell

```
change_delay 1400
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
for i in $(seq 1 6); do date --iso-8601=seconds; readiness; sleep 3; done
pings
```

Record: HTTP / 1400 ms row: the toxic, the timestamped Ready and endpoint changes, peer reports, UID and restarts.

Step 7. Remove the toxic and record recovery in the same Pod

Run in: VM terminal 1, same shell

```
remove_delay
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
k wait --for=condition=Ready pod/goldpinger-slow --timeout=60s
sleep 8
pings
readiness
```

Record: HTTP / removed row: the toxic list, fresh peer reports, Ready and endpoint conditions, UID and restarts.

Step 8. Stop the port-forward and remove the extra Pod

Run in: VM terminal 1, same shell

```
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k get pods -l app=goldpinger
```

Record: Cleanup: the final Pod list and readiness, the delete result and the rollout result.

Recovery check: The toxic is removed and the original three replicas are Ready.

Write your answer

Use the observations recorded beside each step.

Probe / dose	Peer health / latency	Pod Ready / endpoint ready	UID / restarts
TCP / 0 ms			
TCP / 100 ms			
TCP / 400 ms			
HTTP / 0 ms			
HTTP / 400 ms			
HTTP / 1400 ms			
HTTP / removed			

1. Why did TCP readiness stay true at 400 ms while peer pings failed?
2. Which budget did each HTTP dose break, and when did readiness change?
3. What happened to the slow Pod endpoint, and can peers still reach its IP directly?
4. Did the Pod recover by readiness alone, and which values prove that?

Apply the same reasoning

Would adding liveness with the same slow dependency necessarily improve availability?

Save results before removing this lab's resources and files: `./lab.sh 11 reset`

Lab 12 - Locate startup failures and measure recovery

Can one startup SLI locate three different failure stages?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 12 setup` on the host. No other lab is required.

Theory you need

An SLI is a measured service outcome; an SLO is a target for it. This lab measures elapsed time from submitting a fresh Pod and Service to receiving HTTP 200 through that Service. The per-trial target is at most 30 seconds. Pod Running is an earlier, weaker milestone.

Startup includes API submission, scheduling, container startup, readiness and usable routing. The image is cached, so this experiment does not include a cold image download. Repeating a warm-start check cannot establish a cold-start guarantee. With `imagePullPolicy=Always`, the runtime checks the registry to resolve the image to a digest (a content identifier), but can reuse cached layers. Always does not mean every layer is downloaded again; registry access and layer transfer are different startup costs.

Test the checker as well as the workload: a known 45-second startup delay should exceed its 30-second budget, and restoring the normal command should recover. Delete the previous Pod and Service before each trial so an old server cannot answer for the new one.

A completed trial can report `success=0`; that is valid evidence of a missed target. A checker error or stale metrics is not a completed failed trial. The last-run timestamp identifies fresh results. Over many representative trials the fraction meeting the target forms an SLI; these diagnostic trials do not establish

a long-term SLO.

The same HTTP deadline miss can have different causes. An unmatched required nodeSelector leaves a Pod unscheduled: PodScheduled=False, reason Unschedulable, and no nodeName. A delayed application has a node but cannot yet pass readiness. A wrong Service selector leaves a healthy Ready Pod with zero matching endpoints. Use these intermediate observations to locate the failed stage; success=0 alone does not identify it.

A container restart reruns the same configured command. It does not change nodeSelector or Service labels, and a command containing a 45-second sleep repeats that sleep after restart. Recovery must change the setting responsible for the failed stage.

The checker saves a per-trial JSON snapshot of the Pod, Service, EndpointSlices and Pod events before cleanup. Capture occurs after the timed HTTP measurement, so it does not increase the measured startup duration. Each read is a separate observation, not an atomic cluster snapshot; diagnostics errors are recorded explicitly.

Run three healthy trials after removing the faults. The fraction meeting the target is successful completed trials / all completed trials. Keep deliberate fault trials separate from the healthy sample. A small controlled sample tests repeatability, not a production reliability target; a missed trial has no measured eventual time-to-success.

Source: docs/chaos-theory.md: §§11.2, 11.4.1-11.4.3; scheduling in §12.1.1.

Main lesson to learn in this lab

Measure startup as usable HTTP service within a deadline, then locate a miss using scheduling, readiness and endpoint evidence. A single failed outcome cannot identify the failed stage, and a checker error is not a completed trial. Repair that stage and repeat the same measurement; a few successful warm starts establish neither cold-start performance nor a long-term reliability guarantee.

Experiment

Setup prepares the checker and manifest. Each invocation creates a fresh workload, waits up to its 30-second budget, records a result and cleans up. The image is preloaded.

trial takes an evidence label and an optional trial count. It runs the existing 30-second checker, prints diagnostics captured before cleanup, and saves evidence-<label>-<number>.json. A checker error stops that run and invalidates its summary. startup_summary computes successes / 3 for the completed restored run.

Before running: Predict success and the distinguishing Pod/endpoint evidence for delayed startup, impossible placement and wrong Service selection. Predict which faults restarting the container could repair.

Measurement key

Signal	Meaning
success / elapsed	success=1 means an HTTP 200 was observed within the budget. success=0 means none was observed by the deadline. elapsed is seconds from submission.
checker exit	0 means the checker completed and recorded trials, including misses. Nonzero means execution or cleanup failed; inspect the output before using metrics.
ce_last_run_timestamp_seconds	Unix timestamp of the last completed trial. It must advance; old output cannot represent a new run.
NotFound after cleanup	Both Pod and Service should be absent. An API connection error is not evidence that they were deleted.
PodScheduled / nodeName / Ready	No node and Unschedulable locates placement failure. Assigned but unready locates a later stage. Ready alone does not prove Service routing.
diagnostics/run-N.json	Contains the trial timestamp, measurement and pre-cleanup API evidence. Errors mean the corresponding diagnostic was unavailable; they are not an empty successful query.
Healthy sample SLI	Calculate the fraction from the three run output lines, not start.prom, which contains only the most recent trial.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 12 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Open the prepared workspace

Run in: VM terminal 1

```
cd ~/labs/lab12
source env.sh
source exercise.sh
```

Record: No measurement yet. trial runs the checker and displays its saved diagnostic evidence; `startup_summary` counts only a completed restored sample.

Step 2. Run one healthy baseline

Run in: VM terminal 1, same shell

```
cp startup-good.yaml startup.yaml
trial normal
k get pod startup-check
k get svc startup-check
```

Record: Normal row: success and elapsed, the trial timestamp, node and conditions, ready endpoints, checker exit and cleanup results.

Step 3. Run the 45-second startup-delay fault

Run in: VM terminal 1, same shell

```
cp startup-delayed.yaml startup.yaml
trial delayed
k get pod startup-check
k get svc startup-check
```

Record: 45 s delay row: success and elapsed, the trial timestamp, node and conditions, ready endpoints, checker exit and cleanup results.

Step 4. Run the impossible-placement fault

Run in: VM terminal 1, same shell

```
cp startup-unscheduled.yaml startup.yaml
trial unscheduled
k get pod startup-check
k get svc startup-check
```

Record: Unmatched nodeSelector row: success and elapsed, the trial timestamp, node and conditions, ready endpoints, checker exit and cleanup results.

Step 5. Run the wrong-Service-selector fault

Run in: VM terminal 1, same shell

```
cp startup-selector.yaml startup.yaml
trial selector
k get pod startup-check
k get svc startup-check
```

Record: Wrong Service selector row: success and elapsed, the trial timestamp, node and conditions, ready endpoints, checker exit and cleanup results.

Step 6. Restore the healthy manifest and collect three trials

Run in: VM terminal 1, same shell

```
cp startup-good.yaml startup.yaml
trial restored 3
k get pod startup-check
k get svc startup-check
```

Record: Restored trial 1, 2 and 3 rows, separately: success and elapsed, timestamp, node and conditions, endpoints, checker exit and cleanup.

Step 7. Calculate the restored sample success fraction

Run in: VM terminal 1, same shell

```
startup_summary
```

Record: The restored successes out of three, with the fraction and percentage. Exclude the deliberate fault trials.

Recovery check: The healthy manifest is restored and no startup-check Pod or Service remains.

Only if normal recovery fails

Step 1. Manual fallback if the loop was killed

Run in: VM terminal 2

```
source ~/labs/lab12/env.sh
k delete -f ~/labs/lab12/startup.yaml --ignore-not-found --grace-period=1
```

Write your answer

Use the observations recorded beside each step.

Trial	success / elapsed	Scheduled / Ready	Eligible endpoints / cause
Normal			
45 s delay			
Unmatched nodeSelector			
Wrong Service selector			
Restored trial 1			
Restored trial 2			

Trial	success / elapsed	Scheduled / Ready	Eligible endpoints / cause
Restored trial 3			

1. Which startup stage failed in each of the three deliberate faults?
2. How do you distinguish a completed deadline miss, missing diagnostics and a checker execution error?
3. What was the restored success fraction, and what does that sample establish?
4. Which commands restored the workload, and would a restart have repaired the faults?

Apply the same reasoning

Does imagePullPolicy: Always force every image layer to download on each run?

Save results before removing this lab's resources and files: `./lab.sh 12 reset`

Lab 13 - Placement, supervision and replacement

How does cordoning a node differ from losing its kubelet?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 13 setup` on the host. No other lab is required.

Theory you need

Kubelet is the node agent that reports state and asks the container runtime to start or stop containers. The runtime executes them separately. Stopping kubelet removes supervision and reporting without inherently stopping an already-running application.

Missing reports must first be detected. The control plane then marks the node unavailable and applies a NoExecute taint. A Pod's tolerationSeconds says how long it tolerates that taint before eviction begins; it is not measured from the moment kubelet stopped.

This lab explicitly gives node-web 20-second not-ready and unreachable tolerations to shorten the comparison. Total replacement time still includes node detection, eviction processing, scheduling and startup. Do not infer a universal Kubernetes recovery time from this configured example.

A replacement Pod can be created elsewhere while the original process remains on the unsupervised node. The API records desired and reported state; it cannot by itself prove that an unreachable node executed a stop. Compare the original container ID in `crictl` with the new Pod UID, then restore kubelet so the old node can reconcile.

Cordon marks a node unschedulable for ordinary new Pods; it does not evict or stop existing Pods and does not turn off kubelet. A replacement created after you delete a managed Pod must find another eligible node. Uncordon permits future placement again; it does not move existing Pods back.

The node-web workload prefers lab13-worker but does not require it. A preference can be overridden when that node is unavailable; a required hostname constraint could instead leave a replacement Pending. Record placement rather than assuming that replica count guarantees independent failure domains. All kind nodes still share this VM.

Record the Lease renewTime separately from the Node Ready condition. A stale Lease shows missing heartbeats; later conditions and NoExecute taints drive a different stage. Timestamped samples place events between observations; ten-second sampling does not measure exact detection latency.

Source: docs/chaos-theory.md: §§12.1.1-12.1.3 and 12.3.1-12.3.2.

Main lesson to learn in this lab

Cordon changes future placement; losing kubelet removes supervision and reporting while existing containers may keep running. Replacement follows detection, taint tolerance, eviction and startup, so a toleration is not the total recovery time. Compare API state with the original runtime process: a new Pod elsewhere does not prove the old process stopped, and restoring supervision is part of recovery.

Experiment

First cordon the worker hosting node-web and test replacement. Restore scheduling, then stop kubelet on the replacement's worker. Sample 18 times, sleeping ten seconds between samples; API calls add time. Restore kubelet even if no replacement was observed.

sample_node records 18 observations ten seconds apart: Ready, NoExecute taints, replacement UID, Lease and original runtime container. Failed API reads stay unknown. node_intervals calculates the two intervals from samples.tsv; these are sampled bounds, not exact transition times.

Before running: Predict which operation changes scheduling eligibility, which stops reporting, and whether either immediately kills existing containers. Predict whether uncordon moves the replacement back.

Measurement key

Signal	Meaning
Node Ready / taints	A missing heartbeat can produce Ready=Unknown, displayed as NotReady. Record when the taint appears separately from the stop time.
Pod UID / node	A new UID on another node is a replacement. The old object may remain Terminating until its kubelet returns.
crictl ps	Queries the runtime inside the original kind node directly, independent of kubelet's API reports. Match the recorded original container ID.
20-second toleration	Starts when the matching NoExecute taint applies. It neither stops the old process nor guarantees replacement exactly 20 seconds later.
spec.unschedulable	true blocks ordinary new placement but says nothing about the existing process or node health.
Lease renewTime	Node heartbeats are represented by a Lease in kube-node-lease. Compare renewTime before and during loss of kubelet supervision.
Observed time intervals	Compute stop -> first unavailable sample and first taint -> first replacement sample separately; mark unresolved events as not observed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 13 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Record the original Pod and runtime container

Run in: VM terminal 1

```
source ~/labs/lab13/exercise.sh
source ~/labs/lab13/env.sh
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
date --iso-8601=seconds
echo "original pod=$POD node=$NODE"
k get pod "$POD" -o jsonpath='{.metadata.uid}{"\n"}{.spec.tolerations}{"\n"}'
docker exec "$NODE" crictl ps --name web
```

Record: Baseline row: the timestamp, Pod name and UID, node, runtime container ID and both 20-second tolerations.

Step 2. Cordon the host and inspect the existing Pod

Run in: VM terminal 1, same shell

```
ORIGINAL_NODE=$NODE
date --iso-8601=seconds
k cordon "$ORIGINAL_NODE"
k get node "$ORIGINAL_NODE" -o jsonpath='{.spec.unschedulable}{"\n"}'
k get pod "$POD" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
docker exec "$ORIGINAL_NODE" systemctl is-active kubelet
docker exec "$ORIGINAL_NODE" crictl ps --name web
```

Record: Cordoned (same UID) row: the timestamp, unschedulable, the Pod UID and node, the kubelet state and the runtime ID.

Step 3. Delete the Pod while its node is cordoned

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
k delete pod "$POD" --wait=true --timeout=60s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
NEW_POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NEW_NODE=$(k get pod "$NEW_POD" -o jsonpath='{.spec.nodeName}')
NEW_CID=$(docker exec "$NEW_NODE" crictl ps --name web -q)
echo "after deletion: pod=$NEW_POD node=$NEW_NODE container=$NEW_CID"
```

Record: Deleted while cordoned row: the Pod UID and node, the runtime container ID and the rollout result.

Step 4. Uncordon and save a fresh baseline for the kubelet trial

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
k uncordon "$ORIGINAL_NODE"
sleep 5
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
OLD_CID=$(docker exec "$NODE" crictl ps --name web -q)
echo "kubelet trial: pod=$POD node=$NODE container=$OLD_CID"
OLD_UID=$(k get pod "$POD" -o jsonpath='{.metadata.uid}')
echo "kubelet trial UID=$OLD_UID"
kubectrl --kubeconfig="$HOME/labs/lab13/kubeconfig" -n kube-node-lease get lease "$NODE" -o
  jsonpath='{.spec.renewTime}{"\n"}'
```

Record: Uncordoned (no automatic move) row: the timestamp, the Pod UID and node, and the runtime ID. Keep the saved variables.

Step 5. Stop kubelet and save command-completion time

Run in: VM terminal 1, same shell

```
docker exec "$NODE" systemctl stop kubelet
STOP_EPOCH=$(date +%s)
printf '%s\n' "$STOP_EPOCH" > ~/labs/lab13/stopped-at.txt
date --iso-8601=seconds
docker exec "$NODE" systemctl is-active kubelet
docker exec "$NODE" crictl ps --name web
```

Record: Kubelet stopped row: the stop-command time, the kubelet state, and whether OLD_CID still runs.

Step 6. Sample heartbeats, node state and replacement 18 times

Run in: VM terminal 1, same shell

sample_node

Record: Lease stale / taint and Replacement rows, from the timestamped samples. Missing observations stay unknown.

Step 7. Restore kubelet and check convergence

Run in: VM terminal 1, same shell

```
date --iso-8601=seconds
docker exec "$NODE" systemctl start kubelet
k wait --for=condition=Ready "node/$NODE" --request-timeout=0 --timeout=180s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get nodes
k get pods -l app=node-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName,DELETING:.metadata.deletionTimestamp'
docker exec "$NODE" crictl ps --name web
k get nodes -o custom-columns=NAME:.metadata.name,UNSCHEDULABLE:.spec.unschedulable
echo "original runtime ID=$OLD_CID"
```

Record: Kubelet restored row: the timestamp, node readiness, Pod UID and node, unschedulable flags, and whether OLD_CID remains.

Step 8. Calculate the two observed intervals

Run in: VM terminal 1, same shell

node_intervals

Record: The two printed intervals, or not observed. They are sampling differences, not exact latencies.

Recovery check: Kubelet is running on every worker, four nodes are Ready and uncordoned, and one node-web Pod is Ready. If replacement occurred, confirm the recorded old container is gone.

Only if normal recovery fails

Step 1. Restore kubelet even if the Kubernetes API is unavailable

Run in: VM terminal 2

```
for node in lab13-worker lab13-worker2 lab13-worker3; do
  docker exec "$node" systemctl start kubelet
done

# After the API responds, restore placement eligibility too.
source ~/labs/lab13/env.sh
k uncordon lab13-worker lab13-worker2 lab13-worker3
```

Write your answer

Use the observations recorded beside each step.

Event	Time	Node / Pod API state	Original container running?
Baseline			
Cordoned (same UID)			
Deleted while cordoned			
Uncordoned (no automatic move)			
Kubelet stopped			
Lease stale / taint observed			

Event	Time	Node / Pod API state	Original container running?
Replacement observed			
Kubelet restored			

1. What did cordoning change, and what did deleting the Pod change afterwards?
2. How long after the kubelet stopped did the node go unavailable, and the replacement appear?
3. Did the original container keep running while the replacement started?
4. What evidence confirms recovery, and what remains unknown if no replacement was observed?

Apply the same reasoning

Why does force-deleting the Pod object not prove only one copy runs?

Save results before removing this lab's resources and files: `./lab.sh 13 reset`

Lab 14 - Quorum, convergence and application availability

How does quorum loss affect committed configuration, Deployment convergence and existing HTTP traffic?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 14 setup` on the host. No other lab is required.

Theory you need

etcd stores Kubernetes state and uses Raft consensus. A change needs a majority of the configured voting members: $\text{floor}(n/2) + 1$. For three members that is two. Stopping a member does not remove it from membership or lower the required majority. floor rounds down; failure tolerance is n minus the required majority, assuming the survivors can communicate.

A leader coordinates committed changes. Losing one member can cause a transition or election but leaves a possible majority. Losing two prevents new commits. A minority cannot accept independent changes without risking conflicting histories.

The management path is `kubectl` -> API server -> etcd. The existing application path is HTTP client -> NodePort routing -> already-running web Pod. That routing need not perform an etcd write for every request, so these two paths can have different availability during quorum loss.

Test a real ConfigMap change, not only a read that might be cached, and separately probe HTTP. Move a static etcd manifest out of kubelet's watched directory to stop that member. Restore the same manifests through Docker, a path independent of the Kubernetes API. This recovers temporarily stopped members with intact data; it is not a backup-restore experiment.

A successful scale request commits a desired replica count; it does not mean those replicas are already Ready. Compare `spec.replicas` with `status.readyReplicas` and wait for rollout completion. This separates the storage operation from controller, scheduler and kubelet convergence.

A client timeout is an unknown result: the caller lacks an acknowledgement and must not infer that no change committed. After recovery, read the ConfigMap value and Deployment spec before overwriting them. Restore an idempotent target, meaning it is safe to repeat: set replicas to two rather than incrementing on each retry. Verify the observed result. A successful GET during disruption can be cached or use a different path and is weaker evidence than a new committed write.

Record etcd member identities and leader before and after losing one member. This lab stops a fixed member; a leader change is only expected if the stopped member was leader or another election occurred. Do not label every single-member failure a leader-election experiment. Two surviving voters suffice regardless of which member was previously leader.

Source: docs/chaos-theory.md: §§12.1.1, 12.1.4 and 12.3.3-12.3.4.

Main lesson to learn in this lab

etcd needs a majority of configured voters, $\text{floor}(n/2) + 1$, to commit changes; stopping members does not shrink that majority. Existing HTTP traffic can continue while configuration writes fail, and an accepted change still needs controllers to make it real. Measure commits, convergence and HTTP separately; after a client timeout, read back the state before retrying because the write's outcome is unknown.

Experiment

Use Lab 14's own lab14 cluster. Move static etcd manifests to stop members and restore the same manifests to recover them; Docker provides a fallback independent of kubectl.

Setup points control-plane kubelets at the HA API load balancer, so restoring a local etcd member is not tied to that member's unavailable API server.

restore_quorum starts the two stopped members and waits for endpoint health. read_persisted_state retries reads without writing. They set healthy and observed only on success; the final step refuses to overwrite state unless both succeeded.

Before running: Calculate quorum for 3, 4 and 5 voting members. Predict ConfigMap writes, scaling convergence and HTTP for one and two stopped members; predict whether a leader change is required for the fixed first target.

Measurement key

Signal	Meaning
endpoint status / health	Status identifies members and leader. Health checks test whether the endpoint can participate successfully; read all three after restoration.
crictl ps --name etcd	Confirms the selected member actually stopped. Moving the file alone is only the injection request.
ConfigMap patch	Change the state value each phase. A successful committed change exercises the write path; a no-op or cached get is weaker evidence.
HTTP code	200 from the recorded worker NodePort measures the existing data path, independently of kubectl.
spec.replicas / readyReplicas	Desired replica count is committed configuration; Ready count is asynchronous workload convergence. A scale command alone measures only the former.
ConfigMap value / resourceVersion	Read the value after recovery to resolve an uncertain write. resourceVersion is an opaque version identifier, not a number for timing or arithmetic.
Member ID / leader / raft term	Identify whether the chosen stopped member was leader before attributing an election to that fault.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 14 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh` ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Calculate quorum and establish the baseline

Run in: VM terminal 1

```
source ~/labs/lab14/exercise.sh
for voters in 3 4 5; do
  quorum=$((voters / 2 + 1))
  echo "voters=$voters quorum=$quorum tolerated_failures=$((voters - quorum))"
done
source ~/labs/lab14/helpers.sh
h get nodes
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
PORT=$(h get svc quorum-web -o jsonpath='{.spec.ports[0].nodePort}')
IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab14-worker)
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$IP:$PORT/"
h patch configmap quorum-probe --type merge -p '{"data":{"state":"baseline"}}'
h get configmap quorum-probe -o jsonpath='{.data.state}{ " resourceVersion="}{.metadata.resourceVersion}{ "\n"}'
ec lab14-control-plane endpoint status --cluster -w table
ec lab14-control-plane member list -w table
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

Record: Quorum calculations, and the 3 voters / baseline 2 replicas row: stored value and version, replicas, HTTP, members and leader.

Step 2. Stop one member and test writes, HTTP and leader state

Run in: VM terminal 1, same shell

```
stop_etcd lab14-control-plane2
echo "stop member exit=$?"
docker exec lab14-control-plane2 crictl ps --name etcd
if ! h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-down"}}'; then
  echo 'First write failed; wait five seconds and retry the same idempotent value once.'
  sleep 5
  h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-down"}}'
fi
h get configmap quorum-probe -o jsonpath='{.data.state}{ " resourceVersion="}{.metadata.resourceVersion}{ "\n"}'
curl -sS --max-time 3 -o /dev/null -w '%{http_code}\n' "http://$IP:$PORT/"

ec lab14-control-plane endpoint status -w table
ec lab14-control-plane3 endpoint status -w table
```

Record: 2 voters / scale to 3 row: the stop exit and runtime list, the write and value, HTTP, and the leader.

Step 3. Scale with two voters and observe convergence

Run in: VM terminal 1, same shell

```
h scale deployment/quorum-web --replicas=3
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

Record: 2 voters / scale to 3 row, continued: the scale response, the rollout result and the desired and Ready counts.

Step 4. Stop the second member and compare writes with HTTP

Run in: VM terminal 1, same shell

```
stop_etcd lab14-control-plane3
echo "stop member exit=$?"
docker exec lab14-control-plane3 crictl ps --name etcd
h patch configmap quorum-probe --type merge -p '{"data":{"state":"two-down"}}'
echo "ConfigMap write exit=$?"
h scale deployment/quorum-web --replicas=1
echo "scale exit=$?"
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$IP:$PORT/"
```

Record: 1 voter / attempted scale to 1 row: the stop exit, both write errors, any readable replica state and the HTTP status.

Step 5. Restore the two members and wait for endpoint health

Run in: VM terminal 1, same shell

```
restore_quorum
```

Record: Whether every endpoint health check succeeded, and the final healthy flag.

Step 6. Read persisted values before restoring configuration

Run in: VM terminal 1, same shell

```
read_persisted_state
```

Record: Restored before overwriting row: the stored value and version, desired and Ready replicas, and the leader.

Step 7. Restore the explicit baseline and verify all paths

Run in: VM terminal 1, same shell

```
if [[ ${healthy:-0} -eq 1 && ${observed:-0} -eq 1 ]]; then
  h patch configmap quorum-probe --type merge -p '{"data":{"state":"recovered"}}'
  h scale deployment/quorum-web --replicas=2
  h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
  h get configmap quorum-probe -o jsonpath='{.data.state}'
  h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
  ec lab14-control-plane endpoint health --cluster
  curl -fsS --max-time 3 "http://$IP:$PORT/" >/dev/null && echo 'HTTP recovery ok'
else
  echo 'STOP: complete quorum recovery and readback in the previous step before overwriting state.'
fi
```

Record: Restored target 2 replicas row: the stored value, desired and Ready replicas, endpoint health and HTTP.

Recovery check: All three etcd endpoints are healthy, the recovered ConfigMap value is readable, desired and Ready replicas are two, and HTTP succeeds.

Only if normal recovery fails

Step 1. Independent fallback that restores the manifests without kubectl

Run in: VM terminal 2

```
for node in lab14-control-plane2 lab14-control-plane3; do
  docker exec "$node" sh -c 'test -f /root/ce-etcd.yaml && mv /root/ce-etcd.yaml
    /etc/kubernetes/manifests/etcd.yaml; ls /etc/kubernetes/manifests'
done
```

Write your answer

Use the observations recorded beside each step.

Phase	Write / stored value	Desired / Ready replicas	HTTP / leader evidence
3 voters / baseline 2 replicas			
2 voters / scale to 3			
1 voter / attempted scale to 1			
Restored before overwriting			
Restored target 2 replicas			

1. What are quorum and tolerated failures for 3, 4 and 5 voters?
2. With one member stopped, did the write, the convergence and HTTP still work?
3. With two members stopped, why did writes fail while existing HTTP kept working?

4. What did etcd still hold after recovery, and what proves the baseline returned?

Apply the same reasoning

Would four configured members tolerate two failures?

Save results before removing this lab's resources and files: `./lab.sh 14 reset`

Lab 15 - Count how layered retries multiply work

How much extra backend work do layered retries create when the same dependency keeps failing?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 15 setup` on the host. No other lab is required.

Theory you need

An original user request is not the same as an attempt. One retry means up to two attempts. When a client retries a proxy call and the proxy retries each upstream call, the per-layer attempt limits multiply: at most $\text{client attempts} \times \text{proxy attempts} \times \text{backend arrivals}$ per original request.

Here HTTP 503 means Service Unavailable. NGINX can make two upstream attempts for that response, and the client makes either one or two proxy attempts. Both upstreams deliberately return 503 throughout the fault phase. A successful attempt stops retries early, so the maximum is reached only when the earlier attempts continue failing.

Measure amplification as $\text{backend arrivals} / \text{original requests}$, and useful work as $\text{successful original requests} / \text{original requests}$. More attempts do not imply more successes. Keep the original request count and failure condition identical when changing the client retry count.

Layered retry amplification is the first step in the book's emergent retry-storm example. This small, sequential experiment isolates multiplication; it does not simulate overloaded worker queues or prove self-sustaining failure. In a real system, retries of slow requests may add work while earlier attempts are still running; bounded attempts, a total deadline and backoff address different parts of that problem.

Source: docs/chaos-theory.md: §1.2.3 (emergent retry amplification), §12.1.4 (ingress retry accumulation). This fixture isolates attempt multiplication using HTTP 503.

Main lesson to learn in this lab

Retries at multiple layers multiply backend attempts, not successful user requests. Under persistent failure, two client attempts with two proxy attempts can create four backend arrivals for one original request without any success. Count original requests, attempts and useful results separately; bound retry work and total waiting time rather than assuming more attempts improve resilience. This comparison demonstrates amplification, not a complete retry storm.

Experiment

Each case sends 20 sequential requests in three phases: healthy, both upstreams returning HTTP 503, and recovered. NGINX allows two upstream attempts. Repeat with zero and one client retry. The runner owns its proxy inside `ce-lab15.service`, has a 60-second runtime limit, and removes the fault and processes on exit. Reset stops that unit and its children.

`retry_case` takes the client retry count (0 or 1), runs baseline, failure and recovery under a 60-second limit, and saves the full output. A nonzero runner status means the experiment did not complete cleanly.

Before running: For 20 requests, calculate the maximum backend arrivals with zero versus one client retry, given two proxy attempts. Predict success counts during a persistent 503 fault.

Measurement key

Signal	Meaning
original / client attempts	Each case has 20 original requests. Count all proxy attempts, including retries, separately.
backend arrivals	Sum the two upstream request logs within the phase. Probe and health requests are excluded.
amplification	backend arrivals / 20. Compare it with the theoretical upper bound; fewer arrivals can indicate early success or a failure before reaching a backend.
successes	Counts original requests ending in HTTP 200, not the number of attempts. Compare outcomes alongside cost.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 15 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Run the three phases with zero client retries

Run in: VM terminal 1

```
source ~/labs/lab15/exercise.sh
retry_case 0
echo "runner exit=$? (0 means the experiment completed)"
```

Record: No client retry rows, baseline to recovered: the three phase summaries and the runner exit. Stop if it is nonzero.

Step 2. Change only the client retry count from zero to one

Run in: VM terminal 1, same shell

```
retry_case 1
echo "runner exit=$? (0 means the experiment completed)"
```

Record: One client retry rows, baseline to recovered: the three phase summaries and the runner exit.

Step 3. Compare the six summaries and confirm cleanup

Run in: VM terminal 1, same shell

```
cd ~/labs/lab15
cat no-client-retry.txt one-client-retry.txt
test ! -e fail && echo 'fault marker removed'
ss -ltn '( sport = :8090 or sport = :9001 or sport = :9002 )'
```

Record: Backend arrivals divided by 20 for both fault phases, the success fractions, and the cleanup results.

Recovery check: Each runner removes the fail marker and stops its own upstreams and proxy. Both recovered phases should return 20/20 successes.

Write your answer

Use the observations recorded beside each step.

Case / phase	Client attempts	Backend arrivals / amplification	Successes / 20
No client retry: baseline			
No client retry: 503 fault			
No client retry: recovered			

Case / phase	Client attempts	Backend arrivals / amplification	Successes / 20
One client retry: baseline			
One client retry: 503 fault			
One client retry: recovered			

1. What were the client attempts, backend arrivals and successes in each of the six phases?
2. What is the maximum backend arrivals for zero and for one client retry?
3. Why did the extra attempts add no successful requests, and what did cleanup show?

Apply the same reasoning

If both layers permit three retries, what is the maximum amplification?

Save results before removing this lab's resources and files: `./lab.sh 15 reset`

Lab 16 - Design one clear experiment

How would you test one resilience claim so the result has a clear meaning?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 16 setup` on the host. No other lab is required.

Theory you need

An experiment tests a specific service claim under a specific fault. Start with a user-visible measurement: for example, successful HTTP responses divided by total probes, or response time in milliseconds. An SLI is that measured quantity; an SLO is the target you choose for it.

Write the baseline and hypothesis using the same operation, load and measurement window. For example: "With one of three replicas unavailable for 20 seconds, at least 99 of 100 probes return HTTP 200 within one second." This states a condition and a falsifiable outcome; replace the numbers with ones justified by your system.

Specify the exact target and dose, and how to prove the fault reached it. A selector matching nothing gives a misleading pass; one matching too much changes the experiment. A control with no fault and a recovery measurement help distinguish the injected cause from an already-broken baseline.

Define a stop threshold, the command that removes the fault, and a fallback that works when the normal control path fails. Analysis must connect measurements to a mechanism and limit the conclusion to the tested conditions. This lab produces a plan, so leave observations unclaimed until an authorized run supplies them.

Source: docs/chaos-theory.md: §§1.3, 2.5; Appendix C; Master Cheat Sheet §6.

Main lesson to learn in this lab

A useful chaos hypothesis links one precise fault to a measurable service outcome under stated conditions. Use the same operation and measurement window for baseline, fault and recovery; specify scope, dose, duration, proof of injection, stop threshold and rollback. The conclusion must stay within those conditions. A written plan makes a claim testable, but only an observed run can support or challenge it.

Experiment

Fill the prepared card for one service and one fault. Nothing is deployed or disrupted in this lab.

Before running: State what you expect to measure under one fault and why the service should behave that way.

Measurement key

Signal	Meaning
Baseline / hypothesis	Use a named measurement with units, load and duration in both. "Healthy" without a criterion is not enough.
Fault confirmation	Name the evidence that proves the intended target and dose were affected.
Stop / recovery	Stop threshold tells the operator when to abort. Recovery criteria tell them whether service was restored; these are different checks.
Blank-field check	Finds empty fields only. A reviewer must still judge whether the plan is measurable and executable.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 16 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Read the card and choose one service and one fault

Run in: VM terminal 1

```
cd ~/labs/lab16
cat experiment-card.md
```

Record: Your chosen service and environment, and the one fault. Map the card fields to the four questions.

Step 2. Write the complete plan, including executable commands

Run in: VM terminal 1, same shell

```
cd ~/labs/lab16
${EDITOR:-nano} experiment-card.md
```

Record: Each field: the exact command, where it runs, and its values. In nano save with `Ctrl+O`, `Enter`, then `Ctrl+X`.

Step 3. Read the saved plan and check each field

Run in: VM terminal 1, same shell

```
cd ~/labs/lab16
cat experiment-card.md
```

Record: For each field, check that it is filled and testable. Identify the measurement, one fault, its limit, the stop condition and the recovery check.

Recovery check: The card answers each field clearly; this is a design review, not a live test.

Write your answer

Use the observations recorded beside each step.

Design element	Your choice	How another operator checks it
Measurement / baseline		
Fault / target / duration		
Prediction / threshold		

Design element	Your choice	How another operator checks it
Stop / rollback / recovery		

1. Which service, measurement and baseline command did you choose?
2. Which fault, target and duration did you choose, and what threshold do you predict?
3. What stops the experiment, and which commands roll it back and prove recovery?
4. What can this plan not establish?

Apply the same reasoning

Can a completed card or a passing no-fault control establish resilience to the proposed disruption?

Save results before removing this lab's resources and files: `./lab.sh 16 reset`

Lab 17 - Signal delivery and the shutdown budget

Why can the same application shut down cleanly in one container and be killed in another?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 17 setup` on the host. No other lab is required.

Theory you need

Docker stop sends the configured stop signal to the container's main process, PID 1. After the stop timeout it forcibly kills a container that has not exited. Here the signal is explicitly SIGTERM; the worker handles it, spends two seconds cleaning up, writes a marker and exits zero.

A shell wrapper can sit between Docker and the application. This fixture deliberately ignores TERM in the wrapper and starts Python as a child. The child's own handler is installed, but Docker's signal does not reach it. Seeing a handler in application code is therefore insufficient; inspect the process relationship.

The shell exec builtin replaces the shell with the application without creating a child. It gives the worker PID 1 in the second configuration. Exec fixes this delivery path; it does not add a handler or make slow cleanup faster. A general-purpose init can reap orphaned children, but cannot repair an application that ignores its own termination signal.

SIGTERM (15) can be handled; SIGKILL (9) cannot be caught for cleanup. A one-second stop budget cannot accommodate this worker's two-second cleanup, even with correct delivery. Compare TERM receipt with completion. Exit 137 corresponds to $128 + 9$, consistent with SIGKILL, but does not identify its cause; check Docker's OOMKilled flag and the injected operation.

Container restart policies concern what happens after exit. They neither forward signals nor extend the shutdown budget. This experiment disables automatic restart and retains stopped containers so logs and exit evidence remain readable.

Source: docs/chaos-theory.md: §§5.13.1-5.13.2.

Main lesson to learn in this lab

Clean container shutdown requires both signal delivery and enough time to finish cleanup. With exec, the application replaces the wrapper as PID 1 and receives the stop signal directly; it still cannot finish two seconds of cleanup within a one-second budget. Check TERM receipt, completion markers and termination evidence separately, then repair the missing signal path or insufficient budget that the evidence identifies.

Experiment

Compare a non-forwarding wrapper with a five-second budget, exec with the same budget, then exec with one second. Only one comparison variable changes at a time.

start_worker copies the supplied worker into the named container, starts it, and waits for its ready log. The docker create command shows how PID 1 differs; docker stop shows the signal budget.

Before running: Predict the worker PID, TERM receipt, cleanup completion and exit status in all three cases.

Measurement key

Signal	Meaning
ready pid / ppid	The startup log identifies whether the worker is PID 1 or a child. Wait for this log before stopping it.
received / cleanup-complete	Receipt proves delivery to Python. The completion line and file prove this fixture finished cleanup; absence alone needs the exit and timeout evidence.
ExitCode / OOMKilled	Separate a successful handler exit from a forced stop. False OOMKilled plus the controlled stop distinguishes this kill from the memory experiment.
stop --timeout	The grace budget in seconds, not an application request timeout. The Docker command itself can succeed even when the container was killed.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 17 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Start the wrapper case and confirm the worker is ready

Run in: VM terminal 1

```
source ~/labs/lab17/exercise.sh
docker create --name ce-lab17-wrapper --stop-signal SIGTERM python:3.12-slim sh -c 'trap "" TERM; python -u /worker.py & wait'
start_worker ce-lab17-wrapper
```

Record: Wrapper / 5 s row: the ready line, the worker PID and the parent PID.

Step 2. Stop the wrapper with five seconds and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 5 ce-lab17-wrapper
docker logs ce-lab17-wrapper
docker inspect -f '{{json .State}}' ce-lab17-wrapper
docker cp ce-lab17-wrapper:/cleanup-complete ~/labs/lab17/wrapper-complete.txt
```

Record: Wrapper / 5 s row: the stop time, whether received=15 and cleanup-complete appear, ExitCode and OOMKilled.

Step 3. Start the exec case with the same application

Run in: VM terminal 1, same shell

```
docker create --name ce-lab17-exec --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
start_worker ce-lab17-exec
```

Record: Exec / 5 s row: the ready line and the worker PID.

Step 4. Stop exec with five seconds and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 5 ce-lab17-exec
docker logs ce-lab17-exec
docker inspect -f '{{json .State}}' ce-lab17-exec
docker cp ce-lab17-exec:/cleanup-complete ~/labs/lab17/exec-complete.txt
cat ~/labs/lab17/exec-complete.txt
```

Record: Exec / 5 s row: the stop time, the TERM and cleanup logs, the marker, ExitCode and OOMKilled.

Step 5. Start another exec case for the shorter budget

Run in: VM terminal 1, same shell

```
docker create --name ce-lab17-short --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
start_worker ce-lab17-short
```

Record: Exec / 1 s row: the ready line and the worker PID.

Step 6. Stop exec with one second and collect the result

Run in: VM terminal 1, same shell

```
time docker stop --timeout 1 ce-lab17-short
docker logs ce-lab17-short
docker inspect -f '{{json .State}}' ce-lab17-short
docker cp ce-lab17-short:/cleanup-complete ~/labs/lab17/short-complete.txt
```

Record: Exec / 1 s row: the stop time, TERM receipt, whether cleanup completed, ExitCode and OOMKilled.

Step 7. Remove the three stopped containers after recording evidence

Run in: VM terminal 1, same shell

```
docker rm ce-lab17-wrapper ce-lab17-exec ce-lab17-short
docker ps -a --filter 'name=/ce-lab17-' --format '{{.Names}}'
```

Record: Whether any ce-lab17 container remains, once all three rows are complete.

Recovery check: The exec/five-second case completed cleanup, the two forced cases have explicit evidence, and all three test containers are removed after evidence collection.

Write your answer

Use the observations recorded beside each step.

Case	Worker PID	TERM / cleanup	Exit / OOMKilled
Wrapper / 5 s			
Exec / 5 s			
Exec / 1 s			

1. Why did the worker receive TERM under exec but not under the wrapper?
2. Which evidence separates signal delivery from having enough time to finish cleanup?
3. What caused each exit 137, and why is that not an OOM diagnosis?

Apply the same reasoning

Would increasing the wrapper's stop budget to 30 seconds repair its missing signal forwarding?

Save results before removing this lab's resources and files: ./lab.sh 17 reset

Lab 18 - Persistence, ownership and read-only mounts

Is a failed container write caused by lost data, Unix permissions or a read-only mount?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 18 setup` on the host. No other lab is required.

Theory you need

A container's writable layer belongs to that container. Stop and start preserve it; removing and replacing the container does not. A named volume has a separate lifetime and remains when a container using it is removed. Persistence is not a backup or a guarantee against deleting the volume.

Linux permissions compare numeric UIDs and GIDs, not account names. Here the volume directory starts owned by UID 0 with mode 0755. UID 10001 can read and traverse it but cannot create files because only the owner has directory write permission. Creating a file requires write and execute permission on its parent directory.

Correcting the directory owner to 10001 repairs this specific permission mismatch without making it writable by every user. This lab uses rootful Docker without user-namespace remapping; mappings and host security policy can add other permission checks in other environments.

A read-only volume mount independently prevents writes even when ownership is correct. `chmod` or `chown` cannot override that mount property. Inspect the mount's RW field and compare the exact error; Permission denied and Read-only file system point to different layers.

Mounts can hide files that exist at the same path in the image. A missing file therefore does not always mean deletion. This fixture uses a dedicated `/data` path and deliberately compares a volume file with `/ephemeral.txt` outside that mount.

Source: docs/chaos-theory.md: §§5.13.3-5.13.4.

Main lesson to learn in this lab

Data lifetime and write permission are independent. Replacing a container loses its writable layer but can preserve a named volume; writing that volume still depends on numeric ownership and whether the mount permits writes. Match the failed operation to its error and mount state before changing permissions. Persistent storage is not automatically writable, and surviving replacement is not a backup.

Experiment

Use only the named volume `ce-lab18-data` and disposable `ce-lab18` containers. Setup resets this exercise's data; save observations elsewhere.

Before running: Predict which file survives replacement and whether UID 10001 can write before `chown`, after `chown` and with a read-only mount.

Measurement key

Signal	Meaning
UID / GID / directory mode	id and numeric ls output show the identity and permissions used by the kernel, even if no account name exists.
Mounts / RW	Docker inspect identifies the named volume and whether this container mounted it writable. Volume existence alone does not prove write access.
ephemeral.txt / persisted.txt	Compare paths after replacing the container, not merely restarting the original one.
write exit / error	Read the failing command result directly. A successful later ls must not hide the earlier write failure.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 18 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Create one file in each storage layer

Run in: VM terminal 1

```
docker run --name ce-lab18-original --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'echo layer > /ephemeral.txt; echo volume > /data/persisted.txt; cat /ephemeral.txt /data/persisted.txt; ls
-ltn /data'
docker cp ce-lab18-original:/ephemeral.txt ~/labs/lab18/original-layer.txt
docker inspect -f '{{json .Mounts}}' ce-lab18-original
```

Record: Original container row: both file contents, the `/data` owner and mode, the volume name and mount RW.

Step 2. Replace the container and check which file remains

Run in: VM terminal 1, same shell

```
docker rm ce-lab18-original
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'cat /data/persisted.txt; test ! -e /ephemeral.txt && echo writable-layer-file-absent; ls -ldn /data'
```

Record: Replacement container row: the `persisted.txt` contents, the `ephemeral.txt` check and the `/data` ownership.

Step 3. Attempt the write as UID 10001 and diagnose the failure

Run in: VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

Record: UID mismatch row: the process UID and GID, the directory owner and mode, the exact error and write exit.

Step 4. Change only the directory owner and repeat the write

Run in: VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim chown
10001:10001 /data
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

Record: Owner corrected row: the directory owner and mode, and the repeated write exit.

Step 5. Change only the mount to read-only and inspect the failure

Run in: VM terminal 1, same shell

```
docker run --name ce-lab18-readonly --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data,readonly
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
docker inspect -f '{{json .Mounts}}' ce-lab18-readonly
```

Record: Read-only mount row: the identity, the owner and mode, the exact error, the write exit and Mounts.RW.

Step 6. Restore a writable mount and confirm both data and writes

Run in: VM terminal 1, same shell

```
docker run --name ce-lab18-recovered --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'ls -ldn /data; echo recovered > /data/user.txt && cat /data/user.txt
/data/persisted.txt'
echo "write/read exit=$?"
docker inspect -f '{{json .Mounts}}' ce-lab18-recovered
```

Record: Writable recovery row: the owner and mode, RW, the command exit and both file contents.

Step 7. Remove the exercise containers and volume after recording results

Run in: VM terminal 1, same shell

```
docker rm ce-lab18-readonly ce-lab18-recovered
docker volume rm ce-lab18-data
docker ps -a --filter 'name=^/ce-lab18-' --format '{{.Names}}'
docker volume ls --filter name=ce-lab18-data
```

Record: Whether any lab container or the ce-lab18-data volume remains, once all six rows are filled.

Recovery check: A write succeeds again after restoring a writable mount, and the dedicated test volume is removed only after evidence has been recorded.

Write your answer

Use the observations recorded beside each step.

Phase	Files visible	Owner / mount RW	Write result
Original container			
Replacement container			
UID mismatch			
Owner corrected			
Read-only mount			
Writable recovery			

1. Which file survived container replacement, and why did the other one disappear?
2. Why did UID 10001 fail to write, and which change repaired it?
3. Why did the same user still fail through the read-only mount?

Apply the same reasoning

Would chmod 777 repair a read-only mount, or an ownership mismatch?

Save results before removing this lab's resources and files: ./lab.sh 18 reset

Lab 19 - Unscheduled Pods versus memory-killed containers

Did the workload fail before placement or after exceeding its memory limit?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 19` setup on the host. No other lab is required.

Theory you need

Requests and limits answer different questions. The scheduler accounts for requested resources against node allocatable capacity and existing requests. It does not place a Pod merely because live CPU usage looks low. CPU 20m means 0.02 CPU; CPU 1000 means one thousand CPUs, not one thousand millicores.

An impossible request leaves this Pod unassigned with `PodScheduled=False` and reason `Unschedulable`. No application container has started, so application logs cannot diagnose its execution. Scheduling events describe constraints; inspect the current Pod UID because a previous object may have used the same name.

A memory limit is enforced at runtime through the container's memory cgroup. Kubernetes Mi means MiB (1048576 bytes). The OOM fixture requests 16 MiB but has a 32 MiB limit and deliberately touches 96 MiB. Its request fits; its allocation does not. A memory request is not the maximum amount the application may use.

`OOMKilled` in `lastState.terminated`, an exit code and a rising `restartCount` identify a different stage from `Unschedulable`. Exit 137 alone cannot identify OOM. `CrashLoopBackOff` describes a delay before repeated restarts, not their root cause. Inspect the previous instance's logs while the Pod still exists.

These standalone Pods use `restartPolicy Always` by default. Kubelet can restart a container inside the same Pod UID. The procedure intentionally deletes and recreates the Pod between configurations because scheduling and resource fields cannot all be edited in place. That intentional UID change is not evidence of self-healing by a Deployment.

Repair the demonstrated cause. Restarting cannot make a 1000-CPU request fit. Repeated restarts cannot make the same 96 MiB allocation fit a 32 MiB limit. In a real workload, investigate memory growth before choosing a measured limit; simply removing limits can move the failure to the node.

Source: docs/chaos-theory.md: §§10.6.1-10.6.2.

Main lesson to learn in this lab

Requests guide scheduling; memory limits constrain a running container. An impossible request prevents placement, while exceeding the memory limit can kill and restart a container inside the same Pod. Use scheduling events, node assignment, termination reason and UID to identify the stage; neither `Pending`, `CrashLoopBackOff` nor exit 137 alone names the cause. Restarting cannot repair an unchanged resource mismatch.

Experiment

Use namespace `ce-lab19` on this lab's chaos cluster. Compare a healthy HTTP process, an impossible CPU request and a bounded memory allocation; then restore the healthy fixture.

Use `describe` to read Node, Requests, Limits, State, Last State, Restart Count and Events. The separate UID line distinguishes a replaced Pod from a restarted container.

Before running: Predict node assignment, available logs and restart evidence for the impossible request and the memory limit failure.

Measurement key

Signal	Meaning
nodeName / PodScheduled / events	Distinguish no placement from a node-assigned runtime problem. Events are filtered by this Pod UID.
requests / limits / allocatable	Read configured quantities and node capacity. No metrics-server is required for this diagnosis.
lastState.terminated / restartCount	The previous container result and restart count remain attached to this Pod. Capture them before deleting it.
logs --previous	Reads the previous container instance, not a previous Pod with the same name. Missing logs are not proof that no failure happened.
k	kubectl using Lab 19's private kubeconfig, kind-lab19 context and only namespace ce-lab19.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 19 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Record the healthy placement and configured resources

Run in: VM terminal 1

```
source ~/labs/lab19/helpers.sh
k get nodes -o
  custom-columns='NAME:.metadata.name,CPU:.status.allocatable.cpu,MEMORY:.status.allocatable.memory'
k describe pod resource-web
k get pod resource-web -o custom-columns=NAME:.metadata.name,UID:.metadata.uid
k logs resource-web --tail=5
```

Record: Healthy row: the Pod UID, the node, Scheduled and Ready, requests and limits, restarts, and node allocatable capacity.

Step 2. Replace the baseline with the impossible CPU request

Run in: VM terminal 1, same shell

```
k delete pod resource-web --wait=true --timeout=30s
k apply -f ~/labs/lab19/unscheduled.yaml
k wait pod/resource-web --for=jsonpath='{.status.conditions[?(@.type=="PodScheduled")].reason}'=Unschedulable
  --timeout=60s --request-timeout=0
k describe pod resource-web
k get pod resource-web -o custom-columns=NAME:.metadata.name,UID:.metadata.uid
```

Record: 1000 CPU request row: the UID, the node value, the CPU request, and the PodScheduled status and reason.

Step 3. Collect scheduling events and check application-log availability

Run in: VM terminal 1, same shell

```
uid=$(k get pod resource-web -o jsonpath='{.metadata.uid}')
k get events --field-selector "involvedObject.uid=$uid"
k logs resource-web --request-timeout=5s
echo "logs exit=$?"
```

Record: 1000 CPU request row: the current UID, the scheduling event message, and the log-request result and exit.

Step 4. Replace it with the memory failure and wait for OOM evidence

Run in: VM terminal 1, same shell

```
k delete pod resource-web --wait=true --timeout=30s
k apply -f ~/labs/lab19/oom.yaml
oom_uid=$(k get pod resource-web -o jsonpath='{.metadata.uid}')
echo "OOM fixture initial UID=$oom_uid"
k wait pod/resource-web --for=jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'=OOMKilled
--timeout=90s --request-timeout=0
```

Record: The OOM fixture UID at creation and the wait result. Do not delete the Pod yet.

Step 5. Save the runtime crash, prior logs and current-UID events

Run in: VM terminal 1, same shell

```
k get pod resource-web -o json > ~/labs/lab19/oom-evidence.json
k describe pod resource-web
k get pod resource-web -o custom-columns=NAME:.metadata.name,UID:.metadata.uid
k logs resource-web --previous | tee ~/labs/lab19/previous.log
uid=$(k get pod resource-web -o jsonpath='{.metadata.uid}')
printf 'Initial OOM UID=%s; current UID=%s\n' "$oom_uid" "$uid"
k get events --field-selector "involvedObject.uid=$uid"
```

Record: 32 MiB limit / 96 MiB allocation row: node, both UIDs, conditions, limits, the termination reason and exit, restarts and the previous log.

Step 6. Restore the healthy fixture and verify recovery

Run in: VM terminal 1, same shell

```
k delete pod resource-web --wait=true --timeout=30s
k apply -f ~/labs/lab19/good.yaml
k wait pod/resource-web --for=condition=Ready --timeout=60s --request-timeout=0
k describe pod resource-web
k get pod resource-web -o custom-columns=NAME:.metadata.name,UID:.metadata.uid
```

Record: Healthy restored row: the UID, node, conditions, resources, restarts and prior termination state.

Recovery check: The healthy resource-web Pod is Ready, and its current container has no OOM termination or repeated restarts.

Write your answer

Use the observations recorded beside each step.

Phase	UID / node	Scheduled / Ready	Reason / restarts
Healthy			
1000 CPU request			
32 MiB limit / 96 MiB allocation			
Healthy restored			

1. Which stage failed for the 1000-CPU request, and what shows it?
2. What terminated the container under the 32 MiB limit, and how do you know?
3. Which UID changes were deliberate replacements, and which restart happened inside one Pod?

Apply the same reasoning

Would increasing the CPU limit fix an unschedulable Pod whose CPU request exceeds every node's capacity?

Save results before removing this lab's resources and files: `./lab.sh 19 reset`

Lab 20 - Trace DNS, Service ports and the application listener

At which layer does a failed in-cluster HTTP request break?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 20` setup on the host. No other lab is required.

Theory you need

Follow the request through name resolution, Service IP and port, endpoint address and target port, then the application's listener. The Service selector matches Pod labels; EndpointSlices list those backend addresses, ports and readiness conditions. A DNS answer identifies an address; it does not establish that HTTP will succeed.

An ordinary ClusterIP Service has a DNS name scoped by namespace. `web.ce-lab20.svc.cluster.local` identifies this fixture in kind's `cluster.local` domain. A wrong namespace in the name can fail resolution while the correct Service IP still works. Diagnose that difference before changing the cluster DNS server.

Service port is the client-facing port; `targetPort` is the backend port. This Service accepts port 80 and forwards to 8080. Changing `targetPort` to 8099 keeps DNS, selectors and Pod readiness intact, but forwards to a port with no server. `containerPort` metadata does not open a socket or rewrite an application's listen port.

Compare HTTP by Service name, Service IP and direct Pod IP from the same client Pod. Name failure with IP success points to resolution or naming. Service failure with direct Pod success points toward Service selection, port mapping or routing; the endpoint and port evidence narrows the cause.

A listener bound to `127.0.0.1` accepts only connections inside that Pod's network namespace. It can answer a `localhost` request from `kubectl exec` while refusing the client Pod's request to its Pod IP. The HTTP readiness probe also uses the Pod IP by default, so this bind fault makes the server unready and removes ordinary Service eligibility.

A timeout is not a diagnosis of a NetworkPolicy or DNS failure. Inspect the layer actually tested. This fixture does not install a NetworkPolicy-enforcing CNI and makes no NetworkPolicy claims. Other real causes include application failure, firewall rules and CNI routing.

Source: `docs/chaos-theory.md`: §§10.7.1-10.7.2.

Main lesson to learn in this lab

Diagnose connectivity one layer at a time: DNS name, Service address and port mapping, then the Pod's listener. Compare name, Service IP and Pod IP from the same client, and inspect endpoints to explain the difference. DNS success does not prove HTTP reachability, and `localhost` success does not prove remote reachability. Repair the layer supported by the evidence, not the timeout symptom alone.

Experiment

Use namespace `ce-lab20` with one Python server and one Python client. Change the requested name, then `targetPort`, then the bind address; restore each fault before the next comparison.

`probe_local` checks `127.0.0.1` inside the server Pod. `wait_service` retries the client's Service request for a bounded recovery check. `EndpointSlice` YAML exposes addresses, ports and ready directly.

Before running: Predict DNS, Service-IP HTTP, direct-Pod HTTP and readiness for the wrong name, wrong `targetPort` and loopback-only listener.

Measurement key

Signal	Meaning
resolve / probe	Helpers run socket DNS lookup or a bounded HTTP request inside the same client Pod. Errors return nonzero and are printed, not counted as healthy HTTP.
Service ports / EndpointSlices	Compare the requested Service port with the endpoint port and readiness. A nonempty endpoint list can still contain the wrong port.
localhost versus Pod IP	Local success proves a local listener; it does not prove that another Pod can reach that listener.
k	kubectl fixed to kind-lab20 and ce-lab20. The client remains unchanged while the server is intentionally recreated for the bind comparison.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 20 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Load the prepared client commands

Run in: VM terminal 1

```
source ~/labs/lab20/exercise.sh
source ~/labs/lab20/helpers.sh
```

Record: No measurement yet. `resolve` and `probe` run inside the client Pod. `wait_endpoint_port` confirms the Service change reached `EndpointSlices`.

Step 2. Measure the healthy request path from the client Pod

Run in: VM terminal 1, same shell

```
SVC_IP=$(k get svc web -o jsonpath='{.spec.clusterIP}')
POD_IP=$(k get pod web -o jsonpath='{.status.podIP}')
printf 'Service IP=%s; Pod IP=%s\n' "$SVC_IP" "$POD_IP"
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k get pod web
k describe svc web
k get endpointslices -l kubernetes.io/service-name=web -o yaml
```

Record: Baseline row: the resolved address, the three HTTP results, Pod readiness, and the endpoint port.

Step 3. Request the wrong namespace name, then retry the correct path

Run in: VM terminal 1, same shell

```
resolve web.ce-lab20-missing.svc.cluster.local
echo "wrong-name DNS exit=$?"
probe http://web.ce-lab20-missing.svc.cluster.local/
echo "wrong-name HTTP exit=$?"
probe "http://$SVC_IP/"
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
```

Record: Wrong namespace name row: the DNS and HTTP errors and exits, and the correct-name recovery result.

Step 4. Change targetPort to 8099 and compare Service with direct HTTP

Run in: VM terminal 1, same shell

```
k patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8099}] '
```

```
wait_endpoint_port 8099
```

```
resolve web.ce-lab20.svc.cluster.local
```

```
k get pod web
```

```
k describe svc web
```

```
k get endpointslices -l kubernetes.io/service-name=web -o yaml
```

```
probe http://web.ce-lab20.svc.cluster.local/
```

```
probe "http://$SVC_IP/"
```

```
probe "http://$POD_IP:8080/"
```

Record: Wrong targetPort row: DNS, Service and direct-Pod HTTP, Pod readiness, and the endpoint port.

Step 5. Restore targetPort to 8080 before the next fault

Run in: VM terminal 1, same shell

```
k patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8080}] '
```

```
wait_endpoint_port 8080
```

```
wait_service
```

```
probe http://web.ce-lab20.svc.cluster.local/
```

```
probe "http://$SVC_IP/"
```

```
probe "http://$POD_IP:8080/"
```

```
k get endpointslices -l kubernetes.io/service-name=web -o yaml
```

Record: Port restored row: the final HTTP results, and the endpoint port and readiness.

Step 6. Replace only the server bind address and compare local with remote

Run in: VM terminal 1, same shell

```
k delete pod web --wait=true --timeout=30s
```

```
k apply -f ~/labs/lab20/loopback.yaml
```

```
k wait pod/web --for=jsonpath='{.status.phase}'=Running --timeout=60s --request-timeout=0
```

```
POD_IP=$(k get pod web -o jsonpath='{.status.podIP}')
```

```
probe_local
```

```
resolve web.ce-lab20.svc.cluster.local
```

```
probe "http://$POD_IP:8080/"
```

```
k describe pod web
```

```
k get endpointslices -l kubernetes.io/service-name=web -o yaml
```

```
probe http://web.ce-lab20.svc.cluster.local/
```

Record: Loopback listener row: the Pod IP, the bind address, the local and remote results, readiness and endpoint readiness.

Step 7. Restore the listener and check every path again

Run in: VM terminal 1, same shell

```
k delete pod web --wait=true --timeout=30s
```

```
k apply -f ~/labs/lab20/server.yaml
```

```
k wait pod/web --for=condition=Ready --timeout=60s --request-timeout=0
```

```
POD_IP=$(k get pod web -o jsonpath='{.status.podIP}')
```

```
wait_service
```

```
resolve web.ce-lab20.svc.cluster.local
```

```
probe http://web.ce-lab20.svc.cluster.local/
```

```
probe "http://$SVC_IP/"
```

```
probe "http://$POD_IP:8080/"
```

```
k describe pod web
```

```
k describe svc web
```

```
k get endpointslices -l kubernetes.io/service-name=web -o yaml
```

Record: Listener restored row: the bind address, the three HTTP results, readiness and endpoint readiness.

Recovery check: DNS and Service HTTP work, targetPort is 8080, and the server listens on 0.0.0.0 with a Ready endpoint.

Write your answer

Use the observations recorded beside each step.

Phase	DNS	Service / Pod HTTP	Ready / endpoint port
Baseline			
Wrong namespace name			
Wrong targetPort			
Port restored			
Loopback listener			
Listener restored			

1. Which path does each probe test: DNS, the Service IP, or the Pod directly?
2. With the wrong namespace name, which requests failed and which still worked?
3. With targetPort 8099, why did direct Pod HTTP work while Service HTTP failed?
4. Why did the loopback listener answer locally but fail readiness and remote requests?

Apply the same reasoning

If DNS works but direct Pod-IP HTTP fails, is changing the Service selector justified?

Save results before removing this lab's resources and files: `./lab.sh 20 reset`

Lab 21 - Configuration freshness and a stalled rollout

Why does a ConfigMap change not reach running Pods, and why can a rollout stall?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 21 setup` on the host. No other lab is required.

Theory you need

A ConfigMap is a separate API object. This application reads its message through an environment variable when its container starts. Updating the ConfigMap does not rewrite the environment of existing processes and does not change the Deployment's Pod template, so it does not itself create a rollout.

A projected ConfigMap volume behaves differently. Ordinary projected files can update eventually, but the application must reread or reload them; a subPath mount does not receive those updates. Do not infer environment refresh from file-projection behavior. This lab tests environment injection specifically.

A Deployment rollout occurs when its Pod template changes. rollout restart changes a template annotation and creates new Pods; they read the current ConfigMap. Record Pod UIDs and the actual HTTP response from each Pod so a mixture of old and new processes cannot hide behind a single sampled response.

This Deployment has two replicas, `maxUnavailable=0` and `maxSurge=1`. During this update it can add one Pod while keeping two replicas available. `minReadySeconds=2` requires a new Pod to remain Ready for two seconds before it counts as available. The strategy governs replacement; a `PodDisruptionBudget` does not control this rollout.

A wrong readiness port can leave a new process Running but unready. It must not replace an available old replica under this strategy. When no rollout progress occurs for the configured 30-second deadline, the Deployment reports `ProgressDeadlineExceeded`. Kubernetes reports the stall; it does not

automatically undo the change.

Rollout history stores Pod-template revisions, not snapshots of separate ConfigMaps. Undo to the recorded good revision repairs the bad readiness port, but does not restore the ConfigMap's old message. Recovery requires checking both template and configuration, then confirming the intended responses and replica counts.

Source: docs/chaos-theory.md: §§10.8.1-10.8.2.

Main lesson to learn in this lab

Updating a ConfigMap does not refresh an existing process's environment or trigger a Deployment rollout. New Pods read the current value, but a readiness failure can stall replacement while this strategy preserves old available replicas. Rollback restores the Pod template, not the separate ConfigMap. Verify configuration, readiness and responses from every replica together before declaring the rollout or recovery complete.

Experiment

Use namespace ce-lab21, two Python HTTP replicas, an environment-backed message and a deliberately wrong readiness port. Query every Pod directly through the API proxy; this measures application responses, not Service routing.

save_revision records the current Deployment revision for the later undo. describe shows the applied template, replica counts and rollout conditions; responses queries every non-terminating consumer.

Before running: Predict Pod UIDs and response values after only the ConfigMap changes, after restart, during the bad probe rollout and after undo.

Measurement key

Signal	Meaning
ConfigMap value / per-Pod HTTP	Compare desired configuration with each running process. An updated object is not proof that every process consumed it.
UID / revision / ReplicaSets	UID changes show replacement. The Deployment revision identifies a Pod template; ReplicaSet desired/current/ready counts show rollout progress.
Available / Progressing	Availability of old replicas and progress of the new template are separate conditions. Both must be interpreted with the replica counts.
readiness port / restartCount	A bad readiness destination can block rollout without crashing or restarting the process.
k / responses	kubectl targets kind-lab21 and ce-lab21. responses prints each Pod name and its HTTP body; a missing or failed query returns nonzero.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 21 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Record the original configuration and every consumer

Run in: VM terminal 1

```
source ~/labs/lab21/exercise.sh
source ~/labs/lab21/helpers.sh
k get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k describe deployment config-web
responses
```

Record: Version one row: the ConfigMap value, each Pod UID and Ready status, the HTTP bodies, the counts and the revision.

Step 2. Change only the ConfigMap and query the same processes

Run in: VM terminal 1, same shell

```
k patch configmap app-config --type=merge -p '{"data":{"message":"version-two"}}'
k get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k describe deployment config-web
responses
```

Record: ConfigMap version two only row: the ConfigMap value, the Pod UIDs, each response, and the revision.

Step 3. Restart consumers and save the good revision for undo

Run in: VM terminal 1, same shell

```
k rollout restart deployment/config-web
k rollout status deployment/config-web --request-timeout=0 --timeout=120s
k get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
k get configmap app-config -o jsonpath='{.data.message}{"\n"}'
responses
save_revision
k rollout history deployment/config-web
k describe deployment config-web
```

Record: Restarted consumers row: the UIDs, each response, the ConfigMap value, the Ready count and the saved revision.

Step 4. Break the new readiness port and collect stalled-rollout evidence

Run in: VM terminal 1, same shell

```
k patch deployment config-web --type=json -p
  [{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","value":9999}]'
k wait deployment/config-web
  --for=jsonpath='{.status.conditions[?(@.type=="Progressing")].reason}'=ProgressDeadlineExceeded
  --timeout=90s --request-timeout=0
k rollout status deployment/config-web --timeout=5s --request-timeout=0
echo "rollout status exit=$?"
k describe deployment config-web
k get rs -l app=config-web
k describe pods -l app=config-web
responses
```

Record: Wrong readiness port row: the rollout result, the replica counts, both conditions, and the direct HTTP body.

Step 5. Undo to the saved template and inspect configuration separately

Run in: VM terminal 1, same shell

```
stable_revision=$(cat ~/labs/lab21/stable-revision.txt)
k rollout undo deployment/config-web --to-revision="$stable_revision"
k rollout status deployment/config-web --request-timeout=0 --timeout=120s
k get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
responses
k describe deployment config-web
```

Record: Undo to stable revision row: the revision, the probe port, the ConfigMap value and the responses.

Step 6. Restore version-one and verify configuration plus all consumers

Run in: VM terminal 1, same shell

```
k patch configmap app-config --type=merge -p '{"data":{"message":"version-one"}}'
k rollout restart deployment/config-web
k rollout status deployment/config-web --request-timeout=0 --timeout=120s
k get configmap app-config -o jsonpath='{.data.message}'
k get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
responses
k describe deployment config-web
```

Record: Version one restored row: the ConfigMap value, the UIDs, both responses, the probe port and the Ready count.

Recovery check: The ConfigMap says version-one, exactly two Pods are Ready, both HTTP responses say version-one, and the readiness port is 8080.

Write your answer

Use the observations recorded beside each step.

Phase	ConfigMap / responses	UIDs / Ready replicas	Revision / conditions
Version one			
ConfigMap version two only			
Restarted consumers			
Wrong readiness port			
Undo to stable revision			
Version one restored			

1. Did changing only the ConfigMap change the running processes' responses, and why?
2. Which operation made the new configuration take effect, and what proves it?
3. Why did the stalled rollout keep the application available?
4. After the undo, what returned to normal and what stayed at version-two?

Apply the same reasoning

Does undoing to the original template revision restore the original ConfigMap value?

Save results before removing this lab's resources and files: ./lab.sh 21 reset

Lab 22 - Orphan reaping and the container process budget

Why can a container run out of processes while almost nothing is running?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 22 setup` on the host. No other lab is required.

Theory you need

A container starts its own PID namespace, so its main process is PID 1 inside it. When a process exits, the kernel keeps its exit status until the parent collects it with `wait`. That entry is a zombie: it retains a PID and kernel bookkeeping but executes no code, and it cannot be killed, because it has already exited.

An orphan is adopted by the nearest ancestor configured as a child subreaper, or otherwise by the PID namespace's PID 1. That process becomes responsible for reaping it. Here PID 1 adopts the orphans directly. The reaper calls `wait` to collect them; the comparison application never waits, so exited children remain zombies.

The comparison is two containers from the same image with the same budget, differing only in PID 1. The first runs an ordinary application, written inline in its docker create command, that sleeps and never waits; the second runs `reaper.py`, which loops on `waitpid(-1)` and prints every PID it collects. Orphans are then produced deliberately, by a maker started with `docker exec` that forks 40 short-lived children and exits first, so PID 1 inherits all 40.

Each later burst asks for live children, 40 and then 200, and kills and reaps its own children before returning, so it leaves no zombies behind. `docker run --init` installs a packaged init (tini) for the reaping job; the explicit reaper is used here so the mechanism stays visible.

`--pids-limit 64` sets `pids.max` in each container's cgroup, and `pids.current` counts every live process and thread in it. Zombie entries count as well. At the ceiling the kernel refuses the next fork with `EAGAIN`, which programs report as `Resource temporarily unavailable`.

That budget is shared by PID 1, the process you start with `docker exec` and every child it makes. The children a burst can create is therefore roughly 64 minus `pids.current` minus the burst process itself, so predict each burst from the count measured just before it.

Read state, not names. State `Z` in `/proc/PID/stat` identifies a zombie and `ppid` identifies who must reap it. Both containers run python, so use `pid1_cmdline` to tell the application from the reaper, and the reaped lines in the log to watch collection happen.

Two causes then produce the identical fork error: zombies holding the budget, or a genuine burst of live processes reaching the ceiling with none. Compare the zombie count with `pids.current` before repairing, because a larger `pids.max` cannot reap anything and an init cannot reduce live concurrency.

Source: docs/chaos-theory.md: §§5.14.1-5.14.2.

Main lesson to learn in this lab

A container can exhaust its task budget through unreaped zombies or too many live processes. Reaping releases exited children's entries; a PID limit only bounds the total and can produce the same fork error for either cause. Compare zombie state with cgroup task counts before choosing a fix: raising the limit does not repair reaping, and adding a reaper does not control live concurrency.

Experiment

Run two containers with the same image and the same 64-PID budget, changing only PID 1. Create inherited orphans in each, then request live processes until forking fails.

start_process_container copies the supplied process tools, starts the named container and waits for its ready log. The visible docker create command sets the same 64-PID limit in each case.

Before running: Predict the zombie count, pids.current and fork outcome for each container after 40 orphans and after each burst.

Measurement key

Signal	Meaning
pid1_comm / pid1_cmdline	Which program is PID 1. Both containers run python, so use the command line to tell the application from the reaper.
zombies / zombies_reparented_to_pid1	Zombie entries counted from /proc. The second count is those whose parent is PID 1, the ones an init must reap.
pids.current / pids.max	Live processes, threads and zombie entries against this container's cgroup ceiling. Equality means the next fork is refused.
created / fork_error	Children started before forking failed, and the errno. EAGAIN is the pids limit refusing a fork, not a memory error.
reaped pid=	The reaper container logs each entry it collects. Absence of these lines in the no-init container is the difference being measured.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 22 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Start the no-init container and record its baseline process table

Run in: VM terminal 1

```
source ~/labs/lab22/exercise.sh
docker create --name ce-lab22-noinit --pids-limit 64 python:3.12-slim python -u -c 'import os, time;
    print(f"application pid={os.getpid()} reaping=no", flush=True); time.sleep(3600)'
```

```
start_process_container ce-lab22-noinit
docker exec ce-lab22-noinit python -u /procreport.py
```

Record: No init baseline row: pid1_comm and pid1_cmdline, the process count, the zombies, and pids.current and pids.max.

Step 2. Inherit forty orphans in the no-init container

Run in: VM terminal 1, same shell

```
docker exec ce-lab22-noinit python -u /orphan-maker.py 40
sleep 2
docker exec ce-lab22-noinit python -u /procreport.py
docker logs ce-lab22-noinit
```

Record: No init after 40 orphans row: orphans_started, the zombie count, zombies_reparented_to_pid1, pids.current, and any reaped lines.

Step 3. Request forty live processes in the same container

Run in: VM terminal 1, same shell

```
docker exec ce-lab22-noinit python -u /fork-burst.py 40
docker exec ce-lab22-noinit python -u /procreport.py
```

Record: No init burst of 40 row: created, fork_error, pids.current and pids.max at the failure, and the zombies left.

Step 4. Start the init container and inherit the same forty orphans

Run in: VM terminal 1, same shell

```
docker create --name ce-lab22-init --pids-limit 64 python:3.12-slim python -u /reaper.py
start_process_container ce-lab22-init
docker exec ce-lab22-init python -u /procreport.py
docker exec ce-lab22-init python -u /orphan-maker.py 40
sleep 2
docker exec ce-lab22-init python -u /procreport.py
docker logs ce-lab22-init --tail 5
```

Record: Init after 40 orphans row: pid1_cmdline, the zombies before and after, pids.current, and the reaped lines.

Step 5. Request forty live processes with a clean process table

Run in: VM terminal 1, same shell

```
docker exec ce-lab22-init python -u /fork-burst.py 40
docker exec ce-lab22-init python -u /procreport.py
```

Record: Init burst of 40 row: created, fork_error, pids.current and pids.max, and the value after reaping.

Step 6. Request two hundred live processes and read the ceiling

Run in: VM terminal 1, same shell

```
docker exec ce-lab22-init python -u /fork-burst.py 200
docker exec ce-lab22-init python -u /procreport.py
nproc
ps -eo pid --no-headers | wc -l
```

Record: Init burst of 200 row: created, fork_error, pids.current and pids.max, the zombies, and the host process count.

Step 7. Remove both containers after recording the evidence

Run in: VM terminal 1, same shell

```
docker rm -f ce-lab22-noinit ce-lab22-init
docker ps -a --filter 'name=~ce-lab22-' --format '{{.Names}}'
ps -eo pid --no-headers | wc -l
```

Record: Whether any ce-lab22 container remains, and the final host process count.

Recovery check: Both experiment containers were removed after their evidence was recorded. The process limit was local to each container; the VM remained usable.

Write your answer

Use the observations recorded beside each step.

Case	PID 1 / zombies	pids.current / pids.max	Fork result
No init baseline			
No init after 40 orphans			
No init burst of 40			
Init after 40 orphans			
Init burst of 40			
Init burst of 200			

1. Which process removes the zombie entries, and what did each container do with them?
2. Why did the burst of 40 fail in the no-init container?
3. Why did the burst of 200 fail with the same error but no zombies?
4. What did the pids limit contain, and what did it not repair?

Apply the same reasoning

`pids.current` equals `pids.max` with zero zombies. Is a larger limit the fix?

Save results before removing this lab's resources and files: `./lab.sh 22 reset`

Lab 23 - Endpoint removal, preStop and the grace budget

Which requests fail when one replica is deleted, and what actually repairs them?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 23 setup` on the host. No other lab is required.

Theory you need

Deleting a Pod starts two independent sequences at the same time. The API server records a deletion timestamp; the endpoint controller marks that address not ready in its `EndpointSlice` and every kube-proxy must observe the change and rewrite its rules. In parallel the kubelet runs any `preStop` hook and sends `SIGTERM`. Nothing synchronizes the two paths.

The in-flight count at `SIGTERM` tells you how many requests remain unfinished when shutdown begins. It does not prove that new requests arrived after deletion. Compare it with the endpoint timeline and client results: graceful shutdown can finish existing work, while abrupt exit drops that work.

`EndpointSlice` conditions describe the address during termination. `ready` becomes false almost immediately, `terminating` becomes true, and `serving` can remain true, so a going-away Pod is distinguishable from one that failed its probe. Readiness gates new traffic; it does not drain traffic already routed, and an established connection survives a rule change.

Two replicas stand behind one ClusterIP Service in namespace `ce-lab23`. Every request to `/` occupies the server for two seconds while the readiness path `/healthz` answers at once, and the client sends 25 requests per second for 14 seconds, so about fifty are in flight at any moment. One replica is deleted about four seconds in, while the survivor keeps answering.

The same image provides both application behaviours. `SHUTDOWN=graceful` stops accepting, finishes its in-flight requests and logs exited cleanly. `SHUTDOWN=abrupt` logs the same in-flight count and leaves immediately with status 143. The container is PID 1, where the kernel discards a signal left at its default disposition, so this fixture exits explicitly instead.

A `preStop` hook delays the signal without delaying endpoint removal, so sleeping keeps the server accepting while the withdrawal propagates and the in-flight count at `SIGTERM` falls towards zero. The hook is an ordinary command in the container, and `sleep 5` means five seconds of continued service; the application is not changed.

`terminationGracePeriodSeconds` covers `preStop` and application shutdown together. A hook that outlasts the budget leaves no planned time for cleanup; Kubernetes allows a small one-off extension of two seconds before forced termination. Measure signal timing and failures rather than assuming an exact one-second cutoff. A missing `SIGTERM` log is missing evidence.

Source: docs/chaos-theory.md: §§10.9.1-10.9.2.

Main lesson to learn in this lab

Pod deletion starts traffic withdrawal and application shutdown asynchronously. Graceful shutdown finishes accepted work; preStop can keep the server accepting while endpoint withdrawal propagates. Both the hook and cleanup share one grace budget. Compare client failures, endpoint changes and shutdown logs; unfinished work at SIGTERM does not establish when it arrived. Allow time for withdrawal and cleanup, and verify that requests actually finish.

Experiment

Use namespace ce-lab23 with two replicas and one client. Delete one replica under a steady 25 requests per second four times, changing only the shutdown behaviour, then only the hook, then only the budget.

shutdown_settings prints replicas, grace seconds, lifecycle hooks and the SHUTDOWN setting from the current Deployment. The manifest named in each apply command changes the one variable described in that step.

Before running: Predict the in-flight count at SIGTERM and the failed-request count for the graceful, abrupt, preStop and short-budget deletions.

Measurement key

Signal	Meaning
sigterm received inflight=N shutdown=MODE	Requests still unfinished at SIGTERM. This does not by itself prove when endpoint routing stopped.
exited cleanly	Printed when the graceful mode has finished its in-flight requests. The abrupt mode exits with 143 instead and never logs it.
ready / serving / terminating	EndpointSlice conditions for each backend address, sampled twice a second, including the replacement Pod as it becomes ready.
ok / failed per second	Outcomes bucketed by the second each request was sent; the deletion is at t=4s. Any non-200 or transport error is a failure.
k / drain_case	kubectl fixed to kind-lab23 and ce-lab23. drain_case NAME runs one whole case: settle, probe, delete at t=4s, sample endpoints, then print the timeline, probe result and server log.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 23 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Load the helpers and measure the fixture without any fault

Run in: VM terminal 1

```
source ~/labs/lab23/exercise.sh
source ~/labs/lab23/helpers.sh
shutdown_settings
k get pods -l app=drain-web -o
  custom-columns='NAME:.metadata.name,READY:.status.conditions[?(@.type=="Ready")].status,NODE:.spec.nodeName'
endpoint_conditions
k exec client -- python -u /scripts/drain-probe.py http://web/ 6 25
```

Record: Baseline without deletion row: the replica count, the grace budget, the shutdown mode, the endpoint conditions and the probe totals.

Step 2. Delete one replica while the application drains its requests

Run in: VM terminal 1, same shell

```
drain_case graceful
```

Record: Graceful without hook row: the deletion time, the in-flight count at SIGTERM, the sampled endpoint conditions, and the failures.

Step 3. Change only the shutdown behaviour and repeat the same deletion

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab23/web-abrupt.yaml
k rollout status deployment/drain-web --request-timeout=0 --timeout=120s
shutdown_settings
drain_case abrupt
```

Record: Abrupt without hook row: the shutdown mode, the in-flight count at SIGTERM, and the failed requests and their second.

Step 4. Add only the preStop hook and repeat again

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab23/web-prestop.yaml
k rollout status deployment/drain-web --request-timeout=0 --timeout=120s
shutdown_settings
drain_case prestop
```

Record: Abrupt with preStop 5 s row: the hook, the in-flight count at SIGTERM, the deletion-to-SIGTERM delay and the failures.

Step 5. Keep the hook and shorten only the grace budget

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab23/web-shortgrace.yaml
k rollout status deployment/drain-web --request-timeout=0 --timeout=120s
shutdown_settings
drain_case short-grace
```

Record: preStop 5 s with 1 s budget row: the deletion and SIGTERM times, the in-flight count then, and the failures.

Step 6. Restore the base template and re-measure without a deletion

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab23/web.yaml
k rollout status deployment/drain-web --request-timeout=0 --timeout=120s
wait_settled
shutdown_settings
endpoint_conditions
k exec client -- python -u /scripts/drain-probe.py http://web/ 6 25
```

Record: Template restored row: the replica count, the grace budget, the shutdown mode, the endpoint conditions and the probe totals.

Recovery check: The base template is restored, both replicas are Ready and serving, and a probe without any deletion records no failed requests.

Write your answer

Use the observations recorded beside each step.

Case	inflight at SIGTERM	Endpoint conditions after deletion	Failed requests and second
Baseline without deletion			
Graceful without hook			
Abrupt without hook			

Case	inflight at SIGTERM	Endpoint conditions after deletion	Failed requests and second
Abrupt with preStop 5 s			
preStop 5 s with 1 s budget			
Template restored			

1. What does a baseline with zero failures establish before any fault is injected?
2. How many requests were unfinished at SIGTERM, and did graceful shutdown complete them?
3. With abrupt shutdown, how many requests failed, and how does that match the in-flight count?
4. What did the preStop hook change, given that the application did not change?
5. With a 1-second budget, when did SIGTERM arrive, and what does that show?

Apply the same reasoning

With preStop sleep 5 and a 5-second budget, what is left for the shutdown?

Save results before removing this lab's resources and files: `./lab.sh 23 reset`

Lab 24 - Search paths, ndots and a resolver outage

What does one name lookup cost inside a Pod, and what survives a DNS outage?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 24 setup` on the host. Outbound DNS from the VM is required for the external-name comparison. No other lab is required.

Theory you need

The kubelet writes each Pod's `/etc/resolv.conf`. With `dnsPolicy ClusterFirst` it lists the cluster DNS service address, a search list beginning with `NAMESPACE.svc.cluster.local`, `svc.cluster.local` and `cluster.local`, and the option `ndots:5`. Read that file in the Pod you are measuring; its first search entry depends on the namespace.

The glibc resolver applies one rule to it. A name containing at least `ndots` dots is tried as written first; a name with fewer dots is tried against every search entry first. Either order falls back to the remaining candidates when an attempt fails, so a name that resolves nowhere is tried against the whole list regardless of `ndots`. A trailing dot skips the list.

That gives the arithmetic this lab asks you to calculate. Each candidate normally costs two queries, one for A and one for AAAA, although an image that skips AAAA costs one; measure that constant first with a name that answers on its first candidate. Dividing any later count by it gives the candidates actually tried.

A lookup that succeeds costs the position of its first answering candidate, times that constant. A lookup that fails everywhere costs the number of search entries plus one, times the same constant. The ratio between those two is the amplification you are asked to report.

The fixture is one Pod named `web` behind a ClusterIP Service in namespace `ce-lab24`, with two identical clients that differ in one field: `client` keeps the default `ndots:5` and `client-ndots` sets `ndots` to 1 through `dnsConfig`. You measure the short Service name, its absolute form, an external name that exists and a name that exists nowhere.

Counting uses CoreDNS's own metrics. Each replica exposes `coredns_dns_requests_total` on port 9153, and the helper sums that counter across replicas immediately before and after one lookup. CoreDNS answers `cluster.local` itself and forwards anything else to the node's resolver, which is why an external

name can answer at all.

The counter records queries arriving at CoreDNS, including those answered from its cache, so it measures what the client asked for rather than upstream traffic. Only these client Pods query this cluster's DNS, so re-measure a small unexplained difference rather than explaining it away. A count reported as unavailable means the counter itself could not be read.

Cluster DNS is a dependency of an application, not part of the Service data path. Scaling CoreDNS to zero leaves the ClusterIP, its EndpointSlice and kube-proxy's rules untouched, so a request to a known address still succeeds while every request by name fails, and kubectl keeps working through the API server address. Restoring replicas is not instantaneous either: endpoints must be repopulated and each node's rules rewritten first.

The failure symptom depends on the network, so record which one you saw. A Service without endpoints is normally rejected at once, giving a fast error, while a dropped packet produces the resolver's own timeout of several seconds per attempt, multiplied by the candidates. A name failure is a resolution result, not proof that the destination application is down.

Source: docs/chaos-theory.md: §§10.10.1-10.10.2.

Main lesson to learn in this lab

With glibc, search paths and ndots determine candidate order; each candidate can generate A and AAAA queries. A trailing dot avoids search expansion, but lowering ndots may not reduce queries when every candidate fails. Count requests reaching CoreDNS, including cache hits, then compare name and known-IP requests during an outage. Diagnose the resolver separately from the application's network path.

Experiment

Use namespace ce-lab24 with one Service and two clients that differ only in ndots. Count CoreDNS queries for in-cluster, external, absolute and absent names, then remove the resolver and compare paths.

wait_backend_removed waits for an empty backend set; wait_backend_ready waits for a ready endpoint. Both print EndpointSlice evidence and return an error if the expected state is not observed within 30 checks.

Before running: Predict the query count for an in-cluster name, an external name with and without a trailing dot, the same name under ndots 1, and an absent name.

Measurement key

Signal	Meaning
ndots / search	From the measured Pod's /etc/resolv.conf. The number of search entries sets how many candidates a non-absolute name can try.
queries=N	CoreDNS requests for one lookup, summed across replicas. Divide by the per-candidate cost measured in the baseline to get the candidates tried.
elapsed_ms	Wall-clock cost of the lookup inside the Pod, including every candidate tried. Compare with the query count rather than reading it alone.
addresses / error	The resolved addresses, or the resolver error. A failure after the full search is a naming result, not proof the destination is down.
k / ksys / dns_count	kubectl fixed to kind-lab24, for ce-lab24 and for kube-system. dns_count POD NAME... prints the measured cost of each lookup.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 24 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Read both resolver configurations and the CoreDNS inventory

Run in: VM terminal 1

```
source ~/labs/lab24/exercise.sh
source ~/labs/lab24/helpers.sh
k exec client -- cat /etc/resolv.conf
k exec client-ndots -- cat /etc/resolv.conf
ksys get deployment coredns -o jsonpath='{.spec.replicas}{"\n"}'
ksys get pods -l k8s-app=kube-dns -o
    custom-columns='NAME:.metadata.name,IP:.status.podIP,READY:.status.conditions[?(@.type=="Ready")].status'
coredns_endpoints
SVC_IP=$(k get svc web -o jsonpath='{.spec.clusterIP}')
printf 'Service IP=%s\n' "$SVC_IP"
```

Record: Each client's nameserver, ndots and search list, the CoreDNS replicas and addresses, and the Service IP.

Step 2. Measure the in-cluster name in both forms

Run in: VM terminal 1, same shell

```
dns_count client web web.ce-lab24.svc.cluster.local.
probe client http://web/
```

Record: In-cluster short and absolute names rows: the queries, elapsed_ms, addresses and HTTP result. This count is your per-candidate cost.

Step 3. Measure an external name three ways

Run in: VM terminal 1, same shell

```
dns_count client dl.k8s.io
dns_count client-ndots dl.k8s.io
dns_count client-ndots dl.k8s.io
```

Record: External name rows, default ndots, trailing dot and ndots 1: the queries, elapsed_ms and result for each.

Step 4. Measure a name that resolves nowhere from both clients

Run in: VM terminal 1, same shell

```
dns_count client chaos-lab24-absent.example
dns_count client-ndots chaos-lab24-absent.example
```

Record: Absent name from both clients row: the queries, elapsed_ms and the resolver error for each client.

Step 5. Remove the cluster resolver and compare the three paths

Run in: VM terminal 1, same shell

```
original=$(ksys get deployment coredns -o jsonpath='{.spec.replicas}')
printf '%s\n' "$original" | tee ~/labs/lab24/coredns-replicas.txt
ksys scale deployment coredns --replicas=0
wait_backend_removed
dns_count client web
probe client "http://$SVC_IP/"
probe client http://web/
k get pods -o name
```

Record: CoreDNS scaled to zero row: the lookup error and elapsed time, both HTTP results, and whether kubectl answered.

Step 6. Restore the resolver and prove recovery by name

Run in: VM terminal 1, same shell

```
ksys scale deployment coredns --replicas="$(cat ~/labs/lab24/coredns-replicas.txt)"
ksys rollout status deployment/coredns --request-timeout=0 --timeout=120s
wait_backend_ready
for i in $(seq 1 15); do probe client http://web/ >/dev/null 2>&1 && break; sleep 1; done
ksys get pods -l k8s-app=kube-dns -o
    custom-columns='NAME:.metadata.name,IP:.status.podIP,READY:.status.conditions[?(@.type=="Ready")].status'
dns_count client web
probe client http://web/
```

Record: CoreDNS restored row: the replica count and readiness, the endpoint addresses, the queries and elapsed time, and the HTTP result.

Recovery check: CoreDNS is restored to its original replica count, the Service name resolves from the default client, and an HTTP request by name returns 200.

Write your answer

Use the observations recorded beside each step.

Case	Client and ndots	Queries counted	Elapsed and result
In-cluster short and absolute names			
External name default ndots			
External name trailing dot			
External name ndots 1			
Absent name from both clients			
CoreDNS scaled to zero			
CoreDNS restored			

- 1. How many candidates did the short name and the absolute form each try?
- 2. How many queries did the external name cost with ndots 5, a trailing dot, and ndots 1?
- 3. Why does lowering ndots not reduce the cost of a name that resolves nowhere?
- 4. With CoreDNS at zero, what still worked, and what proves the resolver recovered?

Apply the same reasoning

Does setting ndots to 1 reduce the cost of a hostname that no longer exists?

Save results before removing this lab's resources and files: ./lab.sh 24 reset

Lab 25 - Admission control as a synchronous dependency

What happens to a cluster when the webhook that approves new Pods stops answering?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 25 setup` on the host. No other lab is required.

Theory you need

Every write to the API server passes through authentication, authorization, mutating admission, schema validation and validating admission before it is persisted. A `ValidatingWebhookConfiguration` inserts an HTTP call to a service of your own into that path. The call is synchronous: the creating request waits for the answer, bounded by `timeoutSeconds`, which is five seconds here.

`failurePolicy` decides what no answer means. `Fail` turns an unreachable or timed-out webhook into a rejected request, so an outage of the webhook becomes an outage of object creation. `Ignore` lets the request continue without this check, so the same outage becomes a gap in enforcement. Other checks still apply. An explicit rejection from a reachable webhook is enforced with either value.

`Scope` decides the blast radius. This webhook selects only namespaces labelled `ce-lab25=scoped`, with a rule limited to `CREATE` on Pods, so unlabelled namespaces are never consulted and deletions, Deployment writes and other operations are never intercepted.

A webhook that matches every namespace, including its own, can block the creation of the very Pods that would restore it. That is why this backend lives in a namespace the webhook does not select, and why a single backend replica makes `failurePolicy` a single point of failure.

The fixture is therefore two namespaces. `ce-lab25` carries the `scoped` label and holds the workloads you create. `ce-lab25-system` is unlabelled and holds the backend: one Deployment named `admission` with a single replica, and a Service publishing port 443 to the container's 8443. The rule enforced is that a Pod must carry an owner label.

Rejection appears where the request was made, not necessarily where you are looking. Creating a Pod directly returns the webhook error to `kubectl`. A Deployment is accepted, because a Deployment is not a Pod; its `ReplicaSet` then retries Pod creation, records `FailedCreate` events and leaves the Deployment reporting `ReplicaFailure` with zero ready replicas.

The outage is injected by scaling the backend to zero, which empties the Service's endpoints. A Service without endpoints is normally refused at once, so the create fails in a fraction of a second rather than spending the five-second timeout; that timeout is what a slow webhook would consume. Recovery lags too, until each node's rules allow the call again.

The API server calls the webhook over HTTPS and verifies its certificate against the `caBundle` in the configuration. Setup generates one self-signed certificate for `admission.ce-lab25-system.svc` and installs it in both places, so a failure here comes from the injected fault rather than from trust. Server-side dry run also invokes the webhook, which tests enforcement without creating anything.

Source: docs/chaos-theory.md: §§12.4.1-12.4.2.

Main lesson to learn in this lab

An admission webhook is a synchronous dependency of the requests it matches. During a call failure, `Fail` blocks those writes and `Ignore` skips that check; `Ignore` does not override an explicit policy rejection. `Scope` limits the blast radius, and an accepted Deployment can still fail to create Pods. Check controller events and restore both backend availability and enforcement before calling recovery complete.

Experiment

Use the scoped namespace ce-lab25 and the unscoped backend namespace ce-lab25-system. Remove the webhook backend, observe direct creation, controller-driven creation and an unscoped namespace, then compare failurePolicy values before recovering.

wait_backend_removed waits for an empty backend set; wait_backend_ready waits for a ready endpoint. Both print EndpointSlice evidence and return an error if the expected state is not observed within 30 checks.

Before running: Predict the result of creating a labelled Pod, a Deployment and an unscoped Pod while the backend is unavailable, and what changes under failurePolicy Ignore.

Measurement key

Signal	Meaning
failurePolicy / timeoutSeconds / namespaceSelector	The configured behaviour on no answer, the wait before that decision, and which namespaces are consulted at all.
admission error text	A rejection naming the webhook is a dependency failure; a rejection carrying the policy message is enforcement working as designed.
ReplicaSet events	FailedCreate entries record rejections the controller received. The Deployment object itself can be accepted while no Pod exists.
review ... allowed=	The backend logs one line per admission review. No line means the request never reached the webhook.
k / ksys	kubectl fixed to kind-lab25 for the scoped namespace ce-lab25 and for the unscoped backend namespace ce-lab25-system.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 25 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Record the configured dependency and prove enforcement

Run in: VM terminal 1

```
source ~/labs/lab25/exercise.sh
source ~/labs/lab25/helpers.sh
webhook_state
backend_state
k apply -f ~/labs/lab25/labelled-pod.yaml
k wait pod/labelled --for=condition=Ready --timeout=60s --request-timeout=0
k apply -f ~/labs/lab25/unlabelled-pod.yaml
echo "unlabelled create exit=$?"
k get pods -o
  custom-columns='NAME:.metadata.name,OWNER:.metadata.labels.owner,READY:.status.conditions[?(@.type=="Ready")
].status'
ksys logs deployment/admission --tail=10
```

Record: Baseline with backend available row: the policy, timeout and selector, the backend endpoint, both create results and the review lines.

Step 2. Remove the backend and create the same labelled Pod again

Run in: VM terminal 1, same shell

```
k delete pod labelled --wait=true --timeout=30s
ksys scale deployment admission --replicas=0
wait_backend_removed
backend_state
time k apply -f ~/labs/lab25/labelled-pod.yaml
echo "labelled create exit=$?"
k get pods
ksys logs deployment/admission --tail=5 --pod-running-timeout=5s
echo "backend log exit=$?"
```

Record: Backend scaled to zero row: the exact error, the elapsed time, the exit status, and whether any Pod exists.

Step 3. Compare a controller-driven creation with an unscoped namespace

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab25/workload.yaml
echo "deployment apply exit=$?"
sleep 5
k describe deployment policy-web
k get rs -l app=policy-web
k get events --field-selector reason=FailedCreate
ksys apply -f ~/labs/lab25/system-pod.yaml
echo "unscoped create exit=$?"
ksys get pod unscoped -o custom-columns='NAME:.metadata.name,PHASE:.status.phase'
ksys get namespace ce-lab25 ce-lab25-system -o
    custom-columns='NAME:.metadata.name,SCOPED:.metadata.labels.ce-lab25'
```

Record: Deployment in scoped namespace and Pod in unscoped namespace rows: both apply results, the Deployment counts and conditions, and the FailedCreate message.

Step 4. Change only the failure policy and repeat the forbidden creation

Run in: VM terminal 1, same shell

```
k patch validatingwebhookconfiguration ce-lab25-pods --type=json -p
    '[{"op":"replace","path":"/webhooks/0/failurePolicy","value":"Ignore"}]'
webhook_state
k apply -f ~/labs/lab25/unlabelled-pod.yaml
echo "unlabelled create exit=$?"
sleep 5
k get pods -o custom-columns='NAME:.metadata.name,OWNER:.metadata.labels.owner,PHASE:.status.phase'
k describe deployment policy-web
```

Record: failurePolicy Ignore row: the new policy value, the result of creating the forbidden Pod, the Pod list and the Deployment counts.

Step 5. Restore the backend, the Fail policy and enforcement

Run in: VM terminal 1, same shell

```
k delete pod unlabelled --wait=true --timeout=30s
ksys scale deployment admission --replicas=1
ksys rollout status deployment/admission --request-timeout=0 --timeout=120s
wait_backend_ready
k patch validatingwebhookconfiguration ce-lab25-pods --type=json -p
    '[{"op":"replace","path":"/webhooks/0/failurePolicy","value":"Fail"}]'
for i in $(seq 1 30); do
    k apply -f ~/labs/lab25/labelled-pod.yaml --dry-run=server >/dev/null 2>&1 && break
    sleep 2
done
webhook_state
backend_state
k rollout status deployment/policy-web --request-timeout=0 --timeout=120s
k apply -f ~/labs/lab25/labelled-pod.yaml
k wait pod/labelled --for=condition=Ready --timeout=60s --request-timeout=0
k apply -f ~/labs/lab25/unlabelled-pod.yaml --dry-run=server
echo "dry-run exit=$?"
ksys logs deployment/admission --tail=10
```

Record: Backend and Fail restored row: the endpoint, the policy, the ready count, the labelled create result and the dry-run rejection.

Step 6. Remove the experiment objects and confirm the final state

```
Run in: VM terminal 1, same shell
k delete deployment policy-web --wait=true --timeout=60s
k delete pod labelled --wait=true --timeout=30s
ksys delete pod unscoped --wait=true --timeout=30s
k get pods
ksys get pods
webhook_state
```

Record: The remaining Pods in both namespaces and the final webhook configuration.

Recovery check: The backend has one ready endpoint, failurePolicy is Fail, and a server-side dry run of an unlabelled Pod is still rejected by the policy.

Write your answer

Use the observations recorded beside each step.

Case	Pod create result	Controller or namespace effect	Webhook log evidence
Baseline with backend available			
Backend scaled to zero			
Deployment in scoped namespace			
Pod in unscoped namespace			
failurePolicy Ignore			
Backend and Fail restored			

1. Which baseline rejection is the policy working, and which would mean the dependency failed?
2. Why did a correctly labelled Pod fail once the backend was gone?
3. Where did the failure surface for the Deployment, and what limited the blast radius?
4. What did failurePolicy Ignore allow, and what proves enforcement works again?

Apply the same reasoning

If the webhook selected its own namespace, could scaling the backend restore admission?

Save results before removing this lab's resources and files: ./lab.sh 25 reset

Lab 26 - Container privilege boundaries and hardening

How much authority does this container actually have, and which flag granted it?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 26 setup` on the host. No other lab is required.

Theory you need

A container is a process with several independent boundaries around it. Docker's default gives it root inside the container but only a bounded capability set, an active seccomp filter, its own PID namespace and a small `/dev`. None of that is visible from the image; it comes from how the container was started.

Read the boundary from the process, not from the command line. `/proc/self/status` reports `CapEff` as a hexadecimal bitmask of effective capabilities, `NoNewPrivs` as 0 or 1, and `Seccomp` as 0 for no filter or 2 for an active one. `/proc` lists exactly the processes this container can see, and `/dev` lists the devices it was given.

Each flag widens one boundary. `--pid=host` removes the PID namespace, so the container sees the VM's processes, while its capabilities stay exactly as they were. `--privileged` is not one setting: it grants the full capability set, disables the seccomp filter and exposes host devices at the same time, which is why it is rarely the minimum change.

The hardening controls are equally distinct. `--user` changes the identity only; `--cap-drop ALL` empties the capability sets, so privileged operations fail with `EPERM` even for UID 0; `--security-opt no-new-privileges` sets `NoNewPrivs=1`, so a set-user-ID binary cannot raise privileges later; `--read-only` makes the image layer immutable, which is why this case adds `--tmpfs /tmp` for paths that must stay writable.

Compare what was requested with what took effect. `docker inspect` shows the configuration; `/proc/self/status` shows the result. A container whose `CapEff` is still full has not been hardened, whatever its command line says. Reducing authority also does not repair an application that genuinely needs a capability, so check each removal against what the workload does.

Source: docs/chaos-theory.md: §§5.15.1-5.15.2.

Main lesson to learn in this lab

Container authority comes from independent runtime controls, not the image or the word root alone. Host PID visibility does not grant extra capabilities, while privileged mode widens several boundaries together. Compare requested settings with effective identity, capabilities, seccomp, process visibility and mounts; remove only the authority the workload does not need, and verify that its required operations still work.

Experiment

Run the same `python:3.12-slim` image as a default container, with `--pid=host`, with `--privileged` and hardened, recording identity, capabilities, visibility, devices and write access for each.

Before running: Predict uid, `CapEff`, `NoNewPrivs`, `Seccomp`, visible processes and the two probes for the default, host-PID, privileged and hardened containers.

Measurement key

Signal	Meaning
uid / gid	The numeric identity the process runs as inside the container. Root here is still root against the kernel unless capabilities are removed.
CapEff / CapBnd	Effective and bounding capability bitmasks from /proc/self/status. Zero means no capability; the default set and the full set are different nonzero values.
NoNewPrivs / Seccomp	NoNewPrivs=1 blocks later privilege gain through set-user-ID binaries. Seccomp=2 means a syscall filter is active and 0 means none is.
visible_pids / pid1_cmdline	How many processes this container sees, and what its PID 1 is. Seeing the VM's init means the PID namespace boundary is gone.
write_root / capability_check	A write to / shows whether the root filesystem is read-only; the chown probe shows whether CAP_CHOWN is available. Both print the refusal.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 26 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Run the default container and record its boundaries

Run in: VM terminal 1

```
docker run --name ce-lab26-default -v ~/labs/lab26/report.py:/report.py:ro python:3.12-slim python -u /report.py
docker inspect -f 'User={{.Config.User}} Privileged={{.HostConfig.Privileged}} PidMode={{.HostConfig.PidMode}}
  ReadonlyRootfs={{.HostConfig.ReadonlyRootfs}} CapDrop={{.HostConfig.CapDrop}}' ce-lab26-default
```

Record: Default row: uid and gid, CapEff, CapBnd, NoNewPrivs, Seccomp, visible_pids, pid1_cmdline, dev_entries, both probes and the inspected configuration.

Step 2. Remove only the PID namespace boundary

Run in: VM terminal 1, same shell

```
docker run --name ce-lab26-hostpid --pid=host -v ~/labs/lab26/report.py:/report.py:ro python:3.12-slim python
-u /report.py
docker inspect -f 'PidMode={{.HostConfig.PidMode}} Privileged={{.HostConfig.Privileged}}' ce-lab26-hostpid
```

Record: Host PID namespace row: visible_pids, pid1_cmdline, CapEff, Seccomp and the device count. Mark every value identical to the default.

Step 3. Run the same image privileged

Run in: VM terminal 1, same shell

```
docker run --name ce-lab26-privileged --privileged -v ~/labs/lab26/report.py:/report.py:ro python:3.12-slim
python -u /report.py
docker inspect -f 'Privileged={{.HostConfig.Privileged}} SecurityOpt={{.HostConfig.SecurityOpt}}'
ce-lab26-privileged
```

Record: Privileged row: CapEff, CapBnd, Seccomp, dev_entries, the block devices found and both probes.

Step 4. Run the hardened container and check the controls took effect

Run in: VM terminal 1, same shell

```
docker run --name ce-lab26-hardened --user 10001:10001 --cap-drop ALL --security-opt no-new-privileges
--read-only --tmpfs /tmp -v ~/labs/lab26/report.py:/report.py:ro python:3.12-slim python -u /report.py
docker inspect -f 'User={{.Config.User}} CapDrop={{.HostConfig.CapDrop}}
  ReadonlyRootfs={{.HostConfig.ReadonlyRootfs}} SecurityOpt={{.HostConfig.SecurityOpt}}
  Tmpfs={{.HostConfig.Tmpfs}}' ce-lab26-hardened
```

Record: Hardened row: uid and gid, CapEff, NoNewPrivs, Seccomp, write_root, write_tmp, capability_check and the inspected settings.

Step 5. Collect the four results and remove the containers

```
Run in: VM terminal 1, same shell

for name in default hostpid privileged hardened; do
  printf '== %s\n' "$name"
  docker logs "ce-lab26-$name"
done
docker rm -f ce-lab26-default ce-lab26-hostpid ce-lab26-privileged ce-lab26-hardened
docker ps -a --filter 'name=/ce-lab26-' --format '{{.Names}}'
```

Record: The four cases side by side, and whether any ce-lab26 container remains.

Recovery check: All four experiment containers were removed after their evidence was recorded, and the report program remains in the lab workspace.

Write your answer

Use the observations recorded beside each step.

Case	uid / CapEff	NoNewPrivs / Seccomp	Visibility / devices / probes
Default			
Host PID namespace			
Privileged			
Hardened			

- 1. Which boundaries is the default container already relying on?
- 2. With --pid=host, what changed and what stayed the same?
- 3. Which three boundaries did --privileged change at once?
- 4. Which flag produced each difference in the hardened container?

Apply the same reasoning

A team fixes a permission error with --privileged. Which evidence finds the minimum change?

Save results before removing this lab's resources and files: ./lab.sh 26 reset

Lab 27 - Reading a cluster against the CIS benchmark

Which benchmark controls does an untouched cluster fail, and what does remediating one actually cost?

Before you run: Provision the shared VM once with ./lab.sh provision. Run ./lab.sh 27 setup on the host. No other lab is required.

Theory you need

The CIS Kubernetes Benchmark is a numbered list of configuration checks, each with an expected value and a remediation. Nothing in the cluster enforces it. A scan reports the difference between what is configured and what the benchmark expects, and every finding still has to be judged against what the cluster is for.

The settings it checks live in different places, so one scan reads several interfaces. Group 1.1 is file permissions on the control-plane node. Group 1.2 is API server flags. Group 4.2 is the kubelet's own configuration. Group 5 is ordinary RBAC objects.

This lab checks six controls with a small script. 1.1.1 and 1.1.13 stat two files inside the node container. 1.2.1 and 1.2.21 read the flags the API server was started with. 4.2.1 reads the kubelet's running configuration through its configz endpoint. 5.1.1 lists every subject bound to cluster-admin.

A flag in a file is a request; the flag the process was started with is the configuration. The scan reads the API server's command from its mirror Pod rather than from the manifest on disk, so a manifest edit the kubelet has not applied yet cannot be mistaken for a remediated control.

A kubeadm-built cluster passes some controls and fails others before anyone touches it: --anonymous-auth and --profiling are simply not set, and both defaults are permissive. Freshly installed and benchmarked are different states, which is the first measurement this lab takes.

Remediating a control-plane flag changes the static Pod manifest on the node. The kubelet notices and restarts that component, so the API server can be briefly unavailable. Wait for /readyz and the changed mirror-Pod command before scanning again. Time until these checks pass includes detection and startup; it is not a measurement of the outage duration.

Each change here is reverted, so the cluster ends where it started. That also keeps the comparison honest, because the only thing that moved a control was the change made immediately before it.

Source: docs/chaos-theory.md: §§12.6.1-12.6.2.

Main lesson to learn in this lab

A benchmark finding is a comparison with a configuration rule, not enforcement or proof of overall security. Inspect the relevant effective setting, change one control, and rescan; editing a static Pod manifest can restart the API server before the new setting takes effect. Confirm both the intended configuration and recovery, and limit conclusions to the six controls actually checked here.

Experiment

Scan the lab27 cluster, then change one file mode, one API server flag and one RBAC binding, re-scanning after each change and after restoring it.

Setup supplies two API server manifests that differ only by --profiling=false. install_apiserver copies the chosen file into the lab node and atomically installs it. wait_profiling checks readiness plus the applied mirror-Pod argument; it reports convergence time, not measured downtime.

Before running: Predict which of the six controls an untouched cluster fails, and what each of the three changes does to the scan.

Measurement key

Signal	Meaning
PASS / FAIL	One line per control with its number, expected value and observed value. The observed value is what makes a finding actionable.
unset (defaults to true)	The flag is absent, so the permissive default applies. An absent setting is a finding, not a missing measurement.
scan failed	The scan could not read something, usually because the API server is restarting. That is an error, not a failed control; wait and repeat it.
readyz	The API server's readiness endpoint. Use it to tell a restarting control plane from a broken one before drawing conclusions.
k / cis_scan	kubectl fixed to kind-lab27, and the scan helper that runs the six checks against this cluster.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 27 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Scan the untouched cluster

Run in: VM terminal 1

```
source ~/labs/lab27/exercise.sh
source ~/labs/lab27/helpers.sh
cis_scan | tee ~/labs/lab27/baseline.txt
```

Record: Baseline row: every control with PASS or FAIL, the observed value of each failure, and the totals. Keep baseline.txt.

Step 2. Change one file mode and scan again

Run in: VM terminal 1, same shell

```
docker exec lab27-control-plane chmod 644 /etc/kubernetes/admin.conf
docker exec lab27-control-plane stat -c '%n %a %U:%G' /etc/kubernetes/admin.conf
cis_scan
```

Record: admin.conf mode 644 row: the mode reported by stat, which control changed and its observed value.

Step 3. Restore the file mode

Run in: VM terminal 1, same shell

```
docker exec lab27-control-plane chmod 600 /etc/kubernetes/admin.conf
cis_scan
```

Record: admin.conf restored row: the observed value and result for both file controls.

Step 4. Apply the profiling flag and observe control-plane convergence

Run in: VM terminal 1, same shell

```
# The diff shows the single added argument; exit 1 means the files differ.
diff -u ~/labs/lab27/kube-apiserver-original.yaml ~/labs/lab27/kube-apiserver-profiling.yaml
install_apiserver kube-apiserver-profiling.yaml
wait_profiling false
cis_scan
```

Record: profiling flag applied row: the one-line configuration change; seconds until readiness and the expected flag were observed; the new scan result.

Step 5. Revert the flag and confirm the cluster is back

Run in: VM terminal 1, same shell

```
install_apiserver kube-apiserver-original.yaml
wait_profiling default
k get nodes
cis_scan
```

Record: profiling flag reverted row: node state; the observed value for 1.2.21; scan totals.

Step 6. Bind cluster-admin to a ServiceAccount, then remove it

Run in: VM terminal 1, same shell

```
k create serviceaccount audit-demo
k create clusterrolebinding ce-lab27-audit --clusterrole=cluster-admin --serviceaccount=ce-lab27:audit-demo
cis_scan
k delete clusterrolebinding ce-lab27-audit
k delete serviceaccount audit-demo
cis_scan | tee ~/labs/lab27/final.txt
diff ~/labs/lab27/baseline.txt ~/labs/lab27/final.txt && echo 'final scan matches the baseline'
```

Record: cluster-admin bound to a ServiceAccount and binding removed rows: the subject named by 5.1.1, its later result, and the diff against baseline.

Recovery check: Every change is reverted, so the final scan matches the baseline, including the two controls a default cluster fails.

Only if normal recovery fails

Step 1. Restore the API server manifest if it does not come back

Run in: VM terminal 2

```
source ~/labs/lab27/helpers.sh
source ~/labs/lab27/exercise.sh
install_apiserver kube-apiserver-original.yaml
docker exec lab27-control-plane crictl ps --name kube-apiserver
wait_profiling default
k get nodes
```

Write your answer

Use the observations recorded beside each step.

Scan	Control that moved	Observed value	Pass and fail totals
Baseline			
admin.conf mode 644			
admin.conf restored			
profiling flag applied			
profiling flag reverted			
cluster-admin bound to a ServiceAccount			
binding removed			

1. Which controls failed on an untouched cluster, and what observed value makes each a finding?
2. Which control moved when admin.conf changed mode, and what restored it?
3. Why does changing a static Pod manifest restart the API server, and what confirms the new setting was applied?

4. What did control 5.1.1 report, and did the final scan match the baseline?

Apply the same reasoning

A manifest on disk contains `--profiling=false`. What would you check before accepting the pass?

Save results before removing this lab's resources and files: `./lab.sh 27 reset`

Lab 28 - Pod Security admission and what a namespace label blocks

Which Pods does a namespace's security level actually reject, and when is that decision made?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 28 setup` on the host. No other lab is required.

Theory you need

Pod Security admission is built into the API server. Namespace labels choose a mode and level. In this fixture, an unlabelled namespace has no restrictive level; other clusters can configure stricter defaults.

There are three levels. `privileged` allows everything; `baseline` blocks the well-known escalation paths, including `privileged: true` and `host namespaces`; `restricted` additionally requires `runAsNonRoot`, `allowPrivilegeEscalation: false`, `capabilities.drop` of `ALL` and a `RuntimeDefault` or `Localhost` `seccomp` profile.

The three modes differ only in consequence. `enforce` rejects the request and lists every violated rule in one message. `warn` creates the object and returns the same list as a client warning. `audit` records it in the audit log and creates the object. A namespace can hold different levels for different modes, which is how a cluster measures a stricter level before enforcing it.

Pod Security checks Pod creation and relevant updates. Adding an `enforce` label does not evict existing Pods, although it can warn about violations. A `Deployment` and `ReplicaSet` can exist while their attempts to create noncompliant Pods are rejected.

This lab makes the granted authority concrete before restricting it. The `privileged` fixture sets `hostPID` and `privileged`, so it shares the node's PID namespace and can list the node's own processes, including its `kubelet` and `container runtime`. That is what the unlabelled namespace permits, and what the `baseline` level blocks.

The `hardened` fixture satisfies `restricted`, so the same image runs as `UID 10001` with no capabilities, no privilege escalation and the default `seccomp` profile. Pod Security checks the fields of a Pod specification; it does not observe runtime behaviour, scan images or cover rules outside the standards, and system namespaces need deliberate treatment because infrastructure workloads may legitimately need host access.

Source: `docs/chaos-theory.md`: §§12.5.1-12.5.2.

Main lesson to learn in this lab

Pod Security separates the restrictions you choose (`privileged`, `baseline` or `restricted`) from the response to violations (`enforce`, `warn` or `audit`). `Enforce` blocks violating Pod requests, while `warn` and `audit` report them without blocking. Existing Pods are not evicted, so a new policy changes future admission rather than repairing running workloads. Verify the specification checks and actual runtime restrictions separately before claiming the workload is hardened.

Experiment

Use namespace ce-lab28 with three Pod manifests. Move the same namespace from unlabelled to baseline, to restricted, to warn and back, recording which creations succeed and what each message says.

inspect_host_processes reads the privileged Pod's /proc view. inspect_hardening reads the hardened process's UID, effective capabilities, NoNewPrivs and seccomp mode. These helpers observe runtime settings without changing them.

Before running: Predict which of the three Pods is created under no label, under baseline, under restricted and under warn.

Measurement key

Signal	Meaning
namespace labels	The enforce, warn and audit keys currently set on ce-lab28. Removing a key removes that mode; nothing is inherited from the cluster.
violation message	A rejection names the level, such as baseline:latest, and every field that violated it. Read the field list, not only the first line.
Warning versus Error	A warning is advisory and is followed by the object being created. An error from server (Forbidden) means nothing was created.
node_processes	Process names the privileged Pod sees through the node's PID namespace. Seeing kubelet or containerd means it is observing the node.
k	kubectl fixed to kind-lab28 and ce-lab28. The namespace labels are changed with kubectl label, and removed with a trailing hyphen.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 28 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Create a privileged Pod in an unlabelled namespace

Run in: VM terminal 1

```
source ~/labs/lab28/exercise.sh
source ~/labs/lab28/helpers.sh
k get namespace ce-lab28 -o jsonpath='{.metadata.labels}{"\n"}'
k apply -f ~/labs/lab28/privileged.yaml
k wait pod/privileged --for=condition=Ready --timeout=120s --request-timeout=0
inspect_host_processes
```

Record: Unlabelled row: the namespace labels, the creation result, visible_pids, pid1, and the node processes the Pod can see.

Step 2. Apply the baseline level while that Pod is running

Run in: VM terminal 1, same shell

```
k label namespace ce-lab28 pod-security.kubernetes.io/enforce=baseline --overwrite
k get namespace ce-lab28 -o jsonpath='{.metadata.labels}{"\n"}'
k get pod privileged -o
  custom-columns='NAME:.metadata.name,PHASE:.status.phase,PRIVILEGED:.spec.containers[0].securityContext.privileged,HOSTPID:.spec.hostPID'
```

Record: Relabelled to baseline while running row: the warning text, the resulting labels, and the Pod's phase and fields afterwards.

Step 3. Recreate the same Pod under the baseline level

Run in: VM terminal 1, same shell

```
k delete pod privileged --wait=true --timeout=30s
k apply -f ~/labs/lab28/privileged.yaml
echo "privileged create exit=$?"
k get pods
```

Record: Recreated under baseline row: the exact rejection message and the fields it names, the exit status, and whether any Pod exists.

Step 4. Move to restricted and compare a plain Pod with a hardened one

Run in: VM terminal 1, same shell

```
k label namespace ce-lab28 pod-security.kubernetes.io/enforce=restricted --overwrite
k apply -f ~/labs/lab28/plain.yaml
echo "plain create exit=$?"
k apply -f ~/labs/lab28/hardened.yaml
k wait pod/hardened --for=condition=Ready --timeout=120s --request-timeout=0
inspect_hardening
```

Record: Plain under restricted and Hardened under restricted rows: the fields named in the rejection, and the hardened Pod's uid, CapEff and Seccomp.

Step 5. Replace enforcement with a warning and repeat the plain Pod

Run in: VM terminal 1, same shell

```
k label namespace ce-lab28 pod-security.kubernetes.io/enforce-
k label namespace ce-lab28 pod-security.kubernetes.io/warn=restricted --overwrite
k get namespace ce-lab28 -o jsonpath='{.metadata.labels}{"\n"}'
k apply -f ~/labs/lab28/plain.yaml
echo "plain create exit=$?"
k get pods -o custom-columns='NAME:.metadata.name,PHASE:.status.phase'
```

Record: Plain under warn row: the labels, the warning text, the exit status and the Pods that exist afterwards.

Step 6. Restore enforcement and confirm the final state

Run in: VM terminal 1, same shell

```
k delete pod plain --wait=true --timeout=30s
k label namespace ce-lab28 pod-security.kubernetes.io/warn-
k label namespace ce-lab28 pod-security.kubernetes.io/enforce=restricted --overwrite
k get namespace ce-lab28 -o jsonpath='{.metadata.labels}{"\n"}'
k apply -f ~/labs/lab28/plain.yaml --dry-run=server
echo "dry-run exit=$?"
k get pods -o custom-columns='NAME:.metadata.name,PHASE:.status.phase'
k delete pod hardened --wait=true --timeout=30s
k label namespace ce-lab28 pod-security.kubernetes.io/enforce-
k get namespace ce-lab28 -o jsonpath='{.metadata.labels}{"\n"}'
k get pods
```

Record: Enforcement restored row: the labels, the dry-run rejection and exit status, and the Pods remaining.

Recovery check: Restricted enforcement was demonstrated with a rejected dry run, then the hardened Pod and the namespace label were removed, returning ce-lab28 to the unlabelled state the lab started from.

Write your answer

Use the observations recorded beside each step.

Namespace configuration	Pod attempted	Result and message	Evidence after the attempt
Unlabelled			
Relabelled to baseline while running			
Recreated under baseline			

Namespace configuration	Pod attempted	Result and message	Evidence after the attempt
Plain under restricted			
Hardened under restricted			
Plain under warn			
Enforcement restored			

1. What could the privileged Pod see before a Pod Security level was enforced?
2. Why did labelling the namespace warn but not stop the running Pod?
3. Which fields did restricted require, and how did the hardened Pod satisfy them?
4. What did warn allow that enforce rejected?

Apply the same reasoning

A team sets warn=restricted everywhere and calls the cluster restricted. What have they established?

Save results before removing this lab's resources and files: `./lab.sh 28 reset`

Lab 29 - What a Pod's ServiceAccount can reach

What can a workload do against the Kubernetes API, and which change grants or removes it?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 29 setup` on the host. No other lab is required.

Theory you need

Unless a Pod opts out, the kubelet projects its ServiceAccount credential into every container at `/var/run/secrets/kubernetes.io/serviceaccount`: a short-lived token, the cluster `ca.crt` and a namespace file. The API server address arrives separately in the `KUBERNETES_SERVICE_HOST` and `KUBERNETES_SERVICE_PORT` variables, so reaching the API needs no cluster DNS.

The token is a JWT whose payload is ordinary base64 data, so a process can read its own sub claim, which is `system:serviceaccount` followed by the namespace and account name, and its exp expiry, without verifying anything.

That identity is what every request from this Pod carries, and what anyone who reaches the container inherits. Projected tokens are bound to the Pod and rotated by the kubelet, so an expiry in the future is normal rather than a finding.

Authentication and authorization are separate. Presenting a valid token establishes who you are; RBAC then decides what that identity may do. A Role lists verbs on resources, and a RoleBinding attaches it to a subject. Both are namespaced, so a grant in one namespace says nothing about another, and neither is implied by the token existing.

Read the refusal rather than the status code alone. A 403 names the account, the verb, the resource and the namespace, which is exactly the information needed to write the smallest Role instead of a wider one. The same question can be asked from outside with `kubectl auth can-i --as`, which prints yes or no and exits nonzero when the answer is no.

This lab keeps one request and changes one thing at a time. The probe lists Pods in its own namespace and in `kube-system` from inside the Pod, so a namespaced grant is visible as a difference between two responses in a single run rather than as a claim.

Removing the credential is a different control from refusing the request. A Pod with `automountServiceAccountToken false` has no token and no `ca.crt`, so its request arrives as `system:anonymous` and is refused for a different reason, while the network path to the API server is unchanged. Neither control makes the API unreachable; they change identity and authorization.

Source: docs/chaos-theory.md: §§10.11.1-10.11.2.

Main lesson to learn in this lab

Authentication establishes the caller's identity; authorization decides what that identity may do. A `ServiceAccount` token does not itself grant resource access, and a namespaced `RoleBinding` grants only its stated permissions there. Compare the identity and operation in each API response: removing a binding removes its grant, while omitting the token changes authentication. Neither action removes the network path to the API server.

Experiment

Use namespace `ce-lab29` with two identical clients that differ only in whether the token is mounted. Compare the default account, an explicit `Role` and `RoleBinding`, another namespace, and the removal of both the grant and the token.

Before running: Predict both request results for the default account, after the `RoleBinding`, from the other namespace and from the Pod with no token.

Measurement key

Signal	Meaning
token / subject / expires_at	Whether the credential is mounted, the identity it names and its expiry, decoded from the token payload without verification.
GET ... -> status	200 with an item count means the list succeeded. 403 is an authorization refusal, and its message names the account and the resource.
own namespace versus kube-system	Two requests in one run. A namespaced <code>Role</code> changes only the first; a difference between them is the scope of the grant.
auth can-i	The same decision asked from the host. It prints yes or no and exits nonzero for no, so the exit status is evidence too.
k	<code>kubectl</code> fixed to <code>kind-lab29</code> and <code>ce-lab29</code> . The probe runs inside a Pod and uses only what that Pod is given.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 29 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Read the projected credential and try the API

Run in: VM terminal 1

```
source ~/labs/lab29/helpers.sh
k exec client -- ls -l /var/run/secrets/kubernetes.io/serviceaccount
k exec client -- python -u /scripts/apicheck.py
k auth can-i list pods --as=system:serviceaccount:ce-lab29:default --namespace=ce-lab29
echo "can-i exit=$?"
```

Record: Default account row: the mounted files, the subject and expiry, both request results, and the `can-i` answer with its exit status.

Step 2. Grant one namespaced Role and repeat the same requests

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab29/rbac.yaml
k get role,rolebinding pod-reader -o custom-columns='KIND:.kind,NAME:.metadata.name'
k exec client -- python -u /scripts/apicheck.py
k auth can-i list pods --as=system:serviceaccount:ce-lab29:default --namespace=ce-lab29
k auth can-i list pods --as=system:serviceaccount:ce-lab29:default --namespace=kube-system
echo "kube-system can-i exit=$?"
```

Record: Role and RoleBinding applied row: both request results, the item count returned, and both can-i answers.

Step 3. Run the same probe in the Pod with no mounted token

Run in: VM terminal 1, same shell

```
k get pod client-notoken -o jsonpath='{.spec.automountServiceAccountToken}{"\n"}'
k exec client-notoken -- ls /var/run/secrets/kubernetes.io/serviceaccount
k exec client-notoken -- python -u /scripts/apicheck.py
```

Record: Pod without a token row: the automount setting, whether the credential directory exists, the reported identity and both request results.

Step 4. Remove the grant and confirm it was the cause

Run in: VM terminal 1, same shell

```
k delete rolebinding pod-reader --wait=true --timeout=30s
k get role,rolebinding -o custom-columns='KIND:.kind,NAME:.metadata.name'
k exec client -- python -u /scripts/apicheck.py
k auth can-i list pods --as=system:serviceaccount:ce-lab29:default --namespace=ce-lab29
echo "can-i exit=$?"
```

Record: RoleBinding removed row: the remaining objects, both request results and the can-i answer.

Recovery check: Both client Pods are Ready, no RoleBinding grants the default account, and the in-Pod request is refused with 403.

Write your answer

Use the observations recorded beside each step.

Case	Identity presented	Own namespace	kube-system
Default account			
Role and RoleBinding applied			
Pod without a token			
RoleBinding removed			

1. What identity did the default Pod present, and what was it allowed to do?
2. Which of the two requests changed after the RoleBinding, and why only that one?
3. Without a mounted token, which identity did the API server see?
4. What changed when the RoleBinding was deleted, and what stayed?

Apply the same reasoning

Does binding a Role in two namespaces differ from granting it cluster-wide?

Save results before removing this lab's resources and files: ./lab.sh 29 reset

Lab 30 - What a Secret protects, and what it does not

Where does a Secret's value actually rest, and who can read it?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 30 setup` on the host. No other lab is required.

Theory you need

A Secret is an ordinary API object. Its values live in `.data` as base64, which is an encoding and not encryption: anyone allowed to read the object can decode it with one command. `kubectl describe` prints only key names and sizes, which hides the value from a terminal but changes nothing about who may read it.

What separates a Secret from a ConfigMap is how it is usually treated: RBAC is written more narrowly for it, and the kubelet keeps its contents in memory on the node rather than writing them to disk. Neither property is automatic, and neither is what protects the stored copy.

The stored copy lives in etcd. Unless the API server is started with an `encryption-provider-config`, the identity provider is used and the value is written unchanged. This lab reads the same key directly out of etcd with `etcdctl`, through the control plane node, and finds the password beside its key name.

Encryption at rest applies from the moment it is configured. Objects written before it was enabled stay as they were until something rewrites them, so enabling it is a migration rather than a switch.

A Pod can receive the value two ways. An environment variable is copied into the process when the container starts, so it is visible to every process in that container and to anything that reads `/proc/PID/environ`. A projected volume places it on a tmpfs file whose path is a symlink into a timestamped directory the kubelet replaces on update.

The workload's ServiceAccount and the kubelet use different identities. Once a Pod referencing the Secret is authorized and admitted, the kubelet fetches and projects its contents using its own API permissions. The workload's ServiceAccount need not have get permission on that Secret. Having no API access and having no secret material are different statements.

The two surfaces behave differently when the value changes. An environment variable stays at the value the process started with, so a rotation reaches it only when the container is replaced. A mounted file is refreshed by the kubelet on its sync period, which takes tens of seconds rather than being immediate.

Measure the delay rather than assuming it. This lab polls the mounted file until it changes and prints the observed seconds, then reads the environment variable in the same Pod for comparison.

Source: docs/chaos-theory.md: §§12.7.1-12.7.2.

Main lesson to learn in this lab

Base64 does not encrypt a Secret; without encryption at rest, its value is also readable in etcd. A Pod can receive mounted material without its ServiceAccount having Secret API access. Environment values remain fixed for that process; ordinary mounted files update eventually and must be reread. Protect storage and workload access separately, and verify that rotation reaches the actual consumer.

Experiment

Use namespace `ce-lab30` with one Secret and two consumers that differ only in how they receive it, then read the same value through the API, through etcd, through both containers, and after a rotation.

`strings` displays printable data from the raw etcd value. `wait_secret` checks the mounted password every five seconds for at most 30 attempts; it reports a change only after reading the expected value. A timeout

leaves propagation unproven.

Before running: Predict what etcd holds, which surfaces expose the value inside each container, and which surface changes when the Secret is rotated.

Measurement key

Signal	Meaning
.data.password	The stored value as base64. Decoding it is a transport step, not a permission check.
etcd bytes	What the API server wrote. Plaintext here means no encryption provider is configured for Secrets.
env_var / proc_1_environ	The value as the process holds it. Both come from the container start, so both are stale after a rotation.
mounted_file / mounted_target	The projected file and the timestamped directory it points at. The kubelet replaces the target to publish a new value.
can-i get secrets	Whether that ServiceAccount may read the Secret through the API. It says nothing about what the kubelet already mounted.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 30 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Read the Secret through the API

Run in: VM terminal 1

```
source ~/labs/lab30/exercise.sh
source ~/labs/lab30/helpers.sh
k get secret app-credentials -o jsonpath='{.type}{"\n"}{.data.password}{"\n"}'
k get secret app-credentials -o jsonpath='{.data.password}' | base64 -d; echo
k describe secret app-credentials
k -n kube-system get pod kube-apiserver-lab30-control-plane -o jsonpath='{.spec.containers[0].command}{"\n"}'
```

Record: API row: encoded value; decoded password; byte count from describe. In the running API server command, look for --encryption-provider-config; record whether it is present.

Step 2. Read the same key out of etcd

Run in: VM terminal 1, same shell

```
etcd_get /registry/secrets/ce-lab30/app-credentials > ~/labs/lab30/stored.bin
echo "etcd read exit=$?"
ls -l ~/labs/lab30/stored.bin
strings ~/labs/lab30/stored.bin
```

Record: API and etcd row: the read exit status, the size of the stored object, and whether the password is readable.

Step 3. Compare the two consumers inside their containers

Run in: VM terminal 1, same shell

```
k exec env-consumer -- python -u /scripts/secretprobe.py
k exec file-consumer -- python -u /scripts/secretprobe.py
k get pod file-consumer -o jsonpath='{.spec.serviceAccountName}{"\n"}'
k auth can-i get secrets --as=system:serviceaccount:ce-lab30:no-secret-access --namespace=ce-lab30
echo "can-i exit=$?"
```

Record: Environment variable and Mounted file rows: every surface reported for both Pods, and the can-i answer with its exit.

Step 4. Rotate the Secret and measure which surface follows

Run in: VM terminal 1, same shell

```
k create secret generic app-credentials --from-literal=password=rotated-lab30 \
--dry-run=client -o yaml | k apply -f -
k get secret app-credentials -o jsonpath='{.data.password}' | base64 -d; echo
wait_secret rotated-lab30
k exec file-consumer -- python -u /scripts/secretprobe.py
k exec env-consumer -- python -u /scripts/secretprobe.py
```

Record: After rotation row: the new stored value, the measured seconds until the file changed, and both consumers' surfaces afterwards.

Step 5. Restore the original value and confirm both surfaces

Run in: VM terminal 1, same shell

```
k create secret generic app-credentials --from-literal=password=lab30-original \
--dry-run=client -o yaml | k apply -f -
wait_secret lab30-original
k exec file-consumer -- python -u /scripts/secretprobe.py
k get pods -o custom-columns='NAME:.metadata.name,READY:.status.conditions[?(@.type=="Ready")].status'
k exec env-consumer -- python -u /scripts/secretprobe.py
```

Record: Restored row: the value the mounted file returned to, and both Pods' readiness.

Recovery check: Both consumers are Ready, the Secret holds its original value, and the file consumer's ServiceAccount is still refused get on Secrets.

Only if normal recovery fails

Step 1. Restore the original Secret value without the helpers

Run in: VM terminal 2

```
kubectl --kubeconfig="$HOME/labs/lab30/kubeconfig" --context=kind-lab30 --namespace=ce-lab30 \
create secret generic app-credentials --from-literal=password=lab30-original --dry-run=client -o yaml |
kubectl --kubeconfig="$HOME/labs/lab30/kubeconfig" --context=kind-lab30 --namespace=ce-lab30 apply -f -
kubectl --kubeconfig="$HOME/labs/lab30/kubeconfig" --context=kind-lab30 --namespace=ce-lab30 get pods
```

Write your answer

Use the observations recorded beside each step.

Surface	Value seen	What limits it
API and etcd		
Environment variable		
Mounted file		
After rotation		
Restored		

1. What did the API and etcd each show for the same Secret value?
2. Which surfaces exposed the value inside each container?
3. Why could the file consumer read a Secret its ServiceAccount may not get?
4. Which surface changed after the rotation, and how long did it take?

Apply the same reasoning

One Secret is mounted into ten Pods and rotated. Which of them need a restart?

Save results before removing this lab's resources and files: `./lab.sh 30 reset`

Lab 31 - Default deny, and what a policy does not stop

Which traffic does a NetworkPolicy stop, and what does the caller see when it does?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 31 setup` on the host. No other lab is required.

Theory you need

A NetworkPolicy is namespaced and applies to the Pods its podSelector matches. An empty selector matches every Pod in the namespace. Naming a policy type without rules for it is what creates a default deny: policyTypes Ingress with no ingress block grants no incoming connections. Other matching policies can still allow traffic; traffic from a Pod's own node and replies to allowed connections are also exempt.

Policies are additive allow-lists. A second policy that permits one source does not weaken the first; what is permitted is the union of every matching policy. There is no deny rule to write and no ordering to reason about. A Pod-to-Pod connection must be allowed by both the source's egress policies and the destination's ingress policies when those directions are isolated.

Read the selectors twice. A podSelector inside an ingress from entry selects sources in the policy's own namespace, so crossing a namespace boundary needs a namespaceSelector. An entry that lists both selects their intersection, and separate list entries are alternatives.

The API server stores these objects whether or not anything enforces them. Enforcement belongs to the network plugin, so a cluster whose plugin ignores NetworkPolicy accepts every policy and changes no traffic. The first evidence is therefore a measured block, not a created object.

This lab's network plugin drops denied traffic. A refusal returns at once; a drop leaves the caller waiting until its own timeout expires. The probe in this lab carries a five-second budget and prints the elapsed milliseconds, so an allowed request and a denied one are told apart by shape as well as by result.

Egress rules cover name resolution as well as the connection that follows. The resolver is a Pod in another namespace, so a default-deny egress policy stops DNS first, and the lookup fails slowly: every search-list candidate is tried and dropped in turn.

Allowing UDP and TCP port 53 to the kube-dns Pods in kube-system restores resolution without allowing anything else. The lookup then answers in milliseconds again while the connection to that address stays denied, which separates a name that cannot be resolved from a destination that cannot be reached.

Each case in this lab changes exactly one object. The workloads, the image and the probe never change, which is what makes the selector rather than the workload the explanation for a difference.

Source: docs/chaos-theory.md: §§12.8.1-12.8.2.

Main lesson to learn in this lab

For Pod-to-Pod traffic, both source egress and destination ingress must allow the connection; matching policies add allowed paths within each direction. Default deny supplies no allowed paths. A supporting network plugin must enforce these rules. Measure DNS and HTTP separately: allowing name resolution does not grant access to the resolved address, and a policy object alone proves no traffic isolation.

Experiment

Use namespace `ce-lab31` with one server behind a Service and two identical clients that differ only in a label, then add a default deny, an allow rule, an egress deny and a DNS allow, one at a time.

wait_network_recovery retries the labelled client's HTTP request after removing the policies and reports an error if HTTP 200 is not observed. The individual probes still show each client's final result.

Before running: Predict both clients' results after a default deny, after an allow rule for one label, and what a default-deny egress does to name resolution.

Measurement key

Signal	Meaning
ok status=200	The request completed. Its elapsed_ms shows the normal cost of this path inside the cluster.
failed URLError	No response arrived. Compare elapsed_ms with the five-second budget to tell a drop from a refusal.
elapsed_ms near 5000	The caller waited out its own timeout, which is what a dropped packet looks like from the client side.
dns ... resolved / failed gaiererror	Whether the name resolved, and how long it took. A failure after tens of seconds means the queries were dropped rather than answered.
kubectl get networkpolicy	The objects that exist. Accepted objects are not evidence of enforcement; the measurements are.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 31 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Measure both clients with no policy in place

Run in: VM terminal 1

```
source ~/labs/lab31/exercise.sh
source ~/labs/lab31/helpers.sh
k get networkpolicy
k get pods -o custom-columns='NAME:.metadata.name,ROLE:.metadata.labels.role,IP:.status.podIP'
probe client http server
probe client-unlabelled http server
probe client dns server
```

Record: No policy row: both clients' results and elapsed_ms, the DNS result, and which Pod carries the role label.

Step 2. Apply a default deny for all ingress

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab31/deny-ingress.yaml
k get networkpolicy default-deny-ingress -o jsonpath='{.spec}{"\n"}'
sleep 5
probe client http server
probe client-unlabelled http server
```

Record: Default deny ingress row: the policy spec, and both clients' results with elapsed_ms.

Step 3. Allow only the labelled client

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab31/allow-client.yaml
k get networkpolicy -o custom-columns='NAME:.metadata.name,SELECTOR:.spec.podSelector'
sleep 5
probe client http server
probe client-unlabelled http server
```

Record: Allow labelled client row: both policies, and both clients' results with elapsed_ms.

Step 4. Deny all egress from the allowed client

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab31/deny-egress.yaml
sleep 5
probe client dns server
SERVER_IP=$(k get pod server -o jsonpath='{.status.podIP}')
probe client http "$SERVER_IP:8080"
probe client-unlabelled dns server
```

Record: Default deny egress row: the DNS result and elapsed_ms from the selected client, the request by IP, and the unselected client.

Step 5. Allow egress to the cluster resolver only

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab31/allow-dns-egress.yaml
k get networkpolicy default-deny-egress -o jsonpath='{.spec.egress}{"\n"}'
sleep 5
probe client dns server
probe client http server
```

Record: Egress to kube-dns allowed row: the egress rule, the DNS result with elapsed_ms, and the HTTP result.

Step 6. Remove every policy and confirm recovery

Run in: VM terminal 1, same shell

```
k delete -f ~/labs/lab31/deny-ingress.yaml -f ~/labs/lab31/allow-client.yaml -f
~/labs/lab31/allow-dns-egress.yaml --ignore-not-found
k get networkpolicy
wait_network_recovery
probe client http server
probe client-unlabelled http server
probe client dns server
```

Record: Policies removed row: the remaining policies, both clients' results and the DNS result.

Recovery check: No NetworkPolicy remains in the namespace, both clients reach the server by name, and the server is Ready.

Only if normal recovery fails

Step 1. Remove every policy in the namespace without the helpers

Run in: VM terminal 2

```
kubectl --kubeconfig="$HOME/labs/lab31/kubeconfig" --context=kind-lab31 --namespace=ce-lab31 \
delete networkpolicy --all --ignore-not-found
kubectl --kubeconfig="$HOME/labs/lab31/kubeconfig" --context=kind-lab31 --namespace=ce-lab31 get pods
```

Write your answer

Use the observations recorded beside each step.

Policy in place	Labelled client	Unlabelled client	DNS
No policy			
Default deny ingress			
Allow labelled client			
Default deny egress			
Egress to kube-dns allowed			
Policies removed			

1. What did each client see after the default-deny ingress policy, and how long did it take?
2. Why did one client succeed and the identical one fail after the allow rule?
3. What did the default-deny egress break first, and why that?
4. What changed once egress to kube-dns was allowed?

Apply the same reasoning

A namespace has a default-deny ingress policy. Does that protect it from a Pod in another namespace?

Save results before removing this lab's resources and files: `./lab.sh 31 reset`

Lab 32 - Grants that reach further than they name

Which ordinary-looking grant lets an account act as a stronger one?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 32 setup` on the host. No other lab is required.

Theory you need

RBAC answers one question at a time, whether this subject may perform this verb on this resource in this namespace. A grant is therefore exactly as narrow as the objects it names, and reading it as an intention rather than as a list of objects is what produces surprises.

Some objects carry another identity. A Pod spec chooses its own `serviceAccountName`, so an account that may create Pods in a namespace may run a workload as any `ServiceAccount` in that namespace and use the token the kubelet projects into it. Nothing in the create request mentions secrets, and no secret permission is needed to make the request.

The same applies to a Pod that already exists. `create on pods/exec` runs a process inside a container with that container's identity, mounts and network position, so the grant is worth whatever the target Pod holds rather than whatever the caller holds.

One escalation route is blocked in the API server itself. Writing a `Role` or `RoleBinding` that grants permissions the writer does not hold is refused with a message that names the extra rule, unless the writer holds the `escalate` or `bind` verb. A `Role` limited to what the writer already has is accepted, so the guard is about the delta rather than about writing roles at all.

This lab uses three `ServiceAccounts` in one namespace: one that may create Pods, one that may list `Secrets`, and one that may write `Roles`. Each is refused what the others hold, which is what makes any successful read from the wrong account an escalation rather than a configuration.

Ask the authorization question the way the request will be made. `kubectl auth can-i` uses `TYPE/NAME` syntax: `can-i create pods/exec` asks about a Pod named `exec`, not the `exec` subresource. It can answer yes for permission to create Pods while the `exec` operation is refused.

The forms that agree with the request are `can-i create pods --subresource=exec` and a `SubjectAccessReview` whose `resourceAttributes` name resource `pods` and subresource `exec`. This lab runs all four and compares them, so the disagreement is measured rather than described.

Treat a review as a prediction and the operation as the result. When an authorization answer matters, name the subresource, and confirm with the request itself.

Source: docs/chaos-theory.md: §§12.9.1-12.9.2.

Main lesson to learn in this lab

Effective RBAC authority includes what a permitted operation lets you reach. Where admission allows it, creating a Pod can expose a stronger ServiceAccount's token; exec can expose an existing container's credentials and mounts. Role-writing guards do not remove those routes. Check the exact resource, subresource and verb against the real request: a direct denial does not prove the same access is impossible indirectly.

Experiment

Use namespace ce-lab32 with three ServiceAccounts whose Roles do not overlap, then read Secrets from the stronger account, try to widen a Role, and compare four ways of checking exec access.

Before running: Predict whether the builder account can read Secrets directly, whether a Pod it creates can, and whether it can grant itself the missing verb.

Measurement key

Signal	Meaning
subject=	The identity the token names, decoded inside the Pod. It is the account the API server will authorize, not the account that created the Pod.
secrets -> 200 items=N	The list succeeded. From a Pod created by an account that is refused the same read, this is the escalation.
secrets -> 403	An authorization refusal naming the account, the verb and the resource. It is the baseline this lab compares against.
not currently held	The API server's escalation guard. It names the rule that exceeded what the writer holds.
can-i answers	Four ways of asking one question. Where they differ, the operation decides.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 32 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Read what each account is granted

Run in: VM terminal 1

```
source ~/labs/lab32/helpers.sh
k get role -o custom-columns='NAME:.metadata.name,RULES:.rules[*].resources'
k auth can-i create pods --as=system:serviceaccount:ce-lab32:builder
k auth can-i get secrets --as=system:serviceaccount:ce-lab32:builder
echo "builder secrets can-i exit=$?"
k exec builder-pod -- python -u /scripts/apiprobe.py
```

Record: Builder baseline and Pod running as the builder rows: each Role's resources, both can-i answers, and the Pod's own result.

Step 2. Create a Pod that runs as the reader

Run in: VM terminal 1, same shell

```
k --as=system:serviceaccount:ce-lab32:builder apply -f ~/labs/lab32/borrowed-pod.yaml
echo "create exit=$?"
k wait pod/borrowed-pod --for=condition=Ready --timeout=120s --request-timeout=0
k get pod borrowed-pod -o jsonpath='{.spec.serviceAccountName}{"\n"}'
k exec borrowed-pod -- python -u /scripts/apiprobe.py
```

Record: Pod running as the reader row: the create result, the ServiceAccount in the spec, and the result reported inside the Pod.

Step 3. Try to widen a Role beyond its own rights

Run in: VM terminal 1, same shell

```
k auth can-i get secrets --as=system:serviceaccount:ce-lab32:role-writer
k --as=system:serviceaccount:ce-lab32:role-writer create role within-rights --verb=get --resource=roles
echo "within rights exit=$?"
k --as=system:serviceaccount:ce-lab32:role-writer create role beyond-rights --verb=get --resource=secrets
echo "beyond rights exit=$?"
```

Record: Role within rights and Role beyond rights rows: both create results with their exit statuses, and the refusal text.

Step 4. Ask about pods/exec four ways and run the operation

Run in: VM terminal 1, same shell

```
k auth can-i create pods/exec --as=system:serviceaccount:ce-lab32:builder
k auth can-i create pods --subresource=exec --as=system:serviceaccount:ce-lab32:builder
cat ~/labs/lab32/exec-review.yaml
k create -f ~/labs/lab32/exec-review.yaml -o jsonpath='{.status.allowed}'{"\n"}'
k --as=system:serviceaccount:ce-lab32:builder exec borrowed-pod -- echo reached-the-container
echo "exec exit=$?"
```

Record: Four authorization answers row: the answer from each of the four forms, and the exit status of the real exec.

Step 5. Remove the borrowed Pod and the grants

Run in: VM terminal 1, same shell

```
k delete pod borrowed-pod --ignore-not-found --wait=true --timeout=60s
k delete role within-rights --ignore-not-found
k delete rolebinding reader-lists-secrets --ignore-not-found --wait=true --timeout=30s
k exec builder-pod -- python -u /scripts/apiprobe.py
k auth can-i get secrets --as=system:serviceaccount:ce-lab32:secret-reader
echo "reader secrets can-i exit=$?"
k apply -f ~/labs/lab32/rbac.yaml
```

Record: Grants removed row: the remaining objects, the builder Pod's result, and the reader's can-i answer once its binding is gone.

Recovery check: Only the builder Pod remains, no borrowed Pod exists, and the builder account is still refused get on Secrets.

Only if normal recovery fails

Step 1. Remove the borrowed Pod and restore the grants without the helpers

Run in: VM terminal 2

```
kubectl --kubeconfig="$HOME/labs/lab32/kubeconfig" --context=kind-lab32 --namespace=ce-lab32 \
  delete pod borrowed-pod --ignore-not-found
kubectl --kubeconfig="$HOME/labs/lab32/kubeconfig" --context=kind-lab32 --namespace=ce-lab32 \
  delete role within-rights beyond-rights --ignore-not-found
kubectl --kubeconfig="$HOME/labs/lab32/kubeconfig" --context=kind-lab32 --namespace=ce-lab32 \
  apply -f "$HOME/labs/lab32/rbac.yaml"
```

Write your answer

Use the observations recorded beside each step.

Case	Account acting	Result	What it establishes
Builder baseline			
Pod running as the builder			
Pod running as the reader			
Role within rights			

Case	Account acting	Result	What it establishes
Role beyond rights			
Four authorization answers			
Grants removed			

1. What was the builder account refused directly, and what did its own Pod report?
2. How did a Pod created by that same account list the Secrets?
3. Which Role was accepted and which was refused, and what did the refusal name?
4. Which ways of asking about pods/exec disagreed with the real request?

Apply the same reasoning

An account may only create Pods in one namespace. Which credentials in that namespace are within its reach?

Save results before removing this lab's resources and files: `./lab.sh 32 reset`

Lab 33 - When probes cause or repair restarts

When should a probe withdraw traffic, restart a container, or give it more time to start?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 33 setup` on the host. No other lab is required.

Theory you need

Startup, readiness and liveness answer different questions. Startup gates the other two until initialization succeeds. Readiness controls eligibility for ordinary Service traffic. Repeated liveness failure makes kubelet terminate the container; this Pod uses `restartPolicy Always`, so it starts a new container inside the same Pod UID.

The supplied server waits 15 seconds before opening its listener. Its startup probe allows 20 failed checks at a two-second period; this is a nominal observation window, not an exact restart timestamp. Liveness allows only three failed checks at that period. Without startup protection, the server can be killed before it ever finishes initialization, then repeat that work on every restart.

The readiness and liveness paths are independent. A file named `/tmp/unready` makes only readiness fail. Removing it repairs eligibility without a restart. A different file, `/tmp/stalled`, simulates lost application progress: both real requests and liveness fail. Restarting clears this file because it belongs to the old container writable layer, so this particular fault is repairable by restart.

A liveness probe is useful only when restarting can repair the condition it detects. A failed external database does not become healthy because its callers restart. Avoid using the same broad dependency check for every probe; restarting many callers can also remove available capacity.

Record Pod UID, container ID, restart count, last termination and events together. A changed container ID within one UID is a container restart; the procedure deliberately changes UID when replacing immutable probe configurations. Read previous-container logs before deleting that Pod. Neither a nonzero exit nor a failed probe alone proves which policy caused the restart.

Endpoint withdrawal is asynchronous. A failed readiness wait establishes Pod state, not the exact time every routing rule changed. Compare `EndpointSlice` conditions and a bounded request from the separate

client. Missing logs or a wait timeout leave the corresponding conclusion unproven.

Source: docs/chaos-theory.md: §§10.12.1, 10.12.2 (maintained supplement; primary references linked there).

Main lesson to learn in this lab

Choose probes by the action they should trigger: startup protects initialization, readiness withdraws traffic, and liveness restarts a container whose progress is lost. A short liveness budget can create a startup loop, while readiness can recover without replacing anything. Compare client results with UID, container ID, events and restart count; configure a restart only for a condition that restarting can actually repair.

Experiment

Use ce-lab33 in its private cluster. One server initializes for 15 seconds; a separate client calls it through its Service. Two manifests differ only in the startup probe.

probe_state saves the Pod JSON under the named phase and prints identities, restart state and EndpointSlices. wait_restarts checks up to 90 times with two-second pauses; API request time adds to the wait. probe_http makes one Service request with a three-second timeout; a failure is an observation, not an instruction to stop the shell. wait_service retries that request up to 15 times during setup and recovery, with two-second pauses.

Before running: Predict whether each fault changes readiness, restart count, Pod UID and client HTTP, and whether restarting repairs it.

Measurement key

Signal	Meaning
UID / containerID / restartCount	Distinguish a restarted container from a deliberately recreated Pod; retain evidence before deleting it.
Ready / EndpointSlice ready	Pod readiness and routing eligibility converge asynchronously; record both rather than assuming identical timing.
Unhealthy / Killing / previous logs	Match the failing probe and kubelet action to whether initialization ever reached the listening log.
HTTP / elapsed_seconds	One bounded client observation tests the Service path; it does not measure continuous availability.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 33 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Record the protected baseline

Run in: VM terminal 1, same shell

```
source ~/labs/lab33/helpers.sh
probe_state baseline
probe_http
k logs probe-web
```

Record: Protected baseline row: Pod UID, container ID, restarts, Ready, endpoints and HTTP.

Step 2. Fail only readiness

Run in: VM terminal 1, same shell

```
k exec probe-web -- touch /tmp/unready
k wait pod/probe-web --for=condition=Ready=false --timeout=30s --request-timeout=0
probe_state unready
probe_http
```

Record: Readiness fault row: the same identities and restart count; Pod and endpoint conditions; HTTP result.

Step 3. Remove the readiness fault

Run in: VM terminal 1, same shell

```
k exec probe-web -- rm /tmp/unready
k wait pod/probe-web --for=condition=Ready --timeout=30s --request-timeout=0
probe_state ready-again
probe_http
```

Record: Readiness restored row: identity, restart count, Ready and HTTP.

Step 4. Lose progress and observe a repairable restart

Run in: VM terminal 1, same shell

```
k exec probe-web -- touch /tmp/stalled
wait_restarts 1
k logs probe-web --previous
k describe pod probe-web
k wait pod/probe-web --for=condition=Ready --timeout=90s --request-timeout=0
probe_state progress-recovered
probe_http
```

Record: Progress fault recovered row: UID, container IDs, restarts, probe events, previous log and recovered HTTP.

Step 5. Remove startup protection while keeping the application unchanged

Run in: VM terminal 1, same shell

```
k delete pod probe-web --wait=true --timeout=30s
k apply -f ~/labs/lab33/no-startup.yaml
wait_restarts 1
probe_state no-startup
k logs probe-web --previous
k describe pod probe-web
```

Record: No startup protection row: new Pod UID, restart count, failing probe and whether the previous log reached listening.

Step 6. Restore the startup probe and check the client outcome

Run in: VM terminal 1, same shell

```
k delete pod probe-web --wait=true --timeout=30s
k apply -f ~/labs/lab33/protected.yaml
k wait pod/probe-web --for=condition=Ready --timeout=90s --request-timeout=0
probe_state protected-restored
k logs probe-web
wait_service
```

Record: Protected restored row: initialization log, UID, zero restarts, Ready, endpoints and HTTP.

Recovery check: The protected Pod is Ready with zero restarts, and the client receives HTTP 200 through probe-web.

Write your answer

Use the observations recorded beside each step.

Phase	Observed state	Evidence / explanation
Protected baseline		

Phase	Observed state	Evidence / explanation
Readiness fault		
Readiness restored		
Progress fault recovered		
No startup protection		
Protected restored		

1. Why did readiness recover without changing the container identity?
2. What proves the progress fault restarted a container inside the same Pod?
3. Why did removing only the startup probe prevent initialization from completing?
4. Which observations prove the protected configuration recovered?

Apply the same reasoning

Would attaching the liveness probe to a temporarily unavailable shared database repair that database?

Save results before removing this lab's resources and files: `./lab.sh 33 reset`

Lab 34 - Volume binding, placement and retained data

Why can a claim wait, a replacement Pod keep data, and a retained volume still refuse a new claim?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 34 setup` on the host. No other lab is required.

Theory you need

A PersistentVolume (PV) describes storage; a PersistentVolumeClaim (PVC) requests it. Binding associates one claim identity with one volume. Capacity, access mode and storage class must match. This lab supplies one static local PV; its no-provisioner StorageClass never creates extra storage automatically.

WaitForFirstConsumer delays binding until the scheduler can consider a consuming Pod. A Pending claim without a consumer can therefore be normal. The local directory exists only on lab34-worker, and the PV declares that node affinity. A consumer requiring lab34-worker2 cannot satisfy both placement and storage topology, even when CPU, memory and storage capacity are sufficient.

Use nodeSelector rather than nodeName for this comparison. nodeName bypasses the scheduler, which is the component coordinating delayed binding. Compare claim events, PodScheduled and PV node affinity before treating Pending as a disk failure.

The Pod, PVC, PV and backing directory have different lifetimes. Replacing a Pod while retaining its PVC remounts the same data, with a new Pod UID. ReadWriteOnce means read-write access from one node; it is not a promise that only one Pod on that node can access the volume.

PVC protection defers deletion while a Pod uses the claim. A deletion timestamp and the `kubernetes.io/pvc-protection` finalizer describe this wait; the PVC can still show Bound. After the consumer disappears, claim deletion can finish. Do not remove the finalizer to force progress: remove the intended consumer and observe the controller finish its work.

With reclaimPolicy Retain, deleting the claim leaves the PV Released and preserves backing data. Released is not Available: the old claim reference prevents automatic reuse by another claimant. Here an administrator re-registers the same retained directory as a new PV for the same learner, then creates a new claim. Reusing unknown retained data for another tenant would expose that data.

Local persistence is not replication. If lab34-worker is unavailable, its bytes are not automatically accessible on another node. The declared 256 MiB PV capacity is a matching value, not a filesystem quota on this directory. The experiment writes one small marker and never fills the filesystem; deleting the private kind cluster removes this lab storage.

Source: docs/chaos-theory.md: §§10.13.1, 10.13.2 (maintained supplement; primary references linked there).

Main lesson to learn in this lab

Storage recovery depends on separate lifetimes and placement constraints. WaitForFirstConsumer can leave a claim Pending normally; local volume affinity can prevent scheduling; replacing a Pod preserves data through its claim. PVC protection delays deletion while in use, and Retain preserves bytes without making a Released PV reusable automatically. Diagnose binding, consumer placement and backing data separately, and plan replication or backup beyond local persistence.

Experiment

Use one 256 MiB declared local PV on lab34-worker, one 128 MiB claim and a sleeping consumer. The two consumer manifests differ only in nodeSelector. No external storage driver or download is needed.

storage_state saves native YAML for the PV, claim and consumer under each phase name, then prints their state. An absent optional claim or Pod is expected in some phases; API failures still stop the helper.

Before running: Predict which objects and bytes survive deleting the Pod, then requesting claim deletion while the Pod is still running.

Measurement key

Signal	Meaning
PVC Pending / WaitForFirstConsumer	A claim may wait for its consumer; inspect the binding mode before diagnosing capacity.
PodScheduled / nodeAffinity	The consumer and local volume must share an eligible node; low live resource use cannot fix a topology conflict.
deletionTimestamp / finalizers	A delete request has started but protection can retain the object while a Pod still references it.
PV Released / claimRef / marker	Released retains the old binding identity; preserved bytes and availability to a new claim are separate facts.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 34 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Inspect the available volume and its binding mode

Run in: VM terminal 1, same shell

```
source ~/labs/lab34/helpers.sh
k get storageclass ce-lab34-local -o yaml
storage_state baseline
k describe pv ce-lab34-data
```

Record: Volume baseline row: PV capacity, class, node affinity, reclaim policy and phase.

Step 2. Create a claim without a consumer

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab34/claim.yaml
k wait pvc/data --for=jsonpath='{.status.phase}'=Pending --timeout=30s --request-timeout=0
storage_state no-consumer
k describe pvc data
```

Record: Claim without consumer row: phase, events and whether a volume has been bound.

Step 3. Require a node that cannot use the local volume

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab34/wrong-node.yaml
k wait pod/store --for=jsonpath='{.status.conditions[?(@.type=="PodScheduled")].reason}'=Unschedulable
--timeout=60s --request-timeout=0
storage_state wrong-node
k describe pod store
```

Record: Wrong-node consumer row: node selector, scheduled condition, scheduling events and claim state.

Step 4. Repair placement and write one marker

Run in: VM terminal 1, same shell

```
k delete pod store --wait=true --timeout=30s
k apply -f ~/labs/lab34/consumer.yaml
k wait pod/store --for=condition=Ready --timeout=90s --request-timeout=0
k exec store -- sh -c 'echo learned-persistence > /data/lesson.txt'
k exec store -- cat /data/lesson.txt
storage_state bound
```

Record: Bound consumer row: Pod UID and node, claim and PV binding, and marker contents.

Step 5. Replace only the consumer

Run in: VM terminal 1, same shell

```
k delete pod store --wait=true --timeout=30s
k apply -f ~/labs/lab34/consumer.yaml
k wait pod/store --for=condition=Ready --timeout=90s --request-timeout=0
k exec store -- cat /data/lesson.txt
storage_state replaced
```

Record: Pod replaced row: old/new Pod UID, unchanged claim binding and marker contents.

Step 6. Request claim deletion while it is in use

Run in: VM terminal 1, same shell

```
k delete pvc data --wait=false
k wait pvc/data --for=jsonpath='{.metadata.deletionTimestamp}' --timeout=30s --request-timeout=0
k get pvc data -o yaml
k exec store -- cat /data/lesson.txt
storage_state deleting
```

Record: Claim deletion requested row: deletion timestamp, protection finalizer, phase and consumer access.

Step 7. Remove the consumer and inspect retained storage

Run in: VM terminal 1, same shell

```
k delete pod store --wait=true --timeout=30s
k wait pvc/data --for=delete --timeout=60s --request-timeout=0
k wait pv/ce-lab34-data --for=jsonpath='{.status.phase}'=Released --timeout=60s --request-timeout=0
storage_state released
k get pv ce-lab34-data -o yaml
docker exec lab34-worker cat /var/local/ce-lab34-data/lesson.txt
```

Record: Volume released row: absent claim, PV phase and old claimRef, and bytes read directly on the storage node.

Step 8. Re-register the retained directory and verify recovery

Run in: VM terminal 1, same shell

```
k delete pv ce-lab34-data --wait=true --timeout=30s
k apply -f ~/labs/lab34/volume.yaml -f ~/labs/lab34/claim.yaml -f ~/labs/lab34/consumer.yaml
k wait pod/store --for=condition=Ready --timeout=90s --request-timeout=0
k exec store -- cat /data/lesson.txt
storage_state restored
```

Record: Storage restored row: new PV/PVC identities, binding, Pod node and original marker.

Recovery check: The new claim is Bound, store is Ready on lab34-worker, and /data/lesson.txt still contains learned-persistence.

Write your answer

Use the observations recorded beside each step.

Phase	Observed state	Evidence / explanation
Volume baseline		
Claim without consumer		
Wrong-node consumer		
Bound consumer		
Pod replaced		
Claim deletion requested		
Volume released		
Storage restored		

1. Why was the first Pending claim normal, and why did the wrong-node consumer remain unscheduled?
2. Which identities changed when the Pod was replaced, and what happened to the file?
3. What delayed claim deletion, and which action allowed it to finish?
4. Why did Retain preserve the marker without making the PV Available, and what did recovery change?

Apply the same reasoning

Would increasing replicas make this local volume available after its only storage node is lost?

Save results before removing this lab's resources and files: `./lab.sh 34 reset`

Lab 35 - Namespace budgets and admission failures

Why can an idle namespace reject a Pod, and why does lowering its quota leave existing Pods running?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 35 setup` on the host. No other lab is required.

Theory you need

ResourceQuota controls aggregate admission within a namespace. This fixture permits 200m of total CPU requests, 256 MiB of memory requests and ten non-terminal Pods. 100m is one tenth of one CPU. Two admitted 100m requests consume the CPU-request allowance even while their applications sleep; quota accounting is not a live utilization measurement.

LimitRange acts on individual container or Pod resource settings. Here it supplies default requests of 100m CPU and 16 MiB memory, default limits of 200m and 64 MiB, and a per-container maximum of 200m and 64 MiB. The same resource-less manifest is rejected before defaults exist and admitted after they supply the fields required by quota.

Defaults and validation are different effects. The oversized fixture explicitly requests only 100m CPU and 16 MiB memory, so it would fit the remaining aggregate quota, but asks for a 128 MiB memory limit. The LimitRange maximum rejects it before any container runs. It never allocates that memory; this is an admission comparison, not a node-exhaustion experiment.

An admission rejection means the proposed Pod object was not created. This differs from an admitted Pending Pod waiting for scheduling and from a running container reaching a memory limit. A quota rejection can return Forbidden even when RBAC allows create pods; read the named policy and resource in the error, rather than diagnosing every 403 as an RBAC denial.

Quota status reports hard allowances and used accounting and can lag a change. The supplied wait reads both before the next comparison. Lowering the quota below current usage does not evict or resize existing Pods. It prevents additional charged admissions; freeing enough allowance or raising it allows a later create request to succeed.

Quotas and per-container bounds reduce the resource footprint that a namespace can request, but do not reserve physical node capacity or provide complete tenant isolation. A permitted request can still fail placement, and memory/CPU limits are enforced by different runtime mechanisms. RBAC must also keep tenants from raising or deleting their own budgets.

Source: docs/chaos-theory.md: §§12.10.1, 12.10.2 (maintained supplement; primary references linked there).

Main lesson to learn in this lab

Namespace budgets govern admission, not current CPU usage. LimitRange supplies defaults and checks each container; ResourceQuota checks their aggregate requests and object counts. A rejected Pod does not exist, while lowering a quota does not evict existing Pods. Read the policy error and observed accounting, then repair the specific constraint; combine budgets with runtime limits and permission boundaries for meaningful resource isolation.

Experiment

Use `ce-lab35` with sleeping Pods, a 200m CPU-request quota and explicit per-container defaults. The oversized manifest tests a memory-limit rule without consuming extra memory.

`wait_budget` waits for the exact CPU hard/used values for up to 60 observations two seconds apart, then prints native quota details. Stop on its error instead of treating stale accounting as the next phase.

Before running: Predict whether each failure happens before Pod creation, during scheduling, or at runtime, and what lowering a quota changes for existing Pods.

Measurement key

Signal	Meaning
quota hard / used	These are admitted resource quantities, not utilization; wait for the controller to publish the intended accounting.
Forbidden message / can-i	The response identifies an admission constraint; an allowed RBAC review does not bypass quota or LimitRange.
Pod resources / UID / Ready	Read injected defaults from the admitted Pod and compare existing identities after a quota decrease.
NotFound after create failure	A rejected creation produced no Pod; there is no application log or scheduler placement to troubleshoot.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 35 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run `./lab.sh ssh` there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without `set -e`. Stop if the baseline fails.

Step 1. Read the working baseline and authorization

Run in: VM terminal 1, same shell

```
source ~/labs/lab35/helpers.sh
k get pod tenant-a -o wide
k get pod tenant-a -o yaml
wait_budget 200m 100m
k auth can-i create pods
```

Record: Explicit baseline row: UID, Ready, declared requests/limits, quota hard/used and create authorization.

Step 2. Submit a Pod without the required requests

Run in: VM terminal 1, same shell

```
k create -f ~/labs/lab35/defaulted.yaml
echo "create exit=$?"
k get pod tenant-b
wait_budget 200m 100m
```

Record: Missing requests row: rejection text, exit status, missing Pod and unchanged accounting.

Step 3. Add defaults and test an explicit per-container violation

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab35/defaults.yaml
k describe limitrange container-budget
k create -f ~/labs/lab35/oversized.yaml
echo "create exit=$?"
k get pod oversized
```

Record: Container limit rejected row: maximum memory limit, requested limit, rejection and absent Pod.

Step 4. Submit the same resource-less manifest again

Run in: VM terminal 1, same shell

```
k create -f ~/labs/lab35/defaulted.yaml
k wait pod/tenant-b --for=condition=Ready --timeout=60s --request-timeout=0
k get pod tenant-b -o yaml
wait_budget 200m 200m
```

Record: Defaults admitted row: injected requests/limits, Pod UID, Ready and total used requests.

Step 5. Exceed the aggregate request budget

Run in: VM terminal 1, same shell

```
k create -f ~/labs/lab35/extra.yaml
echo "create exit=$?"
k get pod tenant-c
wait_budget 200m 200m
```

Record: Aggregate quota rejected row: named quota, requested/used/hard CPU values, exit and missing Pod.

Step 6. Lower the budget below already admitted usage

Run in: VM terminal 1, same shell

```
k patch resourcequota budget --type=merge -p '{"spec":{"hard":{"requests.cpu":"100m"}}}'
wait_budget 100m 200m
k get pods -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.containerStatuses[0].ready'
```

Record: Quota lowered row: hard below used, and both existing Pod UIDs and readiness.

Step 7. Free allowance and retry the rejected request

Run in: VM terminal 1, same shell

```
k apply -f ~/labs/lab35/quota.yaml
wait_budget 200m 200m
k delete pod tenant-a --wait=true --timeout=30s
wait_budget 200m 100m
k create -f ~/labs/lab35/extra.yaml
k wait pod/tenant-c --for=condition=Ready --timeout=60s --request-timeout=0
wait_budget 200m 200m
```

Record: Capacity freed row: accounting before and after deletion, and the successful new tenant-c creation.

Step 8. Restore the original explicit baseline

Run in: VM terminal 1, same shell

```
k delete pod tenant-b tenant-c --wait=true --timeout=30s
wait_budget 200m 0
k delete limitrange container-budget
k apply -f ~/labs/lab35/baseline.yaml
k wait pod/tenant-a --for=condition=Ready --timeout=60s --request-timeout=0
wait_budget 200m 100m
k get pods
```

Record: Baseline restored row: only tenant-a, Ready, 200m hard and 100m used; no temporary defaults.

Recovery check: The 200m quota has 100m used, only tenant-a remains Ready, and the temporary LimitRange is removed.

Write your answer

Use the observations recorded beside each step.

Phase	Observed state	Evidence / explanation
Explicit baseline		
Missing requests		
Container limit rejected		
Defaults admitted		
Aggregate quota rejected		
Quota lowered		

Phase	Observed state	Evidence / explanation
Capacity freed		
Baseline restored		

1. Why did the unchanged resource-less manifest fail before LimitRange and succeed after it?
2. Why did the 128 MiB limit fail even though its requests fit the remaining quota?
3. Why was the third 100m Pod refused while RBAC allowed creation?
4. Did reducing quota evict existing Pods, and which actions allowed a new create to succeed?

Apply the same reasoning

Does a namespace quota of two CPUs guarantee that a two-CPU Pod can be scheduled?

Save results before removing this lab's resources and files: `./lab.sh 35 reset`

Lab 36 - Secret encryption and key-loss recovery

How can the API return the same Secret while storage changes, and why can a live API lose access to it?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 36 setup` on the host. No other lab is required.

Theory you need

Base64 is an encoding, not encryption. An authorized Secret API read returns the data regardless of whether the API server encrypts its storage representation. To test at-rest protection, inspect the etcd record as well as the decoded API value; the two observations answer different questions.

For a configured resource, the first encryption provider writes new records. Here aescbc uses a randomly generated local key named `lab-key1`. The later identity provider allows existing plaintext records to remain readable. Enabling that configuration does not retroactively rewrite older records: the legacy Secret stays plaintext until a later API update stores it again.

This fixture changes one harmless annotation to force a legacy record update without changing its password. Compare its saved bytes before and after that update. The encryption prefix identifies a provider and key name; the marker no longer appears in the stored value. This demonstrates a changed storage representation, not a complete cryptographic or backup audit.

The second configuration deliberately contains a different key and omits `lab-key1`. Installing it restarts this private cluster API server, so an old in-memory Secret response cannot hide the missing decryption key. Process liveness and the running manifest can recover while reading a record encrypted under the omitted key fails. Record the actual read error and exit status; do not infer data loss from an API failure alone.

Restoring the original key configuration makes the existing ciphertext readable again without deleting or recreating the Secret. The final state remains encrypted. Removing encryption configuration while ciphertext remains would not be a valid recovery. Proper rotation retains old decryption keys until required records and retained backups have been migrated or can still be restored safely.

The encryption rule covers all Secrets in this private cluster, not only `ce-lab36`. The exercise reads only two toy records and generates fresh keys per setup. `install_provider` replaces the static Pod manifest through Docker, so recovery does not depend on a working Kubernetes API. It stages the file outside the watched manifests directory, then moves it into place.

Encryption at rest does not revoke authorized API access or protect data from an administrator who possesses both the stored records and the local key file. This local aescbc fixture makes the mechanism visible; it is not a production key-management design. Review managed KMS integration, authorization, protected backups and restoration together. Cluster reset removes this disposable cluster and its lab keys.

Source: docs/chaos-theory.md: §§12.11.1, 12.11.2 (maintained supplement; primary references linked there).

Main lesson to learn in this lab

Verify Secret protection at the storage boundary: base64 is reversible, and authorized API reads still reveal decrypted values. Enabling encryption protects new writes; existing records need a rewrite. A live API can fail to read ciphertext when its key is missing, while restoring that key recovers the unchanged record. Protect key availability and backups alongside confidentiality, and retain old keys until migration is complete.

Experiment

Use two toy Secrets with password ce-lab36-demo. legacy exists before encryption; fresh is created afterward. Never substitute real credentials. Both providers and static Pod variants are prepared automatically in this private cluster.

provider_summary prints key names, never key material. stored_secret reads one exact etcd record, saves its raw bytes under the phase name and reports its prefix and whether the toy password is visible. A missing record or failed read is an error, not evidence of plaintext or encryption.

install_provider key1 or key2 changes only this lab API server manifest. wait_provider checks liveness and the running mirror Pod configuration for up to 90 attempts, with two-second pauses; API request time adds to this wait. Secret readability is checked separately.

Before running: Predict whether enabling encryption changes the legacy record immediately, and whether a live API can read ciphertext after losing its key.

Measurement key

Signal	Meaning
API value / exit status	A readable API value proves access and decryption, not that the etcd record is plaintext.
storage_prefix / demo_password_visible	Compare raw record representations before and after a write; an absent record cannot establish encryption.
provider key names / mirror Pod	The configured key must match the stored record; wait for the restarted API server before testing the missing-key fault.
livez / readyz / Secret read	Process health, readiness checks and the ability to decrypt a particular record are distinct observations.

Run the steps

After setup, run these commands on the host:

```
./lab.sh 36 verify
./lab.sh ssh
```

This is VM terminal 1. For VM terminal 2, open another host terminal and run ./lab.sh ssh there. Keep each shell open while steps reuse variables or functions. Run the blocks in order using Bash without set -e. Stop if the baseline fails.

Step 1. Compare the API value with the original storage record

Run in: VM terminal 1, same shell

```
source ~/labs/lab36/helpers.sh
secret_value legacy
stored_secret legacy baseline
```

Record: Plaintext baseline row: decoded API value, raw storage prefix and toy-password visibility.

Step 2. Enable encryption and inspect the unchanged old record

Run in: VM terminal 1, same shell

```
provider_summary key1
install_provider key1
wait_provider key1
secret_value legacy
stored_secret legacy enabled
```

Record: Encryption enabled row: active key name, API value and the legacy storage representation.

Step 3. Write a new Secret through the encrypted API server

Run in: VM terminal 1, same shell

```
k create secret generic fresh --from-literal=password=ce-lab36-demo
secret_value fresh
stored_secret fresh new-write
```

Record: New encrypted write row: unchanged toy value, storage prefix and marker visibility.

Step 4. Rewrite the legacy record without changing its value

Run in: VM terminal 1, same shell

```
k annotate secret legacy ce-lab36/rewrite=encrypted --overwrite
secret_value legacy
stored_secret legacy rewritten
```

Record: Legacy rewritten row: API value and storage representation after the annotation update.

Step 5. Omit the key needed to decrypt the existing records

Run in: VM terminal 1, same shell

```
provider_summary key2
install_provider key2
wait_provider key2
k get --raw /livez
k get --raw /readyz
secret_value fresh
echo "Secret read exit=$?"
stored_secret fresh missing-key
```

Record: Missing decryption key row: running provider, health responses, exact Secret error and exit status, retained storage prefix.

Step 6. Restore the original decryption key and verify recovery

Run in: VM terminal 1, same shell

```
install_provider key1
wait_provider key1
k get --raw /readyz
secret_value legacy
secret_value fresh
stored_secret legacy restored
stored_secret fresh restored
```

Record: Key restored row: readiness, both readable values and both storage prefixes.

Recovery check: The key1 API server is ready; legacy and fresh return ce-lab36-demo, and both stored records retain the lab-key1 encryption prefix.

Only if normal recovery fails

Step 1. Restore the readable provider if an API wait fails

```
Run in: VM terminal 1, same shell
source ~/labs/lab36/helpers.sh
install_provider key1
wait_provider key1
secret_value legacy
```

Record: Record the restored provider and whether the legacy Secret is readable.

Write your answer

Use the observations recorded beside each step.

Phase	Observed state	Evidence / explanation
Plaintext baseline		
Encryption enabled		
New encrypted write		
Legacy rewritten		
Missing decryption key		
Key restored		

- 1. Why did the API show the same value for plaintext and encrypted storage?
- 2. Why did legacy remain plaintext until the annotation update?
- 3. What proves the missing-key fault changed readability without deleting the stored record?
- 4. Why did restoring key1 recover the data, and what must real key rotation preserve?

Apply the same reasoning

Would encrypting etcd prevent a principal with permission to read this Secret through the API from obtaining its value?

Save results before removing this lab's resources and files: ./lab.sh 36 reset

Solutions

Compare after recording your own results. The expected patterns below assume a working baseline and a confirmed injection. Differences require analysis; do not replace observations with the expected answer.

Lab 00 - Prepare the learning environment

1. Which baseline checks passed, and what value did each one report?

Report the actual checks. The required cgroup results are cgroup2fs and Docker cgroups=2; hello-world must run successfully and the root filesystem must have at least 60 GiB total capacity. A failed check means this environment is not ready for the later experiments.

2. Where is your baseline record, and which environment identities does it hold?

The record is `~/labs/lab00/baseline.txt`. Cite its actual tool versions, source commit, local image IDs and registry digests so another run can be compared with this environment.

3. Why must you fix a failed baseline before injecting a fault?

A fault comparison requires a working no-fault baseline. If the same check already fails, a later failure does not establish that the injection caused it.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check the kernel, cgroup, Docker and disk baseline: cgroup2fs, Docker cgroups=2, a successful hello-world run, and at least 60 GiB total root capacity.

Step 2. Record the source commit, versions and image digests: The book source commit is 3e3ee64db71f51a5e9f79af8562dd4aa913a0e71. Each listed image has a local ID and a registry digest; an inspection error is a missing prerequisite.

Step 3. Save the baseline record: baseline.txt contains the timestamp, all baseline checks and the recorded environment identities.

Apply the same reasoning: hello-world checks shared container tooling. It does not test the Redis service, its listener or the client path used by Lab 01.

Lab 01 - A timeout makes a silent failure manageable

1. What proves the firewall fault was removed and the client works again?

Baseline and recovery should both print PONG and OK True. check_firewall should report that no Lab 01 rule remains. Cite your observed output; an unsuccessful recovery leaves the comparison incomplete.

2. What ended the wait in each of the three cases, and with which exit status?

REJECT should print DEGRADED ConnectionError promptly and client exit=0 because the error is caught. DROP without an application timeout should reach the five-second watchdog, print client exit=124, and have no handler or elapsed line. DROP with TIMEOUT=0.5 should print DEGRADED TimeoutError near 0.5 seconds and client exit=0. Report actual timings and nonzero packet counters for all three cases; zero matched packets do not demonstrate the intended fault.

3. Why does try/except need an application timeout to handle a silent dependency?

try/except reacts only after the dependency call raises an error. A silent DROP can leave that call waiting beyond the experiment's watchdog. The client's connection/read timeout makes the call raise an error the handler can catch; the external watchdog terminates the process without running its fallback.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check the Redis baseline: PONG, then OK True, an elapsed time and client exit=0. Stop if this baseline fails.

Step 2. Case 1 - REJECT without an application timeout: DEGRADED ConnectionError, a short elapsed time, client exit=0 and a nonzero REJECT packet counter. The runner removes its rule before returning.

Step 3. Case 2 - DROP without an application timeout: The external watchdog stops the client after five seconds with client exit=124. There should be no DEGRADED or elapsed line; the DROP rule must have matched packets.

Step 4. Case 3 - DROP with a 0.5-second application timeout: DEGRADED TimeoutError near 0.5 seconds, client exit=0 and a nonzero DROP packet counter. The application handles its own timeout before the watchdog fires.

Step 5. Prove Redis and firewall recovery: PONG and OK True return, and check_firewall confirms that no Lab 01 firewall rule remains. An unreadable firewall is unknown evidence.

Apply the same reasoning: No. Attempt budgets and any backoff accumulate. A whole-request deadline must also cover retries and other work.

Lab 02 - Prove why a process was killed

1. What exit status did the deliberate SIGKILL produce, and why is its cause certain?

Bash should report 137 for the deliberate SIGKILL. The command explicitly sends signal 9 to its own PID, so its cause is known without inferring it from the status.

2. What terminated the memory-limited allocator, and which log lines prove it?

The allocator should exceed its 128 MiB cgroup limit before the 20-second runtime bound. A unit result of oom-kill together with a matching kernel memory-cgroup kill identifies a kernel OOM kill. Match the allocator PID and the saved time window. If the evidence shows timeout or a matching systemd-oomd action instead, report that cause; missing evidence leaves the diagnosis unproven.

3. Why can neither 137 nor the systemd-run status diagnose an OOM kill alone?

137 is consistent with SIGKILL but does not explain who sent it or why; a program can also exit 137 explicitly. systemd-run reports the unit outcome and need not return the child's Bash exit status. The unit and matching cause evidence supply the diagnosis.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check the memory controller and unit state: The memory controller is available and no previous ce-lab02 allocator is active. Stop if either prerequisite is missing.

Step 2. Record a deliberate SIGKILL: Bash normally reports 137. Here you know the cause because the program deliberately sent SIGKILL.

Step 3. Run the allocator within its memory and time limits: The unit should fail before its 20-second runtime bound. Its nonzero command status alone does not identify the cause; the next step collects the evidence.

Step 4. Match the termination to the unit and kernel logs: A kernel OOM diagnosis requires the unit's oom-kill result and matching allocator PID in the kernel log. A matching oomd action or runtime timeout gives a different diagnosis.

Step 5. Stop the allocator if it remains active: ce-lab02 is inactive or already collected; no allocator remains active.

Apply the same reasoning: No. The allocator is constrained by its cgroup boundary. VM-wide availability and the cgroup's remaining allowance are different quantities.

Lab 03 - Restart policies have limits

1. How did A's PID, state and restart count change after one crash?

After one SIGKILL, A should become active again with a different MainPID and a higher NRestarts value. These observations establish replacement of the process; use your measured values.

2. What stopped A from restarting during the burst, and which journal line proves it?

The burst should exhaust the four-start allowance within 20 seconds, leaving A failed with a start-limit result and a matching journal message. Restart=always requests another start but cannot override the rate limiter. If the limit was not reached, report the observed state and timing instead.

3. Why does the proxy's HTTP result not establish A's health?

The proxy may keep returning HTTP 200 because B remains available. Cite each actual HTTP result. A shared response body does not identify which backend answered, so inspect A independently.

4. Which recovery commands restored A, and what proves both backends and the proxy recovered?

reset-failed clears the failed state and start counter; start requests a new process. Both backends should report active and the proxy should return lab03 OK. Record any failure to recover.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Check baseline and clear the previous start counter: Both backends are active, A has a nonzero PID, Restart=always is set, and the proxy returns lab03 OK with HTTP 200. Stop if this baseline fails.

Step 2. Kill A once and inspect its recovery: A should return to active. A successful proxy response alone cannot prove that A restarted.

Step 3. Crash A repeatedly and inspect the refused restart: Look for "Start request repeated too quickly" and a failed unit. Result and restart counts can vary with timing and systemd version; check the journal.

Step 4. Recover and prove the proxy works again: Both backends are active and the proxy returns lab03 OK with HTTP 200 after the listeners recover.

Apply the same reasoning: It would test whether the rate limit caused the supervisor to give up. The application would still crash, and rapid restarts would still consume resources and need monitoring.

Lab 04 - CPU contention and a quota

1. What were the mean and range of iteration times in each of the four phases?

Use the printed mean and range for all four phases. The unrestricted neighbour should increase iteration time; the capped neighbour should reduce that increase. There is no fixed required speedup, and small differences within baseline variation do not establish a clear effect.

2. Was the neighbour active throughout each loaded phase, and is any phase inconclusive?

Both loaded measurements require an active neighbour before and after work.py. If the neighbour ended early, that phase mixes contention and recovery and is inconclusive.

3. What ratio did cpu.max show, and how much did the throttling counters rise?

cpu.max should have $\text{quota/period} = 0.2$. Subtract the before counters from the after counters; rising nr_throttled and throttled_usec show the competitor used its budget and was throttled.

4. Why did capping the competitor change the measured job's time?

The two jobs share one CPU. Restricting the competitor's CPU time should leave more scheduling time for the measured job. Quota enforcement plus shorter measured times and recovery toward baseline support that explanation; PSI and vmstat are supporting VM-wide signals.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Choose an allowed CPU: One CPU permitted by this shell's affinity; use the same shell and CPU for every phase.

Step 2. Record the baseline three times: Three runs produce 18 finite iteration timings with no deliberate competitor.

Step 3. Add a CPU neighbour on the same CPU: Compare iteration time with baseline. PSI and vmstat provide supporting waiting measurements; neither alone proves which task caused the slowdown.

Step 4. Cap the neighbour and read the throttling counters: `cpu.max` should show a quota/period ratio of 0.2, and `cpu.stat` should show throttling. If the neighbour ends before measurement finishes, this phase is inconclusive.

Step 5. Recover and repeat the baseline: The neighbour is no longer active. Recovered timings should move toward the baseline range; report measured variation rather than assuming exact equality.

Apply the same reasoning: No. It changes placement and removes the intended competition for a single CPU. Keep placement fixed when evaluating the quota.

Lab 05 - A container is several Linux mechanisms

1. Which mechanisms produced the sandbox's filesystem and PID views?

The sandbox should show the exported BusyBox tree under `/` and its shell as PID 1 with only sandbox processes in `ps`. `chroot` changes its filesystem root; the PID namespace and fresh `proc` mount provide the process view. The VM still sees the sandbox under an ordinary host PID.

2. What CPU, memory and task limits did the host read from the sandbox's cgroup?

`cpu.max` should have `quota/period = 0.2`, `memory.max` should be 134217728 bytes, and `pids.max` should be 50. Cite the actual values; these cgroup settings are independent of the filesystem and PID views.

3. How much did `nr_throttled` and `throttled_usec` rise while the busy loop ran?

Subtract the first `cpu.stat` snapshot from the second. Increasing `nr_throttled` and `throttled_usec` while the loop runs show active enforcement. A configured cap or an old nonzero total alone does not prove throttling during this measurement.

4. Why can a separate PID view exist with or without a CPU cap?

Namespaces control visibility while cgroups control resource budgets. A separate PID view can exist with or without a CPU cap; the sandbox still shares the VM's kernel and, here, its network.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Confirm the exported BusyBox root filesystem: The exported tree contains `bin` and `proc`, and no previous sandbox scope is active.

Step 2. Start the sandbox; this shell becomes the sandbox: Terminal 1 presents the BusyBox shell and stays inside the sandbox until you run `exit`.

Step 3. Inspect the view inside the sandbox: `$$` is 1, `ps` lists only the sandbox processes, and `/` is the BusyBox tree.

Step 4. Read the limits and PID namespaces from the host: `cpu.max` 20000 100000, `memory.max` 134217728, `pids.max` 50; `lsns` shows a separate PID namespace whose command is `sh`.

Step 5. Run a finite busy loop inside the sandbox: The loop runs for at most 30 seconds before timeout terminates it and the sandbox prompt returns. While it runs, switch immediately to VM terminal 2 and run the next step.

Step 6. Read the throttling counters from the host: `nr_throttled` and `throttled_usec` increase during the loop. `top` may show the busy `sh` process near 20 percent CPU; the cgroup counter differences provide the direct enforcement evidence.

Step 7. After the loop finishes, exit the sandbox: After the finite loop ends, `exit` closes the BusyBox shell and returns terminal 1 to the VM shell.

Step 8. Confirm the scope stopped: The scope is inactive or already collected after the sandbox exits.

Apply the same reasoning: No. The PID view is controlled by the PID namespace and `proc` mount. Removing a bandwidth limit changes resource consumption, not visibility.

Lab 06 - Separate containers can share a full filesystem

1. What did the same one-MiB write return when the mount was empty, full and freed?

The one-MiB probe should succeed before filling, fail while the shared mount is full, and succeed after the files consuming space are removed. Cite the actual exit statuses and df values for all phases.

2. Which error and capacity readings prove the failure was shared-space exhaustion?

The filling write should reach No space left on device and df should show no available space on the 32 MiB tmpfs. The later probe should report the same capacity error. A permission error or a different mount does not establish this explanation.

3. Why did a fresh container still fail, and which action restored its ability to write?

Each docker run creates a fresh container but bind-mounts the same tmpfs. Removing the writer container leaves its file on that shared filesystem. Deleting the filling file restores capacity; the successful recovery probe demonstrates that shared-space exhaustion caused the failure.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Mount and identify the bounded shared filesystem: This exact lab path is a tmpfs with 32 MiB total capacity. Stop if the mount failed or a previous experiment remains.

Step 2. Baseline - write one MiB before filling: The one-MiB write succeeds before space is consumed.

Step 3. Fill the 32 MiB tmpfs from one container: The 40 MiB attempt exceeds the 32 MiB mount, dd reports No space left on device and df shows no available space. The printed dd status is the write's status; df may leave the container status zero.

Step 4. Repeat the probe from a fresh container while full: This new container's one-MiB write also fails with No space left on device and a nonzero status.

Step 5. Free capacity and repeat the identical write: The same probe succeeds after capacity is freed.

Step 6. Remove the probe and unmount the temporary filesystem: The directory is no longer a mountpoint and no ce-lab06 container remains.

Apply the same reasoning: Its file is stored in the host bind mount, outside the disposable container layer. Remove that file or the temporary filesystem to release the capacity.

Lab 07 - Network delay accumulates across sequential calls

1. What were the mean and range for one and four exchanges in each phase?

Report your observed mean, minimum and maximum for both exchange counts in each of the four phases. The 1 ms control should add a small delay, with timing noise; it is not a zero-cost control.

2. How much time did the 25 ms delay add to one exchange, and to four?

For the 25 ms phase, subtract the baseline for one exchange from its delayed mean and do the same independently for four exchanges. Expected additions are roughly 25 ms and 100 ms because each reply is another sequential wait. Report deviations from these estimates rather than replacing measured values.

3. Which qdisc and counter evidence proves the delay reached the measured traffic?

netem delay 1ms and then 25ms should appear only on ce-lab07's eth0. Increasing Sent packet/byte counters between the before and after snapshots establish that measured traffic crossed the queue; configuration alone does not.

4. Did exiting tc remove the delay, and which measurements show recovery?

The qdisc remains after the installing tc command exits. `net qdisc del dev eth0 root` removes it. Recovery requires both the absence of netem and echo timings moving toward each case's own baseline.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Measure the unchanged dependency: The client receives every echo; eth0 has its default noqueue, with no custom root qdisc. Stop if either check fails.

Step 2. Measure a 1 ms control: netem remains installed after tc exits, with traffic counters. Added time should be small; record the range.

Step 3. Change only the dose to 25 ms: Four sequential exchanges should accumulate more delay than one. Compare baseline-subtracted times, not raw ratios.

Step 4. Remove the rule and repeat both probes: No netem remains and timing returns toward baseline.

Step 5. Calculate added time from each case's own baseline: Added time at the 25 ms dose should be roughly 25 ms for one exchange and 100 ms for four, with packet and scheduling variation. Recovery deltas should approach zero.

Apply the same reasoning: No. Sequential dependency waits add along a request's critical path. Independent parallel waits can overlap; other overhead still needs measurement.

Lab 08 - A syscall error tests application handling

1. What did the normal trace show, and did its fsync error stop the server?

A normal request should show writes followed by fsync and close. This fixture ignores the socket fsync EINVAL error and continues serving; an error line alone does not identify an outage cause.

2. What did the injected close return, and why did the server exit?

The injected close should show -1 EIO and INJECTED. The matching "error closing socket" log and exit status 1 identify the application decision to exit. If the marker or exit was not observed, report the missing evidence instead of assuming the error path ran.

3. Why did the triggering request still return 200 while the next one failed?

The triggering request may receive HTTP 200 and body bytes before close fails, because writing precedes closing. After the server exits, the next request should show 000 and no body. Restart should restore HTTP 200. Use the recorded result for each request; a completed triggering response does not prove continued availability.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Start the server and save its PID: HTTP should return 200 and body bytes. If it does not, inspect server.log before continuing.

Step 2. Attach the normal tracer and leave it running: The tracer waits for server syscalls; it does not inject faults.

Step 3. Send one normal traced request: The request should succeed and the server should remain running.

Step 4. Read the normal trace after detaching: Many write calls, one fsync and one close per request. fsync on a socket returns -1 EINVAL, which is not an outage cause.

Step 5. Attach the close-error injector and leave it running: The next close is replaced by an EIO return; trace output is saved in injected-trace.txt.

Step 6. Trigger the fault and collect the process result: The injected close should show EIO and INJECTED. The server should log "error closing socket" and exit 1; the response may already have reached the client.

Step 7. Test the next request independently: After the server exits, expect connection failure, HTTP 000, no body and a nonzero curl exit.

Step 8. Restart, verify HTTP recovery and stop the server: HTTP responds after restart; the test server is then stopped.

Apply the same reasoning: No. strace can skip the real call and substitute an error. Real close errors have syscall-specific semantics. The supported conclusion is about this application's reaction to the injected

return value.

Lab 09 - A syscall denial need not crash the process

1. What did getpid return before and after the filter, and with which errno?

Before filtering, a successful getpid returns a positive PID. After filtering, syscall returns -1 and errno=13 (EACCES), matching the rule. Only interpret errno after an error return.

2. Why can the program still print, and what does its exit status mean?

The filter denies getpid while allowing other syscalls, including writing output. Exit 0 means the expected denial was detected, not that the denied getpid succeeded. Exits 1-3 mean filter setup failed; exit 4 means the expected denial was absent.

3. Does a fresh process inherit the filter, and what shows that?

A fresh process started by the original shell should obtain a positive PID. Its Seccomp state may reflect an inherited environmental policy, but it does not inherit a filter installed only in the exited demonstration child. The filter is not host-wide.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Run the filter and capture its exit status: Before filtering, getpid prints a positive PID. After filtering, getpid result=-1 errno=13 and exit=0. Exit 1-3 means filter setup failed; 4 means the expected denial did not occur.

Step 2. Start a fresh instance from the original shell: The fresh instance should obtain a positive PID before installing its own filter, then observe EACCES and exit 0 again. Seccomp can reflect a pre-existing inherited policy; the first child did not install a host-wide filter.

Apply the same reasoning: No. It frees the library context in userspace. The kernel filter remains installed; the demonstration's later getpid call tests exactly this.

Lab 10 - Ownership, disruption budgets and Service routing

1. Which controller replaced the deleted Pod, and how does the UID prove replacement?

The Pod is owned by a ReplicaSet, which is owned by the Deployment. Deletion leaves desired replicas at three, so the ReplicaSet creates a new Pod. A new name and UID establish replacement; the old UID should be absent after deletion completes.

2. Why was the eviction refused while the direct deletion succeeded?

With three healthy Pods and minAvailable=3, disruptionsAllowed should be zero and eviction should be rejected for violating the budget. Direct DELETE bypasses that guard. Forbidden or a successful eviction invalidates this comparison and needs investigation.

3. How many of the 100 probes failed, and what does that number establish?

For 100 completed probes, failures are non-200 rows and availability is $(100 - \text{failures}) / 100 \times 100\%$. Report the measured value. Zero failures means these probes saw none; replacement alone and unobserved time between probes do not establish uninterrupted service.

4. Why did Ready Pods stop answering when the Service selector changed?

With the unmatched selector, Pods can remain Ready while ready Service endpoints fall to zero and HTTP fails. Restoring the selector should restore three ready endpoints and HTTP without a workload rollout. Repeat endpoint observations if updates are still converging.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Checkpoint the prepared chaos cluster: Three Ready replicas, matching endpoints and a working /healthz probe. The workload may list Pods but may not delete them.

Step 2. Trace the Pod, ReplicaSet and Deployment owners: The Pod owner should be a ReplicaSet whose owner is deployment/goldpinger.

Step 3. Create the disruption budget and attempt eviction: The eviction is refused because it would violate the budget; the original UID still exists. Stop if the error is Forbidden or if eviction succeeds, since those do not establish the intended comparison.

Step 4. Start 100 probes just before the deletion: Each probes.log row contains a timestamp and HTTP code. A 000 means no HTTP status was received.

Step 5. Delete the same Pod directly and watch the replacement: Direct deletion should be accepted despite the PDB and a new Pod should appear. Exit 124 only ends the timed watch.

Step 6. Collect recovery evidence after the probes finish: The deleted UID should be absent and three Ready replicas restored. Use the actual failed-sample count; zero failures only describes these probes.

Step 7. Break only the Service selector: Ready Pods can coexist with zero eligible Service endpoints and a failed request. Recheck endpoints if updates are still converging.

Step 8. Restore the selector and confirm routing recovery: Three eligible endpoints and successful HTTP return without a workload rollout.

Apply the same reasoning: No. Replica count does not guarantee placement across failure domains. This experiment deletes one Pod; it does not test node or zone loss.

Lab 11 - Health depends on what you test

1. Why did TCP readiness stay true at 400 ms while peer pings failed?

With adequate timing margin, 100 ms should increase peer latency while remaining within the 300 ms budget. At 400 ms peer pings should time out, while TCP readiness can still succeed because the proxy accepts a connection before forwarding delayed data. Record the actual result from each peer and use fresh PingTime values.

2. Which budget did each HTTP dose break, and when did readiness change?

At HTTP / 0 both checks should succeed. At 400 ms peers can miss their 300 ms budget while HTTP readiness remains within one second. At 1400 ms both budgets are exceeded; readiness changes after consecutive failed checks. Missing or stale peer reports do not establish a timed-out request.

3. What happened to the slow Pod endpoint, and can peers still reach its IP directly?

When HTTP readiness fails, the slow Pod endpoint should become ineligible for ordinary Service routing. Baseline replicas may keep the Service available. Goldpinger discovers labelled Pod IPs directly, so an unready endpoint does not necessarily stop direct peer pings.

4. Did the Pod recover by readiness alone, and which values prove that?

Removing the toxic should restore fresh peer success and Ready/endpoint readiness with the same HTTP-comparison UID and restart counts. A changed UID between TCP and HTTP baselines was intentional recreation. A change during fault removal would require a different explanation than readiness-only recovery.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Connect the proxy and record the TCP baseline: The extra replica is Ready and peer reports for its IP show OK true. Establish this baseline before adding delay.

Step 2. Set 100 ms delay and record the TCP comparison: With enough margin, peer pings remain healthy but take longer. Readiness should remain true; record actual responses.

Step 3. Set 400 ms delay and record the TCP comparison: Look for failed peer pings while the Pod remains Ready. The 400 ms delay exceeds the 300 ms ping budget; readiness only checks TCP connection establishment.

Step 4. Switch the extra Pod to HTTP readiness: Record a fresh UID for the HTTP configuration and a healthy baseline before injecting again.

Step 5. Set 400 ms delay and record the HTTP comparison: Peer requests can fail at 300 ms while HTTP readiness still succeeds within one second.

Step 6. Set 1400 ms delay and watch HTTP readiness change: The slow Pod becomes unready and its endpoint is not ready. UID and restart counts stay unchanged; other Service endpoints remain available.

Step 7. Remove the toxic and record recovery in the same Pod: The toxic list should be empty, fresh peer pings should recover, and Ready/endpoint readiness should return without a UID or restart-count change.

Step 8. Stop the port-forward and remove the extra Pod: The baseline Deployment should finish its rollout with three Ready replicas.

Apply the same reasoning: No. A healthy process can fail a dependency-based liveness check and restart repeatedly without repairing that dependency. Readiness controls routing; liveness should detect a condition a restart can plausibly fix.

Lab 12 - Locate startup failures and measure recovery

1. Which startup stage failed in each of the three deliberate faults?

A normal completed trial should meet the 30-second target. The delay trial should show an assigned but unready Pod; the impossible nodeSelector should show no node and Unschedulable; the selector fault can leave a Ready Pod with zero ready endpoints. Each should miss the HTTP deadline. Use the saved timestamped conditions, not success=0 alone, to locate the failed stage.

2. How do you distinguish a completed deadline miss, missing diagnostics and a checker execution error?

Exit 0 plus a fresh result with success=0 is a completed deadline miss. A diagnostic_error means that resource observation is unavailable even if timing completed. Nonzero checker exit means execution or cleanup failed; do not reuse old metrics as a new result. Compare trial timestamps and ce_last_run_timestamp_seconds to establish freshness.

3. What was the restored success fraction, and what does that sample establish?

Divide successful restored trials by three completed restored trials; startup_summary calculates the fraction and percentage. Three successes give $3/3=1$, but report actual results. Exclude deliberate faults and do not count an execution error as a completed miss. This tests repeatability on cached images and does not establish a production SLO.

4. Which commands restored the workload, and would a restart have repaired the faults?

Copying startup-good.yaml to startup.yaml removes the injected configuration, and run.sh creates fresh resources for each restored trial. Final Pod and Service gets must both return NotFound; connection errors are inconclusive. A container restart cannot change placement or Service selectors and would repeat the configured 45-second sleep.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Open the prepared workspace: Loading the supplied commands changes no cluster state. A diagnostic_error in a later trial remains missing evidence, not a zero-endpoint result.

Step 2. Run one healthy baseline: A completed healthy trial should report success=1 within 30 seconds, with an assigned Ready Pod and a ready Service endpoint. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 3. Run the 45-second startup-delay fault: The configured 45-second sleep should miss the 30-second budget. The Pod should be assigned to a node but not Ready at capture. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 4. Run the impossible-placement fault: Expect a completed miss with no nodeName and PodScheduled=False / Unschedulable. No application container can start on a node. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 5. Run the wrong-Service-selector fault: Expect a completed miss with an assigned Ready Pod and no matching ready Service endpoint. Both final gets should report NotFound; an API error is not cleanup evidence.

Step 6. Restore the healthy manifest and collect three trials: Each completed healthy run should meet the 30-second target; record actual misses. start.prom contains only the last trial, so it cannot supply the three-trial fraction.

Step 7. Calculate the restored sample success fraction: Three successful restored runs give $3/3 = 1.000$ (100%); otherwise use the observed count. This small controlled sample is not a long-term SLO.

Apply the same reasoning: No. It checks the registry for the image resolution, but cached layers can still be reused. A cold-image experiment must control the node's cache as well.

Lab 13 - Placement, supervision and replacement

1. What did cordoning change, and what did deleting the Pod change afterwards?

Cordon should set `unschedulable=true` while the same Pod UID and container remain and kubelet stays active. Deleting that Pod should produce a new UID on another eligible node. Uncordon permits future placement but should not move that replacement automatically.

2. How long after the kubelet stopped did the node go unavailable, and the replacement appear?

After kubelet stops, Lease `renewTime` should become stale, followed by node-unavailable handling, NoExecute taint and possible replacement. Report the measured stop-to-first-unavailable and first-taint-to-first-replacement sample differences. They include sampling uncertainty and separate stages; the 20-second toleration starts at tainting, not at the stop command. Mark absent events not observed.

3. Did the original container keep running while the replacement started?

If `crictl` still lists `OLD_CID` while the API shows a different Pod UID elsewhere, the old process and replacement coexist. Kubelet reporting and desired API state do not themselves stop a runtime container. A runtime query error is unknown, not evidence that the old process stopped.

4. What evidence confirms recovery, and what remains unknown if no replacement was observed?

After restoring kubelet, all four nodes should become Ready, workers should be uncordoned and node-web should have one Ready Pod. If replacement occurred, `OLD_CID` should disappear as the old node reconciles. If no replacement appeared within the sampling bound, report that limit and still verify kubelet recovery; do not invent an eviction or replacement time.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the original Pod and runtime container: One Ready node-web Pod on a worker; the two NoExecute tolerations explicitly say 20 seconds. Save its container ID.

Step 2. Cordon the host and inspect the existing Pod: The same UID and container remain while the node is cordoned; kubelet still runs.

Step 3. Delete the Pod while its node is cordoned: The Deployment should replace the deleted Pod on another eligible node.

Step 4. Uncordon and save a fresh baseline for the kubelet trial: Uncordon should not move the current Pod back. Treat its UID/container as the original identities for the separate kubelet failure.

Step 5. Stop kubelet and save command-completion time: kubelet should report inactive (a nonzero is-active exit is expected). The existing application container can remain running.

Step 6. Sample heartbeats, node state and replacement 18 times: The Lease should stop advancing before unavailable handling and eviction. A replacement may appear while `OLD_CID` still runs; if it does not appear within this bound, record not observed and restore kubelet.

Step 7. Restore kubelet and check convergence: Four Ready nodes. If a replacement was created, the old container should disappear as kubelet reconciles. Recheck after a short wait if it is still terminating.

Step 8. Calculate the two observed intervals: These are differences between observation times, not exact transition latencies. A 20-second toleration is only one component of recovery.

Apply the same reasoning: Removing an API object does not fence or stop an unreachable machine. The old process can continue; preventing concurrent writers requires an appropriate fencing or application coordination mechanism.

Lab 14 - Quorum, convergence and application availability

1. What are quorum and tolerated failures for 3, 4 and 5 voters?

Quorum is $\text{floor}(n/2)+1$ and tolerated failures are $n-\text{quorum}$: 3 voters need 2 and tolerate 1; 4 need 3 and tolerate 1; 5 need 3 and tolerate 2. Stopping a member does not remove it from configured membership.

2. With one member stopped, did the write, the convergence and HTTP still work?

Two surviving voters can commit the one-down value and the desired replica count of three. A completed rollout and Ready count of three separately establish convergence. HTTP can remain available. A leader change is required only if the stopped member was leader or another election occurred; compare recorded member IDs and leader status rather than assuming an election.

3. With two members stopped, why did writes fail while existing HTTP kept working?

With one surviving voter, new commits cannot proceed. Record the actual ConfigMap/scale errors and HTTP result separately: existing routing and web processes can keep serving without new etcd writes. A timeout means the caller did not receive an acknowledgement; it is not a general proof that a submitted operation never committed.

4. What did etcd still hold after recovery, and what proves the baseline returned?

After endpoint health returns, read stored ConfigMap state/resourceVersion and Deployment spec before changing them. The expected unchanged targets are one-down and three replicas, but the observed values resolve uncertainty. Then set recovered and replicas=2 idempotently; successful readback, two Ready replicas, three healthy etcd endpoints and HTTP verify distinct parts of recovery.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Calculate quorum and establish the baseline: Four Ready nodes, 2/2 web pods, HTTP ok, "configmap/quorum-probe patched" and a table with three members, one IS LEADER=true.

Step 2. Stop one member and test writes, HTTP and leader state: Stop exit must be zero and the runtime list empty before continuing. Two voters keep quorum, so the write should succeed after any transient delay while HTTP continues. Stopping a follower should not change the leader.

Step 3. Scale with two voters and observe convergence: The scale to three should commit and converge while quorum survives. A successful scale response alone does not prove the replicas are Ready.

Step 4. Stop the second member and compare writes with HTTP: Confirm the second etcd process actually stopped. One of three voters cannot commit new writes; existing HTTP may continue. A client timeout is an unacknowledged outcome that must be checked after recovery.

Step 5. Restore the two members and wait for endpoint health: Continue only with healthy=1. Restoring the original manifests recovers stopped members with intact data; it does not test backup restoration.

Step 6. Read persisted values before restoring configuration: The stored value normally remains one-down and desired replicas remain three, but use actual readback to resolve uncertain responses. Do not overwrite state unless health and both reads succeeded.

Step 7. Restore the explicit baseline and verify all paths: All three endpoints are healthy; the recovered value is readable, desired and Ready replicas are both two, and HTTP works.

Apply the same reasoning: No. Four members require three votes and tolerate one failure, just as three members tolerate one. Five members require three and tolerate two.

Lab 15 - Count how layered retries multiply work

1. What were the client attempts, backend arrivals and successes in each of the six phases?

Use the six printed phase summaries as your measurements. Each phase submits 20 originals; amplification is $\text{backend_arrivals} / 20$ and success fraction is $\text{successes} / 20$. Check both runner exits are zero before comparing cases.

2. What is the maximum backend arrivals for zero and for one client retry?

With zero client retries, $20 \text{ originals} \times 1 \text{ client attempt} \times 2 \text{ proxy attempts}$ allows 40 backend arrivals (2x). One client retry allows $20 \times 2 \times 2 = 80$ (4x). Persistent 503 should reach those bounds in this fixture; explain any lower observed count from the logs rather than replacing it with the prediction.

3. Why did the extra attempts add no successful requests, and what did cleanup show?

Both fault cases can finish with zero successes because every backend attempt returns 503. Healthy and recovered phases should need 20 client attempts and 20 backend arrivals for 20 successes because the first success ends retrying. No fail marker or listeners on 8090, 9001 and 9002 confirms the runner removed its fault and processes. This establishes work multiplication, not a self-sustaining overload.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Run the three phases with zero client retries: The runner measures healthy, persistent-503 and recovered phases in order, with 20 originals each. It removes the fault on exit.

Step 2. Change only the client retry count from zero to one: A client retry can double backend work while both upstreams keep failing. Successful phases end retrying after the first success.

Step 3. Compare the six summaries and confirm cleanup: The saved summaries retain both cases. The fault marker is absent and no listeners remain on the three lab ports.

Apply the same reasoning: Three retries means four attempts at each layer, so at most $4 \times 4 = 16$ backend arrivals per original request, provided every attempt reaches the backend and keeps failing.

Lab 16 - Design one clear experiment

1. Which service, measurement and baseline command did you choose?

The completed card should identify one service/environment and an executable measurement with units, load and duration. The baseline command and fault-phase measurement must use the same operation and window so their results can be compared. A statement such as "service is healthy" is insufficient.

2. Which fault, target and duration did you choose, and what threshold do you predict?

The card should name one fault and bounded target/duration, provide the exact injection command, and provide a separate command that confirms the target count and dose. The predicted threshold must use the chosen measurement and have a stated mechanism; it remains a prediction until tested.

3. What stops the experiment, and which commands roll it back and prove recovery?

The stop threshold states when to abort. Rollback states exactly how to remove the fault; the fallback covers loss of the normal removal path. Recovery requires an executable check against an explicit service criterion, not merely a successful cleanup command.

4. What can this plan not establish?

A completed plan is not evidence of resilience. Review each field for a clear target, executable command, measurement and recovery criterion. The hypothesis remains untested until you run the experiment.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read the card and choose one service and one fault: Setup creates the card only if it is absent, so an existing plan is preserved. This lab does not execute the proposed fault.

Step 2. Write the complete plan, including executable commands: The saved card should let another operator understand each operation without guessing missing commands or values. Leave observations explicitly unmeasured.

Step 3. Read the saved plan and check each field: Every field should be filled with a concrete choice. Reading the plan is a review of its meaning; a nonempty field alone does not make an experiment valid.

Apply the same reasoning: No. The card is a plan, and the control establishes the measurement path and baseline. Only an observed fault trial and analysis can support the specific resilience claim.

Lab 17 - Signal delivery and the shutdown budget

1. Why did the worker receive TERM under exec but not under the wrapper?

The wrapper case should report a worker PID other than 1, no received=15 line and no cleanup marker: the wrapper ignores TERM and does not forward it. The exec/five-second case should report PID 1, received=15, a completion line/file and exit zero. Cite the actual logs and Docker state for each case.

2. Which evidence separates signal delivery from having enough time to finish cleanup?

Both exec cases should receive TERM. Five seconds accommodates the two-second cleanup; one second should allow the receipt/start log but prevent the completion marker. Correct signal delivery and sufficient cleanup time are separate requirements.

3. What caused each exit 137, and why is that not an OOM diagnosis?

For the wrapper and short-budget cases, exit 137 with OOMKilled=false and the controlled docker stop operation is consistent with a forced shutdown after the grace budget. Exit 137 alone is not an OOM diagnosis. The final container-list command should be empty after evidence collection and removal.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Start the wrapper case and confirm the worker is ready: The wrapper is PID 1 and the Python worker is its child, so the worker PID is not 1.

Step 2. Stop the wrapper with five seconds and collect the result: The child should receive no TERM, create no marker and be forcibly stopped after the grace budget. A missing marker-copy source is expected in this case.

Step 3. Start the exec case with the same application: Exec makes the worker PID 1 so Docker can deliver its stop signal directly.

Step 4. Stop exec with five seconds and collect the result: The worker should receive TERM, finish its two-second cleanup, write completed and exit zero.

Step 5. Start another exec case for the shorter budget: The worker is PID 1 again; the next stop changes only the time budget.

Step 6. Stop exec with one second and collect the result: TERM should reach the worker, but the one-second budget is shorter than its cleanup. A missing marker-copy source is expected.

Step 7. Remove the three stopped containers after recording evidence: No ce-lab17 containers remain; the successfully copied exec-complete.txt file remains in the VM workspace.

Apply the same reasoning: No. More time cannot make this wrapper forward a signal it ignores. Repair the process relationship or implement correct forwarding first.

Lab 18 - Persistence, ownership and read-only mounts

1. Which file survived container replacement, and why did the other one disappear?

The original container creates /ephemeral.txt in its writable layer and /data/persisted.txt in the named volume. After that container is removed, the replacement should read persisted.txt but find no ephemeral.txt. The named volume has a separate lifetime; this observation does not establish a backup.

2. Why did UID 10001 fail to write, and which change repaired it?

Initially /data is owned by UID 0 with mode 0755, so UID 10001 lacks directory write permission. chown 10001:10001 /data assigns only the lab directory to the intended identity. The repeated write should then return zero without changing its mode or granting world write access.

3. Why did the same user still fail through the read-only mount?

With ownership unchanged, a read-only mount should fail with Read-only file system and RW=false. Restoring a writable mount should produce RW=true, a successful write and readable recovered/volume contents. This separates mount enforcement from Unix ownership and from data loss. Record those results before deleting the dedicated test volume.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Create one file in each storage layer: Both files exist in the original container; only /data is backed by ce-lab18-data.

Step 2. Replace the container and check which file remains: The volume file survives replacement while the removed container's writable-layer file does not.

Step 3. Attempt the write as UID 10001 and diagnose the failure: UID 10001 should get Permission denied because UID 0 owns the directory and mode 0755 permits only the owner to create files.

Step 4. Change only the directory owner and repeat the write: The write should succeed as the new owner; no chmod 777 is needed.

Step 5. Change only the mount to read-only and inspect the failure: Correct ownership cannot override RW=false; the write should fail with Read-only file system.

Step 6. Restore a writable mount and confirm both data and writes: The write should succeed with RW=true, and the output should contain recovered plus the original volume content.

Step 7. Remove the exercise containers and volume after recording results: The cleanup removes only the dedicated exercise storage. original-layer.txt remains outside the volume as saved evidence.

Apply the same reasoning: No. Mount read-only enforcement is independent of Unix mode bits. For an ownership mismatch, inspect the required identity and grant only the needed access.

Lab 19 - Unscheduled Pods versus memory-killed containers

1. Which stage failed for the 1000-CPU request, and what shows it?

The healthy fixture should be node-assigned and Ready with a 20m CPU request. The 1000-CPU request should remain unassigned with PodScheduled=False/Unschedulable and current-UID scheduling events showing insufficient CPU on eligible workers. Its application has not started, so a failed logs request is not an application crash.

2. What terminated the container under the 32 MiB limit, and how do you know?

The OOM fixture retains a fitting 16 MiB memory request but has a 32 MiB limit and deliberately allocates 96 MiB. It should be assigned to a node, then show OOMKilled in the previous termination, typically exit 137 and a positive restart count. The previous log should identify the allocation. Use OOMKilled and the resource evidence, not exit 137 or CrashLoopBackOff alone, to diagnose the runtime failure.

3. Which UID changes were deliberate replacements, and which restart happened inside one Pod?

Applying good.yaml after deleting the faulty Pod should produce a new Ready Pod with the healthy resource configuration and no previous OOM/repeated restarts. The command sequence deliberately changes UID between fixtures. Compare the OOM UID captured at creation with the crash snapshot to show that kubelet restarted a container inside the same Pod.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the healthy placement and configured resources: The baseline should be assigned and Ready. Requests describe scheduling demand; limits describe runtime bounds. The readiness probe fetches the page every second, so the log tail only confirms that the application is serving.

Step 2. Replace the baseline with the impossible CPU request: The request is 1000 CPUs, not 1000m. No eligible worker can satisfy it, so the Pod should remain unassigned.

Step 3. Collect scheduling events and check application-log availability: Events should explain failed placement. The logs request should fail because there is no started application container.

Step 4. Replace it with the memory failure and wait for OOM evidence: This request fits, but the deliberately oversized allocation should exceed the runtime memory limit.

Step 5. Save the runtime crash, prior logs and current-UID events: The previous instance should be OOMKilled inside the same Pod UID. Capture the reason and log while the Pod still exists; rerun this observation if restart status is updating.

Step 6. Restore the healthy fixture and verify recovery: The new healthy Pod should be Ready with no OOM termination or repeated restarts. Saved fault evidence remains in the lab workspace.

Apply the same reasoning: No. Placement uses the request and available allocatable capacity. Increasing a limit does not reduce that request or add a suitable node.

Lab 20 - Trace DNS, Service ports and the application listener

1. Which path does each probe test: DNS, the Service IP, or the Pod directly?

The healthy fixture should resolve the Service name and return HTTP 200 through the name, ClusterIP:80 and PodIP:8080, with a Ready endpoint at 8080. DNS tests naming; direct Pod HTTP bypasses Service routing; Service HTTP includes the Service path. Record actual results for each path.

2. With the wrong namespace name, which requests failed and which still worked?

The missing namespace name should fail resolution and name-based HTTP while the correct ClusterIP and correct name still work. Returning to the correct name restores the request without changing the DNS server or the cluster.

3. With targetPort 8099, why did direct Pod HTTP work while Service HTTP failed?

With targetPort 8099, DNS and direct PodIP:8080 HTTP should still work, while Service traffic goes to an unused endpoint port and fails. A Ready Pod or a nonempty EndpointSlice does not prove that the selected port is correct. Restoring targetPort 8080 should restore Service HTTP.

4. Why did the loopback listener answer locally but fail readiness and remote requests?

A server bound to 127.0.0.1 should answer inside its own Pod but reject access to its Pod IP from the client. Its HTTP readiness probe uses the Pod IP and should fail, making the endpoint unready. Recreating it with a 0.0.0.0 listener should restore readiness, endpoint eligibility and DNS/Service/direct-Pod HTTP. Local success alone does not prove remote reachability.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Load the prepared client commands: Sourcing the helpers targets only Lab 20. The port check succeeds only when a nonempty current slice list uses the requested port.

Step 2. Measure the healthy request path from the client Pod: The correct name resolves, all three HTTP paths should succeed, and the ready endpoint should use port 8080.

Step 3. Request the wrong namespace name, then retry the correct path: The incorrect name should fail while the correct IP and name work, isolating the naming error.

Step 4. Change targetPort to 8099 and compare Service with direct HTTP: DNS and direct Pod HTTP should work, but Service traffic should fail because it forwards to unused port 8099.

Step 5. Restore targetPort to 8080 before the next fault: Restoring 8080 should repair the Service path; allow a short bounded interval for the routing update.

Step 6. Replace only the server bind address and compare local with remote: Local HTTP should succeed while remote Pod HTTP fails and readiness is false. DNS can continue working even though the Service has no ready backend.

Step 7. Restore the listener and check every path again: The 0.0.0.0 listener should restore HTTP from other Pods, a Ready endpoint at 8080 and successful Service requests.

Apply the same reasoning: Not from that evidence. The direct request bypasses the Service selector. Investigate the process listener, destination port and Pod network path first.

Lab 21 - Configuration freshness and a stalled rollout

1. Did changing only the ConfigMap change the running processes' responses, and why?

Changing only the ConfigMap to version-two should leave the existing Pod UIDs and their version-one HTTP responses unchanged. The message entered each process through its startup environment; updating the separate ConfigMap does not refresh that environment or change the Pod template.

2. Which operation made the new configuration take effect, and what proves it?

rollout restart changes the Pod template and replaces consumers. The new UIDs should return version-two after completion. Save the resulting revision before the fault so undo selects the known-good readiness template; the revision identifies a template, not a ConfigMap snapshot.

3. Why did the stalled rollout keep the application available?

With readiness port 9999, the surge Pod should run but remain unready while its process can still answer on 8080. maxUnavailable=0 keeps the two old replicas available; maxSurge=1 limits added replicas. The stalled update should eventually report ProgressDeadlineExceeded. Record actual counts, conditions and restart counts; a readiness failure does not itself imply a crash or automatic rollback.

4. After the undo, what returned to normal and what stayed at version-two?

Undo to the saved revision should restore readiness port 8080 and complete the rollout, while the ConfigMap and consumer responses remain version-two. Explicitly changing the ConfigMap back to version-one and restarting consumers should produce two Ready consumers returning version-one. Verify the ConfigMap, probe port, replica counts and every response separately.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the original configuration and every consumer: The baseline should have two Ready consumers returning version-one. responses queries each current Pod directly on port 8080.

Step 2. Change only the ConfigMap and query the same processes: The ConfigMap should say version-two while unchanged consumers still return version-one; this edit alone does not start a rollout.

Step 3. Restart consumers and save the good revision for undo: New consumers should read version-two. The good probe remains 8080 and the completed rollout should have two Ready current consumers; responses excludes terminating Pods.

Step 4. Break the new readiness port and collect stalled-rollout evidence: The new Pod should be unready at port 9999 while two old replicas remain available. Its process can still answer direct HTTP on 8080. ProgressDeadlineExceeded reports the stall without undoing it.

Step 5. Undo to the saved template and inspect configuration separately: The good readiness port should return to 8080, while the ConfigMap and running responses remain version-two.

Step 6. Restore version-one and verify configuration plus all consumers: The ConfigMap should say version-one, readiness should use 8080, and two Ready current consumers should return version-one after a successful rollout.

Apply the same reasoning: No. That revision still refers to the same ConfigMap key. Newly created containers read its current value; the ConfigMap must be restored separately or configurations versioned under distinct names.

Lab 22 - Orphan reaping and the container process budget

1. Which process removes the zombie entries, and what did each container do with them?

Both containers should report the same image and the same 64-PID budget, and both should inherit 40 orphans after their maker exits. The no-init container's PID 1 is the inline application, which never waits, so those entries should remain as zombies with ppid 1 and keep counting in pids.current. The init container's PID 1 is reaper.py, which should log reaped lines and return the zombie count to zero and pids.current close to its baseline.

2. Why did the burst of 40 fail in the no-init container?

In the no-init container roughly forty of the sixty-four PIDs are already held by zombie entries, so the burst of 40 should create only the remainder before forking fails with EAGAIN, reported as Resource temporarily unavailable. At that moment pids.current should equal pids.max. The live workload is small; the unreaped exit statuses are what consumed the budget.

3. Why did the burst of 200 fail with the same error but no zombies?

With a clean process table, the burst of 40 should create all forty children, because PID 1, the burst parent and forty children stay below sixty-four. The burst of 200 should stop at the ceiling with the same EAGAIN error and pids.current equal to pids.max, but with zero zombies. Identical errors, different causes; reaping cannot help the second one and a larger limit only postpones exhaustion if zombies keep accumulating.

4. What did the pids limit contain, and what did it not repair?

The host can see the container processes, so its count can change. The 64-task ceiling applies to this container's cgroup; it does not impose a 64-task limit on the VM. After the bursts reap their children, pids.current should fall back toward the baseline. Removing both containers should leave no ce-lab22 containers. The limit bounded the blast radius of a fork storm; it did not reap zombies or reduce demand.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Start the no-init container and record its baseline process table: PID 1 should be the application itself, with no zombies and a pids.current far below the 64-PID ceiling.

Step 2. Inherit forty orphans in the no-init container: The maker exits first, so PID 1 inherits the children. Without a wait loop those exited children should remain as zombie entries counted in pids.current.

Step 3. Request forty live processes in the same container: Only the unused part of the budget is available, so the burst should stop early with EAGAIN while pids.current equals pids.max.

Step 4. Start the init container and inherit the same forty orphans: The only changed variable is PID 1. Its wait loop should collect the inherited entries and log them, leaving no zombies.

Step 5. Request forty live processes with a clean process table: The same request should now succeed, because the budget is not held by zombie entries.

Step 6. Request two hundred live processes and read the ceiling: This burst should reach pids.max with no zombies present, producing the same errno from a different cause. The host still sees container processes, but this failure does not mean the VM has reached its own process limit.

Step 7. Remove both containers after recording the evidence: No ce-lab22 containers should remain, and the host count should be close to the value recorded before the bursts.

Apply the same reasoning: No. Zero zombies means nothing is waiting to be reaped, so live processes or threads are holding the budget. Identify what creates them and how many are required before changing

the ceiling; a larger limit only moves the same failure further out.

Lab 23 - Endpoint removal, preStop and the grace budget

1. What does a baseline with zero failures establish before any fault is injected?

The baseline probe should complete with every request successful and both addresses ready, serving and not terminating. That establishes that the client, Service, DNS and both replicas work at this rate, so later failures can be attributed to the injected deletion rather than to the fixture or to capacity.

2. How many requests were unfinished at SIGTERM, and did graceful shutdown complete them?

A nonzero in-flight count shows unfinished requests at SIGTERM. Few or no client failures and an exited cleanly log support that graceful shutdown completed them. Endpoint conditions may show ready=false, serving=true and terminating=true. Neither this count nor zero failures proves the exact order of endpoint withdrawal and signal delivery.

3. With abrupt shutdown, how many requests failed, and how does that match the in-flight count?

Abrupt exit abandons the requests still in flight, so client failures should increase when the deletion overlaps work. Compare the actual counts and timestamps. They need not match exactly: requests may fail during routing changes too, and the client groups results by request start time.

4. What did the preStop hook change, given that the application did not change?

The hook gives routing withdrawal and in-flight work time to finish before the same abrupt handler runs. A lower in-flight count and fewer failures support that explanation. Five seconds is this fixture's delay, not a guarantee that every cluster has stopped routing traffic.

5. With a 1-second budget, when did SIGTERM arrive, and what does that show?

The five-second hook no longer fits inside the configured one-second budget. Compare the observed signal time, any remaining requests and client failures with the previous case. The two-second termination extension and asynchronous processing mean neither an exact one-second signal nor a failure count is guaranteed. Restore the base template and check both ready replicas and successful baseline requests.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Load the helpers and measure the fixture without any fault: The baseline should show two ready and serving addresses, no terminating address and no failed requests at this rate.

Step 2. Delete one replica while the application drains its requests: Compare unfinished requests at SIGTERM, completed requests and the sampled endpoint conditions. A nonzero count alone does not show whether new traffic arrived after deletion.

Step 3. Change only the shutdown behaviour and repeat the same deletion: Only the application's response to the signal changed. The requests it was holding are no longer finished.

Step 4. Add only the preStop hook and repeat again: The hook delays SIGTERM while endpoint withdrawal and request completion proceed. Measure whether five seconds is enough in this run.

Step 5. Keep the hook and shorten only the grace budget: The hook is longer than the whole budget, so it cannot run to completion. Compare when the signal arrived with the previous case.

Step 6. Restore the base template and re-measure without a deletion: The restored fixture should again serve every request from two ready addresses, with no terminating address present.

Apply the same reasoning: No planned application shutdown time remains if the hook consumes all five seconds. Allow time for the hook, application cleanup and timing variation; the small termination extension is not a budget to rely on. Verify with request and shutdown evidence.

Lab 24 - Search paths, ndots and a resolver outage

1. How many candidates did the short name and the absolute form each try?

Both clients should list the same three cluster search entries, with ndots 5 in the default client and ndots 1 in the other. The short Service name has no dots, so its first candidate is web.ce-lab24.svc.cluster.local, which answers immediately; the absolute form skips the search list. Both lookups therefore try one candidate, and their equal counts give you the per-candidate cost, normally two for A and AAAA, to use in the next comparisons.

2. How many queries did the external name cost with ndots 5, a trailing dot, and ndots 1?

The external name has fewer than five dots, so the default client should try every search entry before the name itself. Using your own search list and measured per-candidate cost, the expected count is the number of entries plus one, times that cost. The trailing-dot form and the ndots 1 client should both answer on the first candidate, so each should cost one candidate. Report the counts you measured and their ratio.

3. Why does lowering ndots not reduce the cost of a name that resolves nowhere?

The absent name produces no answer anywhere, so the resolver exhausts every candidate in both Pods. The counts should be close to equal, because ndots changes the order of candidates and not the set that a failed lookup tries. Lowering ndots therefore does not bound the cost of names that never resolve; correcting or removing the name does.

4. With CoreDNS at zero, what still worked, and what proves the resolver recovered?

With no CoreDNS replica the lookups should fail while HTTP to the Service ClusterIP still returns 200 and kubectl keeps working, because the Service data path and the API path do not use the cluster resolver. The query counter should be unavailable during the outage. After restoring the original replica count, the same name should resolve again, HTTP by name should succeed and the counter should increase once more.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read both resolver configurations and the CoreDNS inventory: Both clients should share the same search list and differ only in ndots. The metrics endpoints are addresses, so reading them needs no name resolution.

Step 2. Measure the in-cluster name in both forms: The first candidate for the short name is the namespace-qualified entry, so both forms should answer on their first candidate.

Step 3. Measure an external name three ways: Two dots is fewer than ndots 5, so the default client should try the search list first. The absolute form and the ndots 1 client should reach the name itself immediately.

Step 4. Measure a name that resolves nowhere from both clients: No candidate can answer, so both clients should exhaust the same candidate set; only the order differs.

Step 5. Remove the cluster resolver and compare the three paths: Name resolution should fail while the address path keeps working. A fast connection error and a multi-second timeout are both possible symptoms; record the one you saw.

Step 6. Restore the resolver and prove recovery by name: Ready replicas are not enough on their own; the Service endpoints must be repopulated before the first lookup can succeed, which is what the two bounded waits allow. The short name should then return to its original cost.

Apply the same reasoning: It stays the same. A failed attempt at the absolute name falls back through the search list, so every candidate is tried either way; only the order changes. Reducing ndots helps names that resolve, while absent names need the name corrected, removed or their negative answers cached.

Lab 25 - Admission control as a synchronous dependency

1. Which baseline rejection is the policy working, and which would mean the dependency failed?

The baseline should show failurePolicy Fail, a five-second timeout, the ce-lab25=scoped selector and one ready backend endpoint. The labelled Pod should be created and the unlabelled Pod rejected with the policy message, with one review line logged for each. A rejection quoting the policy is the check working; a rejection naming the webhook call itself would mean the dependency failed.

2. Why did a correctly labelled Pod fail once the backend was gone?

With no backend endpoint the API server cannot complete its admission call, so creating even a correctly labelled Pod should fail with an error naming the webhook and the HTTPS request it could not complete, usually a refused connection to the Service address. The request fails during validating admission, before anything is persisted, so no Pod object exists and the backend logs no review line. The wait is usually short because a Service without endpoints is refused rather than left to time out.

3. Where did the failure surface for the Deployment, and what limited the blast radius?

The Deployment should be accepted, because the rule matches Pods and not Deployments. Its ReplicaSet then fails to create Pods and records FailedCreate events carrying the same webhook error, leaving zero replicas while the Deployment reports a ReplicaFailure condition alongside its usual ones. The Pod created in the unscoped namespace should succeed, because the namespaceSelector never selects that namespace; that scoping is what keeps the rest of the cluster, and the backend's own namespace, usable.

4. What did failurePolicy Ignore allow, and what proves enforcement works again?

Under Ignore the unlabelled Pod should be created although the policy forbids it, which is the cost of choosing availability over enforcement during an outage. After scaling the backend back to one replica, waiting for its endpoint and restoring Fail, the Deployment should converge to two ready replicas, a labelled Pod should be admitted, and a server-side dry run of the unlabelled Pod should be rejected with the policy message again.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the configured dependency and prove enforcement: The labelled Pod should be admitted and the unlabelled Pod rejected by the policy message, with one logged review for each request.

Step 2. Remove the backend and create the same labelled Pod again: Deleting a Pod is unaffected, because the rule matches creation only. The creation should fail during validating admission with an error naming this webhook, and no new review should be logged for it.

Step 3. Compare a controller-driven creation with an unscoped namespace: The Deployment object should be accepted while its ReplicaSet cannot create Pods. The unlabelled namespace is never consulted, so creation there should succeed.

Step 4. Change only the failure policy and repeat the forbidden creation: The backend is still absent; only the meaning of no answer changed. Creation should now succeed without any check being performed.

Step 5. Restore the backend, the Fail policy and enforcement: A ready backend Pod is not proof yet; each node's proxy rules must be rewritten first, which the bounded dry-run loop waits for. The controller should then create its Pods, and the dry run should be rejected without creating anything.

Step 6. Remove the experiment objects and confirm the final state: Only the backend Pod should remain, and the webhook should still be configured with Fail and its namespace selector.

Apply the same reasoning: No. Creating the backend's own Pods would itself require an admission call to the unavailable webhook, so the Deployment would stay at zero replicas. Recovery would need the webhook configuration deleted or switched to Ignore first, which is why a webhook must never select its own namespace.

Lab 26 - Container privilege boundaries and hardening

1. Which boundaries is the default container already relying on?

The default container should run as uid 0 with a bounded nonzero CapEff, Seccomp=2, NoNewPrivs=0, a single visible PID, a short /dev with no block devices, a writable root layer and an allowed chown. It is already relying on a capability set, a syscall filter and a private PID namespace, none of which the image chose.

2. With --pid=host, what changed and what stayed the same?

Adding --pid=host should leave uid, CapEff, Seccomp and the device list identical while the visible process count rises to the VM's processes and pid1_cmdline becomes the VM's init rather than the report program. Visibility and capability are separate boundaries, so this flag grants observation of every process on the machine without adding a single capability.

3. Which three boundaries did --privileged change at once?

The privileged container should show CapEff as the full mask rather than the default subset, Seccomp=0 instead of 2, and a much longer /dev including block devices. One flag therefore removed the capability limit, the syscall filter and the device restriction together, which is why it is almost never the smallest change that fixes a specific failure.

4. Which flag produced each difference in the hardened container?

The hardened container should show uid 10001 from --user, CapEff all zeros from --cap-drop ALL with the chown probe refused with EPERM, NoNewPrivs=1 from no-new-privileges, and a failed write to / from --read-only while /tmp stays writable because of --tmpfs. docker inspect should report the same ReadonlyRootfs, CapDrop and User values, which is the check that a requested control actually took effect.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Run the default container and record its boundaries: The default container should already be limited by a capability set, a seccomp filter and its own PID namespace, without the image asking for any of it.

Step 2. Remove only the PID namespace boundary: Only visibility should change. Capabilities, the seccomp filter and the device list should match the default case exactly.

Step 3. Run the same image privileged: This single flag should change the capability mask, the seccomp filter and the device list at the same time.

Step 4. Run the hardened container and check the controls took effect: Identity, capabilities, privilege gain and root-file-system writes should each change, and the inspected configuration should agree with the measured result.

Step 5. Collect the four results and remove the containers: The four rows should differ only where a flag changed a boundary, and no ce-lab26 container should remain afterwards.

Apply the same reasoning: They widened capabilities, the seccomp filter and device access at once. Instead, reproduce the failure, read the refused operation and the errno, compare CapEff before and after, then grant only the single capability that operation requires and verify it in /proc/self/status.

Lab 27 - Reading a cluster against the CIS benchmark

1. Which controls failed on an untouched cluster, and what observed value makes each a finding?

The baseline should pass the two file-permission controls, the kubelet control and the RBAC control, and fail 1.2.1 and 1.2.21 with the observed value reported as unset. kubeadm does not set those two flags, so the permissive defaults apply. A cluster nobody has touched is therefore not a benchmarked cluster, and the observed value is what tells you which remediation to apply.

2. Which control moved when admin.conf changed mode, and what restored it?

Changing admin.conf to mode 644 should move only 1.1.13, with the observed value reading 644 root:root while 1.1.1 still passes. Restoring 600 should return it to PASS on the next scan. The check reads the file each time, so the control follows the file rather than an earlier result.

3. Why does changing a static Pod manifest restart the API server, and what confirms the new setting was applied?

The kubelet detects the changed static Pod manifest and restarts the API server. The wait checks both /readyz and the profiling argument in the mirror Pod. Its elapsed time is the time until those checks passed, not the duration of an API outage; a timeout is incomplete evidence. The scan should then report 1.2.21 as PASS.

4. What did control 5.1.1 report, and did the final scan match the baseline?

Binding cluster-admin to the test ServiceAccount should fail 5.1.1 and name that subject, because it is not one of the two groups the installer creates. Deleting the binding should return it to PASS, and the final scan should match the baseline exactly, including the two controls that fail by default.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Scan the untouched cluster: The two file controls, the kubelet control and the RBAC control should pass, and the two unset API server flags should fail.

Step 2. Change one file mode and scan again: Only the control covering that file should change, and its observed value should show the new mode.

Step 3. Restore the file mode: The control reads the file on each scan, so restoring the mode should restore the result.

Step 4. Apply the profiling flag and observe control-plane convergence: The kubelet restarts the API server after the manifest changes, so the control should flip only once the new process is running and answering.

Step 5. Revert the flag and confirm the cluster is back: Removing the flag restarts the API server once more, so the control should return to its baseline result with every node still present.

Step 6. Bind cluster-admin to a ServiceAccount, then remove it: The control names any subject beyond the two groups the installer creates, and the final scan should be identical to the baseline.

Apply the same reasoning: Check what the running process was started with. Read the API server's command in its mirror Pod, or its process arguments on the node; a manifest can contain a flag the kubelet has not applied yet, or a component may have failed to restart after the edit.

Lab 28 - Pod Security admission and what a namespace label blocks

1. What could the privileged Pod see before a Pod Security level was enforced?

In this fixture, no Pod Security level blocks the privileged Pod. With hostPID it sees the node's PID namespace, including node processes and the node's PID 1. An unlabelled namespace is not a universal permission grant: cluster-wide admission defaults, RBAC and other policies can still restrict creation.

2. Why did labelling the namespace warn but not stop the running Pod?

Labelling the namespace should return a warning that existing Pods violate the new level and name the violating Pod, while that Pod keeps running with privileged still true. Recreating the same manifest should then be rejected with a Forbidden error naming baseline:latest, host namespaces and the privileged container. Admission applies to creation, so a policy change protects the next Pod rather than the current one.

3. Which fields did restricted require, and how did the hardened Pod satisfy them?

The plain Pod should be rejected under restricted with four named requirements, covering allowPrivilegeEscalation, dropped capabilities, runAsNonRoot and the seccomp profile. The hardened manifest sets exactly those fields and should be created; inside it the process should report UID 10001 and an empty effective capability set, which is the runtime confirmation that the requested context took effect.

4. What did warn allow that enforce rejected?

With enforce removed and warn set, the same plain Pod should produce a warning listing the same violations and then be created, because warn is advisory. Restoring enforce=restricted and deleting that Pod should return the namespace to rejecting it, leaving only the hardened Pod running as the recovery evidence.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Create a privileged Pod in an unlabelled namespace: With no level applied, the Pod should start and observe the node's own processes through the shared PID namespace.

Step 2. Apply the baseline level while that Pod is running: The label command should warn about the existing Pod, and that Pod should keep running unchanged.

Step 3. Recreate the same Pod under the baseline level: The same manifest that was accepted before should now be refused at creation, naming both violations.

Step 4. Move to restricted and compare a plain Pod with a hardened one: The plain manifest should be rejected with a list of required fields, while the manifest that sets them should be admitted and show the requested identity at runtime.

Step 5. Replace enforcement with a warning and repeat the plain Pod: The same violations should be reported, but as an advisory warning followed by the object being created.

Step 6. Restore enforcement and confirm the final state: The namespace should reject the plain Pod again without creating anything while the hardened Pod still runs. Removing the Pod and the label afterwards restores the state this lab started from, so the comparison can be repeated.

Apply the same reasoning: Only that they will be told about violations. warn creates every object it complains about, so the cluster's authority is unchanged; it is a measurement of how much would be rejected if enforce were set, which is a useful step before enforcing but not enforcement.

Lab 29 - What a Pod's ServiceAccount can reach

1. What identity did the default Pod present, and what was it allowed to do?

The default Pod should have the token, ca.crt and namespace files mounted, and its subject should read system:serviceaccount followed by ce-lab29 and default, with an expiry in the future because the kubelet rotates projected tokens. Both requests should return 403 quoting that account, so the Pod is authenticated but not authorized for either Pod-list request tested here; a mounted credential is not a permission.

2. Which of the two requests changed after the RoleBinding, and why only that one?

After the Role and RoleBinding, the request for Pods in its own namespace should return 200 with an item count, while the kube-system request should still return 403 with the same message shape. Only the first changed, because a Role and RoleBinding are namespaced; auth can-i should answer yes for the namespace and no, with a nonzero exit, for kube-system.

3. Without a mounted token, which identity did the API server see?

The Pod with `automountServiceAccountToken` false should have no credential directory at all, so it reports no token and no `ca.crt`, and, because the namespace file came from the same projection, the probe falls back to naming the default namespace in its request. Those requests should be refused as `system:anonymous` rather than as the `ServiceAccount`. The API server was still reachable, so this control removed the identity rather than the connectivity, and it is not affected by the `RoleBinding` that the other Pod benefits from.

4. What changed when the `RoleBinding` was deleted, and what stayed?

Deleting the `RoleBinding` should return the first request to 403 with the original message, while the `Role` object and the token remain. That isolates the binding as the cause of the earlier success and leaves the fixture in its starting state.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read the projected credential and try the API: The credential should be present and name this Pod's account, while authorization for listing Pods has not been granted anywhere.

Step 2. Grant one namespaced Role and repeat the same requests: The grant covers one namespace and one pair of verbs, so only the matching request should change.

Step 3. Run the same probe in the Pod with no mounted token: Without the projection there is no credential to present, so the request should be attributed to an anonymous identity rather than to the `ServiceAccount`.

Step 4. Remove the grant and confirm it was the cause: Removing only the binding should restore the original refusal while the `Role` object and the mounted token stay exactly as they were.

Apply the same reasoning: Yes. Two `RoleBindings` grant exactly those two namespaces and nothing else, while a `ClusterRoleBinding` grants the verb in every namespace, including ones created later. Prefer the narrower pair and verify each with `auth can-i` against both namespaces.

Lab 30 - What a Secret protects, and what it does not

1. What did the API and etcd each show for the same Secret value?

The API returns `.data.password` as base64 and one decode prints the password; `kubectl describe` shows only the key and its size. `etcd` holds the same password as readable bytes beside the key `/registry/secrets/ce-lab30/app-credentials`, because no `encryption-provider-config` is configured on this API server. Encoding and storage are separate from access control, and only the second is changed by encryption at rest.

2. Which surfaces exposed the value inside each container?

The environment consumer exposes the value in its own environment and in `/proc/1/environ`, so every process in that container and anything that can read that file sees it. The file consumer exposes it as a `tmpfs` file whose path is a symlink into a timestamped directory, readable by the container's processes but not present in its environment. Cite the actual values from both Pods.

3. Why could the file consumer read a Secret its ServiceAccount may not get?

The kubelet projects whatever the Pod spec references; RBAC was applied to whoever created the Pod, not to the running workload. `can-i get secrets` should answer no for that `ServiceAccount` while the mounted file still holds the password, so "cannot read Secrets through the API" and "holds no secret material" are different statements.

4. Which surface changed after the rotation, and how long did it take?

The mounted file can update in the existing Pod after kubelet propagates the change; report the observed delay, or state that the expected value was not observed within the wait. The environment variable remains the startup value in the same container. That consumer needs replacement to read the rotated value.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read the Secret through the API: The type is Opaque, .data.password is base64 that decodes to the password, describe prints only the key and its size, and the flag count is 0.

Step 2. Read the same key out of etcd: The stored object contains the password as readable text beside its key name, because nothing encrypts it on the way to etcd.

Step 3. Compare the two consumers inside their containers: The environment consumer reports the value twice and no mounted file; the file consumer reports the file, its mode and its timestamped target and no environment variable. can-i answers no while the file still holds the password.

Step 4. Rotate the Secret and measure which surface follows: The file consumer should show the new value after tens of seconds, while the environment consumer still reports the original value from its running process.

Step 5. Restore the original value and confirm both surfaces: The mounted file returns to the original value after the same kind of delay and both consumers stay Ready; no Pod was replaced during the lab.

Apply the same reasoning: Only the ones that read it as an environment variable, or that read the file once at startup and cache it. A mounted file is refreshed in place, so a consumer that re-reads the file picks up the new value without a restart; the delay is the kubelet's sync period, not zero.

Lab 31 - Default deny, and what a policy does not stop

1. What did each client see after the default-deny ingress policy, and how long did it take?

With the default deny in place both clients should fail with no response, each waiting out the probe's own five-second budget, so elapsed_ms lands near 5000 rather than returning at once. That shape is the evidence that the traffic was dropped: a refusal would have returned in milliseconds. Cite your measured values for both clients.

2. Why did one client succeed and the identical one fail after the allow rule?

The allow rule selects sources by label, not by workload. The client carrying role=allowed matches the ingress from entry and should return HTTP 200 in a few milliseconds, while the unlabelled client, which runs the same image and the same probe in the same namespace, keeps timing out. Policies are additive, so the earlier default deny still covers everything the allow rule does not name.

3. What did the default-deny egress break first, and why that?

The egress policy selects the labelled client and permits nothing, so name resolution fails before any connection is attempted. The resolver is a Pod in kube-system, and its queries are dropped rather than refused, so the lookup fails slowly after every search-list candidate has been tried. The request by IP fails as well, because egress covers that connection too.

4. What changed once egress to kube-dns was allowed?

Allowing UDP and TCP 53 to the kube-dns Pods restores resolution only. The lookup should answer in milliseconds instead of tens of seconds, while the HTTP request to the server still fails: nothing in that rule permits the connection. Report both measurements; the change in the failure's speed is the evidence that the queries now reach CoreDNS.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Measure both clients with no policy in place: Both clients should return HTTP 200 in a few milliseconds and the name should resolve, because nothing restricts this namespace yet.

Step 2. Apply a default deny for all ingress: An empty podSelector with Ingress and no rules should stop both clients, and each should wait out its own five-second budget rather than being refused.

Step 3. Allow only the labelled client: The client carrying role=allowed should return HTTP 200 quickly while the identical unlabelled client keeps timing out; the deny policy is still in force for everything the allow rule does not name.

Step 4. Deny all egress from the allowed client: Resolution from the selected client should fail slowly because every search-list candidate is dropped in turn, and the request by IP should fail as well. The unlabelled client, which this policy does not select, should still resolve in milliseconds.

Step 5. Allow egress to the cluster resolver only: Resolution should answer in milliseconds instead of tens of seconds, while the HTTP request still fails because nothing in that rule permits the connection.

Step 6. Remove every policy and confirm recovery: With no policy left, both clients should return HTTP 200 in a few milliseconds and the name should resolve again, matching the first row.

Apply the same reasoning: Yes, for traffic arriving at those Pods. An ingress policy applies to the destination Pods, so it denies sources in every namespace until a rule names them, and crossing a boundary requires a namespaceSelector. It says nothing about what those Pods may reach, which is what an egress policy covers.

Lab 32 - Grants that reach further than they name

1. What was the builder account refused directly, and what did its own Pod report?

can-i get secrets should answer no for the builder account with a nonzero exit, and the Pod running as that account should report subject=system:serviceaccount:ce-lab32:builder and secrets -> 403, with a message naming the account, the verb and the resource. That is a genuine refusal, and it is the baseline the rest of the lab is compared against.

2. How did a Pod created by that same account list the Secrets?

The builder creates a Pod whose spec sets serviceAccountName to the reader account. Nothing in that request mentions Secrets and the builder still cannot read them, but the kubelet projects the reader's token into the container, so the probe inside it reports that subject and secrets -> 200 with the namespace's Secret names. Permission to create Pods can therefore expose another ServiceAccount's rights when admission allows selecting that account.

3. Which Role was accepted and which was refused, and what did the refusal name?

The Role limited to what the writer already holds should be created. The Role that adds get on secrets should be refused with is attempting to grant RBAC permissions not currently held, and the message should quote the extra rule with its API group, resource and verb. The guard compares the new rules against what the writer holds, so writing Roles is allowed while widening them is not.

4. Which ways of asking about pods/exec disagreed with the real request?

TYPE/NAME syntax makes pods/exec ask about a Pod named exec, so it can answer yes for the builder's Pod-creation grant. The explicit --subresource=exec check and SubjectAccessReview ask about the exec subresource and should deny it. The real exec request should also be denied; record its actual verb and error, which can vary with the client transport.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read what each account is granted: The builder may create Pods and is refused get on Secrets, and its own Pod reports that subject with secrets -> 403 naming the account and the resource.

Step 2. Create a Pod that runs as the reader: The create should be accepted for an account that cannot read Secrets, and the probe inside the new Pod should report the reader subject with secrets -> 200 and the Secret names.

Step 3. Try to widen a Role beyond its own rights: The Role limited to what the writer holds is created; the Role adding secrets is refused as an attempt to grant permissions not currently held, quoting the extra rule.

Step 4. Ask about pods/exec four ways and run the operation: The slashed form answers yes while the subresource form answers no, the review returns false, and the operation is refused with cannot create resource pods/exec.

Step 5. Remove the borrowed Pod and the grants: With the binding removed the reader is refused as well, which shows the binding rather than the account was the grant; reapplying the fixture returns the namespace to its starting state.

Apply the same reasoning: Every credential that any ServiceAccount in that namespace can reach, plus anything mounted into the Pods it may create. Creating a Pod selects an identity and the kubelet supplies its token, so the reachable set is the union of the namespace's accounts, not the creator's own rights.

Lab 33 - When probes cause or repair restarts

1. Why did readiness recover without changing the container identity?

The /tmp/unready marker changes only the readiness result. The same process keeps running and its liveness succeeds. Removing the marker should restore Ready and Service eligibility with the same Pod UID, container ID and restart count; record when the client request succeeds.

2. What proves the progress fault restarted a container inside the same Pod?

The stalled marker makes both the application request and liveness fail. After the failure threshold, kubelet terminates that container. A rising restart count, new container ID within the saved Pod UID, matching Killing/Unhealthy events and eventual HTTP recovery identify the mechanism. The new writable layer has no marker.

3. Why did removing only the startup probe prevent initialization from completing?

The application still needs 15 seconds, but liveness now begins before initialization finishes and exhausts its much shorter budget. Previous logs should show initialization starting without reaching the listening message. Startup protection changes when liveness begins; it does not make initialization faster.

4. Which observations prove the protected configuration recovered?

The intentionally recreated protected Pod should finish startup, become Ready and return HTTP 200. Its UID differs because you deleted the prior Pod; its initial container should have no liveness restarts. Save the previous fault evidence first.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Record the protected baseline: Initialization should finish once; the protected server should be Ready with zero restarts.

Step 2. Fail only readiness: Readiness should fail without restarting the container. Ordinary Service requests should fail once endpoint withdrawal converges.

Step 3. Remove the readiness fault: The same container should return to readiness; retry the observation if Service routing is still converging.

Step 4. Lose progress and observe a repairable restart: The container should restart inside the same Pod and initialize again. The marker disappears with its old writable layer.

Step 5. Remove startup protection while keeping the application unchanged: Liveness should kill the slow initializer before it listens. A new Pod UID came from your explicit replacement, not from liveness.

Step 6. Restore the startup probe and check the client outcome: The protected initializer should finish and serve requests. Saved phase JSON remains in this lab workspace.

Apply the same reasoning: No. Restarting callers leaves the external failure unchanged and repeats their initialization. Choose an observation and response that match a repairable local condition; use readiness only when withholding that caller from traffic is the intended response.

Lab 34 - Volume binding, placement and retained data

1. Why was the first Pending claim normal, and why did the wrong-node consumer remain unscheduled?

The StorageClass delays binding until a consumer exists. The wrong consumer then requires worker2, while the only local PV requires worker. Record Pending/Unschedulable events and both constraints; the mismatch does not show disk exhaustion or missing CPU.

2. Which identities changed when the Pod was replaced, and what happened to the file?

Deleting and recreating store changes its Pod UID. Keeping the PVC retains the binding and its data, so the replacement on the same storage node should read learned-persistence. This is Pod replacement, not replicated recovery on an independent node.

3. What delayed claim deletion, and which action allowed it to finish?

The claim receives a deletion timestamp but retains its protection finalizer while store uses it. Removing store lets protection clear and the claim disappear. The experiment does not manually strip a finalizer or delete an in-use volume.

4. Why did Retain preserve the marker without making the PV Available, and what did recovery change?

Retain keeps the directory and marker, while the PV remains Released with the old claim identity. Deleting that retained PV object and recreating its definition re-registers the same directory; the new PVC can then bind. New API identities do not imply new bytes. The final consumer must read the original marker.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Inspect the available volume and its binding mode: The static volume should be Available and tied to lab34-worker; the class delays binding.

Step 2. Create a claim without a consumer: Pending should describe waiting for a consumer, not a failed application or full disk.

Step 3. Require a node that cannot use the local volume: No node can satisfy both the consumer selector and the only PV topology.

Step 4. Repair placement and write one marker: The correct consumer should bind the claim and mount the directory on worker.

Step 5. Replace only the consumer: A new Pod should read the same data through the unchanged claim.

Step 6. Request claim deletion while it is in use: The in-use claim should remain protected even though deletion was requested.

Step 7. Remove the consumer and inspect retained storage: The claim should disappear, but the Released PV and marker should remain because of Retain.

Step 8. Re-register the retained directory and verify recovery: Re-registering this retained local directory should allow a fresh claim to bind without erasing its data.

Apply the same reasoning: No. More Pods cannot copy or relocate the local directory. Recovery needs the storage node to return, or a separate replication/backup design and an appropriate volume binding; Retain alone supplies neither.

Lab 35 - Namespace budgets and admission failures

1. Why did the unchanged resource-less manifest fail before LimitRange and succeed after it?

Before LimitRange, tenant-b has no CPU/memory requests, which this quota requires. After defaults are installed, the admitted Pod should show 100m CPU and 16 MiB memory requests plus the configured limits. Inspect its actual spec rather than assuming the source manifest was rewritten.

2. Why did the 128 MiB limit fail even though its requests fit the remaining quota?

The oversized Pod requests 100m/16 MiB, which fits beside tenant-a, but its explicit 128 MiB limit exceeds the per-container 64 MiB maximum. The refusal should name that rule, and no oversized Pod should exist. A runtime OOM is not involved.

3. Why was the third 100m Pod refused while RBAC allowed creation?

Two admitted 100m requests total 200m. A third needs 300m in total, exceeding the namespace allowance, despite the applications sleeping. The error should name budget and requests.cpu. An affirmative can-i answers the authorization question only.

4. Did reducing quota evict existing Pods, and which actions allowed a new create to succeed?

The two running Pods should retain their UIDs and Ready state after hard is reduced to 100m while used remains 200m. Restoring hard to 200m and deleting one 100m consumer frees a slot for tenant-c. The earlier rejected request is not queued: the learner must submit it again.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Read the working baseline and authorization: One explicit 100m request should be running within the 200m allowance.

Step 2. Submit a Pod without the required requests: Quota should reject missing requests before creating a Pod. NotFound from get is expected.

Step 3. Add defaults and test an explicit per-container violation: The explicit 128 MiB memory limit should exceed the 64 MiB maximum; its requests alone would fit quota.

Step 4. Submit the same resource-less manifest again: Defaulted resources should allow tenant-b to run and fill the CPU-request allowance.

Step 5. Exceed the aggregate request budget: The third 100m request should exceed 200m total; no tenant-c Pod should be created.

Step 6. Lower the budget below already admitted usage: Existing Pods should remain running; quota reduction changes admission rather than evicting them.

Step 7. Free allowance and retry the rejected request: After one consumer is removed, the repeated request should fit. The previous rejection did not queue a Pod.

Step 8. Restore the original explicit baseline: The original explicit Pod and quota should be restored without any stress load.

Apply the same reasoning: No. Quota is permission to admit aggregate requests, not a reservation on any node. The scheduler still needs an eligible node with sufficient allocatable capacity after existing requests and other placement constraints.

Lab 36 - Secret encryption and key-loss recovery

1. Why did the API show the same value for plaintext and encrypted storage?

The API server transforms stored records on reads and writes. Its authorized response still contains the same base64-encoded data, and secret_value decodes it. Only the separate etcd observation distinguishes the plaintext marker from the encrypted record in this experiment.

2. Why did legacy remain plaintext until the annotation update?

Changing provider configuration affects subsequent writes. identity keeps old plaintext readable, but does not migrate it. Updating the annotation writes the existing object through the first provider; the password remains ce-lab36-demo while its stored prefix changes to the key1 encryption prefix.

3. What proves the missing-key fault changed readability without deleting the stored record?

After the key2 manifest is running, record a live API response and the failed fresh Secret read with its error and nonzero status. stored_secret should still find the key1-prefixed record. An unreadable record is not necessarily absent or corrupt: its matching key is unavailable to this API server.

4. Why did restoring key1 recover the data, and what must real key rotation preserve?

The restored key1 configuration can decrypt the same stored record, so both Secret values return without recreation. Verify their storage prefixes too. Rotation must keep required old keys readable while new writes migrate; backup restoration may still require older keys after live records have moved.

Expected observations by step

Use the matching commands in this lab's question. These are expected patterns, not recorded results.

Step 1. Compare the API value with the original storage record: The toy value should be readable and visible inside the original unencrypted storage record.

Step 2. Enable encryption and inspect the unchanged old record: The old record should still be plaintext: changing configuration has not rewritten it.

Step 3. Write a new Secret through the encrypted API server: The new record should use k8s:enc:aescbc:v1:lab-key1: and hide the toy marker in its stored bytes.

Step 4. Rewrite the legacy record without changing its value: The existing record should now be encrypted with key1 while its password remains unchanged.

Step 5. Omit the key needed to decrypt the existing records: The API process should become live, but the fresh record should fail to decrypt. Record readiness separately; the etcd record should still exist under key1.

Step 6. Restore the original decryption key and verify recovery: The existing encrypted records should become readable again. Keep key1 configured; do not restore a plaintext-only API server.

Apply the same reasoning: No. The API decrypts an authorized read. Encryption at rest addresses access to stored bytes; least-privilege authorization and workload access controls remain necessary. Protecting local key files or using an appropriate KMS is also a separate responsibility.